

УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

=====

Дата генерации: 11.09.2026, 14:32:33
Файлов исходного кода: 85
Суммарный объём кода: 1.14 МВ
Глав/билетов в пособии: 30

=====

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

=====

1. 1. Дерево отрезков с операциями на отрезках [id: seg-seg-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: prefix-sums]
6. Планарность: K5, K3,3 и формула Эйлера [id: planarity]
7. 6. Графы. Представление, DFS, BFS [id: graph-dfs-bfs]
8. 7. Графы. Планарные. Покраска [id: graph-planarity-coloring]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sorting]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 12. Графы. Точки сочленения [id: graph-articulation-points]
14. 13. Графы. Эйлеров цикл [id: graph-euler-cycle]
15. Билет 14. Дейкстра [id: dijkstra]
16. Билет 15. Форд-Беллман [id: bellman-ford]
17. Билет 16. Флойд-Уоршелл [id: floyd]
18. 17. Графы. Ускорения поиска [id: pathfinding-accelerations]
19. 17. Графы. Алгоритм Джонсона (Фокус с перевзвешиванием)
20. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
21. 19. Графы. Остовное дерево. Прима [id: mst-prims]
22. 20. Графы. Остовное дерево. Борувка [id: mst-boruvka]
23. 21. Алгоритм Кнута-Морриса-Пракса (KMP) [id: kmp]
24. 22. Z-функция [id: string-z-function]
25. Билет 23. Алгоритм Ахо-Корасик [id: aho-corasick]

Билеты (учебный контент)

- 24. src/data/tickets/ticket1_5.ts
- 25. src/data/tickets/ticket14_20.ts
- 26. src/data/tickets/ticket21_24.ts
- 27. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

- 28. src/components/ArticulationPointsViz.tsx
- 29. src/components/BellmanFordViz.tsx
- 30. src/components/BfsViz.tsx
- 31. src/components/ChapterImage.tsx
- 32. src/components/ChapterNav.tsx
- 33. src/components/ChapterView.tsx
- 34. src/components/DfsBridgesSimulator.tsx
- 35. src/components/DijkstraViz.tsx
- 36. src/components/DPVisualizer.tsx
- 37. src/components/EulerSimulator.tsx
- 38. src/components/ExpressQuiz.tsx
- 39. src/components/FloydViz.tsx
- 40. src/components/GraphTraversalViz.tsx
- 41. src/components/HeapViz.tsx
- 42. src/components/hints/demoKit.tsx
- 43. src/components/hints/demosExtra.tsx
- 44. src/components/hints/demosGraph.tsx
- 45. src/components/hints/demosStruct.tsx
- 46. src/components/hints/MiniDemo.tsx
- 47. src/components/hints/SimulatorModal.tsx
- 48. src/components/hints/TermPopover.tsx
- 49. src/components/JohnsonViz.tsx
- 50. src/components/KosarajuViz.tsx
- 51. src/components/KruskalSimulator.tsx
- 52. src/components/MnemonicCards.tsx
- 53. src/components/MobileToc.tsx
- 54. src/components/Navbar.tsx
- 55. src/components/PlanarityDemo.tsx
- 56. src/components/PythonCompiler.tsx
- 57. src/components/QueueViz.tsx

РАЗДЕЛ: КОНФИГУРАЦИЯ ПРОЕКТА

ФАЙЛ 1/85: add.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 1224 | РАЗМЕР:

Изменённые файлы

```text

```
A .github/workflows/deploy-pages.yml
M .github/workflows/rebuild-pdf.yml
M index.html
M public/export/full_code_all_pages.pdf
M public/export/full_code_all_pages.txt
A public/favicon.svg
M src/App.tsx
M src/components/Navbar.tsx
A src/components/PythonCompiler.tsx
M vite.config.ts
```

```

`github/workflows/deploy-pages.yml`

Полное содержимое после изменений

```yaml

```
name: Deploy app to GitHub Pages
```

```
on:
```

```
 push:
```

```
 branches:
```

```
 - main
```

```
 # Позволяет проверить Pages прямо из рабочей ветки
```

```
 uses: actions/upload-pages-artifact@v5
 with:
 path: ./dist

 deploy:
 name: Publish to GitHub Pages
 needs: build
 runs-on: ubuntu-latest
 environment:
 name: github-pages
 url: ${ steps.deployment.outputs.page_url }
 steps:
 - name: Deploy
 id: deployment
 uses: actions/deploy-pages@v5

Patch относительно `main`

```diff
diff --git a/.github/workflows/deploy-pages.yml b/.github/workflows/deploy-pages.yml
new file mode 100644
index 00000000..17418e3
--- /dev/null
+++ b/.github/workflows/deploy-pages.yml
@@ -0,0 +1,60 @@
+name: Deploy app to GitHub Pages
+
+on:
+  push:
+    branches:
+      - main
+      # Позволяет проверить Pages прямо из рабочей ветки
+      - "arena/**"
+  workflow_dispatch:
+
+permissions:
+  contents: read
```

```
+   name: Publish to GitHub Pages
+   needs: build
+   runs-on: ubuntu-latest
+   environment:
+     name: github-pages
+     url: ${ steps.deployment.outputs.page_url }
+   steps:
+     - name: Deploy
+       id: deployment
+       uses: actions/deploy-pages@v5
+   ....
```

`github/workflows/rebuild-pdf.yml`

Полное содержимое после изменений

```yaml

name: Rebuild code PDF

# Пайплайн пересборки PDF при каждом коммите:

```
#
исходный код → full_code_all_pages.txt (script)
#
|
|→ page-0001.png ... page-NNNN.png (
#
|
|→ full_code_all_page
```

# Готовые TXT и PDF коммитаются обратно в ветку (их отда  
# промежуточные PNG сохраняются как artifacts запуска.

on:

push:

branches:

- "\*\*"

paths-ignore:

# чтобы коммит самого workflow не запускал бескон

- "public/export/\*\*"

- "\*\*/\*.md"

```
На Ubuntu политика ImageMagick иногда запрещает экспорт
разрешаем то, что нужно пайплайну (text -> pdf)
for policy in /etc/ImageMagick-6/policy.xml /etc/ImageMagick-6/policy.xml.d/*; do
 if [-f "$policy"]; then
 sudo sed -i 's/rights="none" pattern="\{PDF,PS\}" rights="write" pattern=".*" /rights="write" pattern=".*" /' "$policy"
 fi
done
convert -version
```

- name: Install dependencies  
run: npm ci --no-audit --no-fund
- name: Step 1 – код → txt  
run: npm run export:txt
- name: Step 2 – txt → png  
env:  
 EXPORT\_DENSITY: \${GITHUB\_EVENT\_INPUTS.density:-300}  
run: npm run export:png
- name: Step 3 – png → pdf  
run: npm run export:pdf
- name: Type-check приложения  
run: npx tsc --noEmit  
continue-on-error: true
- name: Summary  
run: |  
 {  
 echo "### Экспорт пересобран"  
 echo ""  
 echo "| Артефакт | Размер |"  
 echo "| --- | --- |"  
 echo "| \`\${GITHUB\_WORKSPACE}/public/export/full\_code\_all\_pages\` | \$(du -sh "\${GITHUB\_WORKSPACE}/public/export/full\_code\_all\_pages" | cut -f1) |"  
 echo "| \`\${GITHUB\_WORKSPACE}/public/export/full\_code\_all\_pages\` | \$(du -sh "\${GITHUB\_WORKSPACE}/public/export/full\_code\_all\_pages" | cut -f1) |"  
 echo "| PNG-страниц | \$(ls tmp/export-pages | wc -l) |"  
 } >> "\$GITHUB\_STEP\_SUMMARY"

```
+++ b/.github/workflows/rebuild-pdf.yml
```

```
@@ -42,15 +42,15 @@ jobs:
```

```
 steps:
```

```
 - name: Checkout
```

```
- uses: actions/checkout@v4
```

```
+ uses: actions/checkout@v6
```

```
 with:
```

```
 fetch-depth: 0
```

```
 persist-credentials: true
```

```
 - name: Setup Node.js
```

```
- uses: actions/setup-node@v4
```

```
+ uses: actions/setup-node@v7
```

```
 with:
```

```
- node-version: "22"
```

```
+ node-version: "24"
```

```
 cache: npm
```

```
 - name: Install ImageMagick + DejaVu fonts
```

```
@@ -97,7 +97,7 @@ jobs:
```

```
 } >> "$GITHUB_STEP_SUMMARY"
```

```
 - name: Upload artifacts (PDF + TXT)
```

```
- uses: actions/upload-artifact@v4
```

```
+ uses: actions/upload-artifact@v7
```

```
 with:
```

```
 name: full-code-export
```

```
 path: |
```

```
@@ -107,7 +107,7 @@ jobs:
```

```
 retention-days: 30
```

```
 - name: Upload artifacts (PNG-страницы)
```

```
- uses: actions/upload-artifact@v4
```

```
+ uses: actions/upload-artifact@v7
```

```
 with:
```

```
 name: full-code-pages-png
```

```
 path: tmp/export-pages/*.png
```

## `public/favicon.svg`

### Полное содержимое после изменений

````xml

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64"
  <rect width="64" height="64" rx="16" fill="#4f46e5"/>
  <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#4f46e5"/>
  <circle cx="47" cy="21" r="4" fill="#34d399"/>
</svg>
```

````

### Patch относительно `main`

````diff

```
diff --git a/public/favicon.svg b/public/favicon.svg
new file mode 100644
index 00000000..3bbbba0
```

```
--- /dev/null
```

```
+++ b/public/favicon.svg
```

```
@@ -0,0 +1,5 @@
```

```
+<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64"
+  <rect width="64" height="64" rx="16" fill="#4f46e5"/>
+  <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#4f46e5"/>
+  <circle cx="47" cy="21" r="4" fill="#34d399"/>
+</svg>
```

````

## `src/App.tsx`

### Полное содержимое после изменений

````tsx

```
import { useCallback, useEffect, useMemo, useState } from "react";
import { chapters } from "../data/content";
import { Navbar } from "../components/Navbar";
import { Sidebar } from "../components/Sidebar";
```

```
const progress = useMemo(() => ((index + 1) / chapters
```

```
return (
```

```
  <div className="min-h-screen bg-slate-950 text-slate-50">
```

```
    <Navbar activeTab={activeTab} setActiveTab={setActiveTab}>
```

```
      {activeTab === "guide" ? (
```

```
        <>
```

```
          <MobileToc
```

```
            selectedChapterId={activeChapterId}
```

```
            setSelectedChapterId={goTo}
```

```
            currentTitle={activeChapter.title}
```

```
            index={index}
```

```
            total={chapters.length}
```

```
          />
```

```
        <div className="flex flex-col lg:flex-row max-w-7xl">
```

```
          {/* Сайдбар только на десктопе – на мобильном скрывается */}
```

```
          <div className="hidden lg:block lg:w-80 shrink-0">
```

```
            <Sidebar selectedChapterId={activeChapterId}>
```

```
          </div>
```

```
          <main id="main-content" className="flex-1 min-w-0">
```

```
            {/* Шапка главы с быстрыми переходами */}
```

```
            <div className="flex items-center justify-between">
```

```
              <div className="min-w-0">
```

```
                <div className="text-[10px] uppercase">
```

```
                  {activeChapter.category || "Универсальное пособие"}>
```

```
                </div>
```

```
                <h2 className="text-base sm:text-xl font-medium">
```

```
                  {activeChapter.title}>
```

```
                </h2>
```

```
              </div>
```

```
              <ChapterNav prev={prev} next={next} index={index}>
```

```
            </div>
```

```
          <div className="h-1 w-full bg-slate-800 rounded">
```

```
            <div
```

```
    });
  }
  ....
```

Patch относительно `main`

```
````diff
```

```
diff --git a/src/App.tsx b/src/App.tsx
```

```
index 2091f33..4583756 100644
```

```
--- a/src/App.tsx
```

```
+++ b/src/App.tsx
```

```
@@ -7,9 +7,10 @@ import { ChapterView } from "../components/ChapterView";
```

```
import { ChapterNav } from "../components/ChapterNav";
```

```
import { SimulatorModal } from "../components/hints/SimulatorModal";
```

```
import { SimulatorHub } from "../components/SimulatorHub";
```

```
+import { PythonCompiler } from "../components/PythonCompiler";
```

```
export default function App() {
```

```
- const [activeTab, setActiveTab] = useState<"guide" | "simulator">("guide");
```

```
+ const [activeTab, setActiveTab] = useState<"guide" | "simulator">("guide");
```

```
 const [activeChapterId, setActiveChapterId] = useState<string>("");
```

```
 const [modalVizId, setModalVizId] = useState<string>("");
```

```
@@ -95,7 +96,7 @@ export default function App() {
```

```
 </main>
```

```
 </div>
```

```
 </>
```

```
-) : (
```

```
+) : activeTab === "simulator" ? (
```

```
 <main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8">
```

```
 <SimulatorHub
```

```
 onOpen={setModalVizId}
```

```
@@ -105,6 +106,8 @@ export default function App() {
```

```
 </>
```

```
 </>
```

```
 </main>
```

```
+) : (
```

```
+ <PythonCompiler />
```

```

 k-0">
 <Sparkles className="w-6 h-6" />
 </div>
 <div className="min-w-0 flex-1">
 <h1 className="text-xl font-extrabold text-white">
 Универсальное пособие
 </h1>
 <p className="text-xs text-indigo-400 font-extrabold">
 Подготовка к экзаменам: Алгоритмы, Структуры
 </p>
 </div>
</div>

<div className="flex items-center w-full lg:w-auto">
 >
 <button
 id="tab-guide-btn"
 onClick={() => setActiveTab('guide')}
 className={`flex-1 lg:flex-none flex items-center justify-center text-white font-extrabold cursor-pointer transition-200 whitespace-nowrap ${
 activeTab === 'guide'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <BookOpen className="w-4 h-4" /> Учебное пособие
 </button>
 <button
 id="tab-simulator-btn"
 onClick={() => setActiveTab('simulator')}
 className={`flex-1 lg:flex-none flex items-center justify-center text-white font-extrabold cursor-pointer transition-200 whitespace-nowrap ${
 activeTab === 'simulator'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <Cpu className="w-4 h-4" /> Тренажёры

```

```
 id="download-html-btn"
 onClick={saveBook}
 disabled={busy}
 className="flex items-center justify-center
er:bg-amber-600/20 border border-amber-500/
 title="Скачать всё пособие одним автономным
 >
 {busy ? <Loader2 className="w-4 h-4 animate
 </button>
 </div>
 </div>
</header>
);
};
},
```

### Patch относительно `main`

```
```diff
diff --git a/src/components/Navbar.tsx b/src/components
index a528778..6f8cd36 100644
--- a/src/components/Navbar.tsx
+++ b/src/components/Navbar.tsx
@@ -1,11 +1,11 @@
  import React, { useState } from 'react';
- import { BookOpen, Cpu, Sparkles, FileDown, FileText,
+ import { BookOpen, Cpu, Sparkles, FileDown, FileText,
  import { chapters } from '../data/content';
  import { downloadBookHtml } from '../utils/exportHtml'

  interface NavbarProps {
-   activeTab: 'guide' | 'simulator';
-   setActiveTab: (tab: 'guide' | 'simulator') => void;
+   activeTab: 'guide' | 'simulator' | 'compiler';
+   setActiveTab: (tab: 'guide' | 'simulator' | 'compile
  }

  export const Navbar: React.FC<NavbarProps> = ({ active
```

```
-----
````tsx
import { useCallback, useState } from "react";
import { CheckCircle2, ExternalLink, Info, Loader2, Play } from "lucide-react";

const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/pyodide.js`;
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/index.html`;

const STARTER_CODE = `# Первый запуск загрузит Python в браузер
name = "алгоритмы"

for number in range(1, 4):
 print(f"{number}. Учим {name}!")
`;

type PyodideRuntime = {
 runPythonAsync: (source: string) => Promise<unknown>;
 setStdout: (options: { batched: (message: string) => void }) => void;
 setStderr: (options: { batched: (message: string) => void }) => void;
 setStdin: (options: { stdin: () => string | number | null }) => void;
};

type PyodideModule = {
 loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
};

let runtimePromise: Promise<PyodideRuntime> | null = null;

function getPythonRuntime() {
 if (!runtimePromise) {
 runtimePromise = import(/* @vite-ignore */ PYODIDE_INDEX_URL)
 .then((module as PyodideModule).loadPyodide({ indexURL: PYODIDE_MODULE_URL }));
 }
 return runtimePromise;
}

function errorText(error: unknown) {
 if (error instanceof Error) {
 return error.message;
 }
 return String(error);
}
```

```

 setRuntimeMessage("Не удалось выполнить код — проп
 } finally {
 setRunning(false);
 }
}, [code, stdin, running, runtimeReady]);

const reset = () => {
 setCode(STARTER_CODE);
 setStdin("");
 setOutput("Нажмите «Запустить», чтобы увидеть резул
};

return (
 <main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
 <div className="max-w-6xl mx-auto">
 <div className="flex flex-col sm:flex-row sm:items-center">
 <div>
 <div className="inline-flex items-center gap-2">
 <Terminal className="w-4 h-4" /> Боковая панель
 </div>
 <h2 className="text-2xl sm:text-3xl font-extrabold">
 <p className="text-slate-400 mt-2 max-w-2xl">
 Пишите небольшой код и запускайте его прямо в браузере,
 который переводит CPython в WebAssembly.
 </p>
 </div>
 <a
 href="https://pyodide.org/"
 target="_blank"
 rel="noreferrer"
 className="inline-flex items-center gap-1.5"
 >
 Как это работает <ExternalLink className="w-4 h-4" />

 </div>
 </div>
 <div className="grid xl:grid-cols-[minmax(0,1.33fr),minmax(0,1.33fr)]>
 <section className="rounded-2xl border border-slate-200 p-4">

```

```

 aria-label="Код Python"
 className="block w-full min-h-[360px] res
 -none focus:ring-2 focus:ring-inset focus:r
 placeholder="Напишите Python-код..."
 />

<div className="border-t border-slate-800 b
 <label className="block px-4 pt-3 text-[1
 Ввод для input() • по одной строке на к
 </label>
 <textarea
 id="python-stdin"
 value={stdin}
 onChange={(event) => setStdin(event.targ
 spellCheck={false}
 rows={2}
 placeholder={'Например:\nАлиса'}
 className="block w-full resize-y bg-tra
 eholder:text-slate-700"
 />
</div>
</section>

<section className="rounded-2xl border border
 <div className="flex items-center justify-b
 <div className="flex items-center gap-2 t
 <Terminal className="w-4 h-4 text-emera
 </div>
 <span className={`inline-flex items-cente
 {runtimeReady && <CheckCircle2 classNam
 {runtimeReady ? "готов" : "не загружен"

 </div>
 <pre className="min-h-[360px] max-h-[560px]
 x] leading-6 text-slate-300">
 {output}
 </pre>
 <div className="border-t border-slate-800 p

```

```
+
+const STARTER_CODE = `# Первый запуск загрузит Python
+name = "алгоритмы"
+
+for number in range(1, 4):
+ print(f"{number}. Учим {name}!")
+`;
+
+type PyodideRuntime = {
+ runPythonAsync: (source: string) => Promise<unknown>
+ setStdout: (options: { batched: (message: string) =>
+ setStderr: (options: { batched: (message: string) =>
+ setStdin: (options: { stdin: () => string | number |
+});
+
+type PyodideModule = {
+ loadPyodide: (options: { indexURL: string }) => Prom
+};
+
+let runtimePromise: Promise<PyodideRuntime> | null = null
+
+function getPythonRuntime() {
+ if (!runtimePromise) {
+ runtimePromise = import(/* @vite-ignore */ PYODIDE
+ (module as PyodideModule).loadPyodide({ indexURL
+ });
+ }
+ return runtimePromise;
+}
+
+function errorText(error: unknown) {
+ if (error instanceof Error) return error.message;
+ return String(error);
+}
+
+export function PythonCompiler() {
+ const [code, setCode] = useState(STARTER_CODE);
+ const [stdin, setStdin] = useState("");
+}
```



```

+ setCode(STARTER_CODE);
+ setStdin("");
+ setOutput("Нажмите «Запустить», чтобы увидеть резу
+ });
+
+ return (
+ <main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
+ <div className="max-w-6xl mx-auto">
+ <div className="flex flex-col sm:flex-row sm:items-center">
+ <div>
+ <div className="inline-flex items-center gap-2">
+ <Terminal className="w-4 h-4" /> Боковая панель
+ </div>
+ <h2 className="text-2xl sm:text-3xl font-extrabold">
+ <p className="text-slate-400 mt-2 max-w-2xl">
+ Пишите небольшой код и запускайте его прямо в браузере,
+ который переводит CPython в WebAssembly.
+ </p>
+ </div>
+ <a
+ href="https://pyodide.org/"
+ target="_blank"
+ rel="noreferrer"
+ className="inline-flex items-center gap-1.5"
+ >
+ Как это работает <ExternalLink className="text-slate-400">
+
+ </div>
+
+ <div className="grid xl:grid-cols-[minmax(0,1fr)minmax(0,1fr)]">
+ <section className="rounded-2xl border border-slate-200 p-4">
+ <div className="flex flex-wrap items-center gap-2">
+ <div className="flex items-center gap-2">
+
+
+
+ </div>
+ <div className="flex items-center gap-2">

```

```

+ <label className="block px-4 pt-3 text-[
+ Ввод для input() • по одной строке на
+ </label>
+ <textarea
+ id="python-stdin"
+ value={stdin}
+ onChange={(event) => setStdin(event.ta
+ spellCheck={false}
+ rows={2}
+ placeholder={'Например:\nАлиса'}
+ className="block w-full resize-y bg-tr
ceholder:text-slate-700"
+ />
+ </div>
+ </section>
+
+ <section className="rounded-2xl border borde
+ <div className="flex items-center justify-l
+ <div className="flex items-center gap-2
+ <Terminal className="w-4 h-4 text-emerg
+ </div>
+ <span className={`inline-flex items-centr
+ {runtimeReady && <CheckCircle2 classNa
+ {runtimeReady ? "готов" : "не загружен
+
+ </div>
+ <pre className="min-h-[360px] max-h-[560px
px] leading-6 text-slate-300">
+ {output}
+ </pre>
+ <div className="border-t border-slate-800
+ </section>
+ </div>
+
+ <div className="mt-5 grid md:grid-cols-3 gap-3
+ <div className="flex gap-2 rounded-xl border
+ <Info className="w-4 h-4 shrink-0 text-ind
+ Ctrl или Cmd + Enter запускает код.

```

```

 alias: {
 "@": path.resolve(__dirname, "src"),
 },
 },
 server: {
 host: "0.0.0.0",
 port: 5173,
 strictPort: true,
 // предпросмотр может открываться через внешний про
 allowedHosts: true,
 },
 preview: {
 host: "0.0.0.0",
 port: 4173,
 strictPort: true,
 allowedHosts: true,
 },
});
\`\`\`

```

### Patch относительно `main`

```

\`\`\`diff
diff --git a/vite.config.ts b/vite.config.ts
index c0456e7..e19d945 100644
--- a/vite.config.ts
+++ b/vite.config.ts
@@ -10,6 +10,9 @@ const __dirname = path.dirname(__file)

// https://vite.dev/config/
export default defineConfig({
+ // На GitHub Pages приложение живёт в /algv0/, локал
+ // VITE_BASE можно переопределить, если репозиторий
+ base: process.env.VITE_BASE || "/",
 plugins: [react(), tailwindcss(), viteSingleFile()],
 resolve: {
 alias: {
\`\`\`

```

3. **\*\*Строгая структура контента:\*\***

- Заголовок с цветным бейджем (например, `bg-indigo-100`)
- Определение/правило в центре (`border-blue-500` или `border-blue-500/2px`)
- Обязательная сетка аналогий (2 колонки: неформальный текст и код)
- "Душные" детали (код, формулы, сложные доказательства)

4. **\*\*Не ломай верстку:\*\*** Не придумывай свои CSS-классы. Используй только классы в файлах `src/data/content.ts`. Не используй чистый markdown.

---

ФАЙЛ 4/85: index.html

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 14 | РАЗМЕР: 600 Б

---

```
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100">
 <head>
 <meta charset="UTF-8" />
 <meta name="viewport" content="width=device-width, height=device-height, initial-scale=1.0" />
 <link rel="icon" type="image/svg+xml" href="%BASE_URL%/icon.svg" />
 <title> Дискретка для тупых – Интерактивный гид по дискретке
 </head>
 <body class="bg-slate-950 text-slate-100 min-h-screen">
 <div id="root"></div>
 <script type="module" src="/src/main.tsx"></script>
 </body>
</html>
```

ФАЙЛ 5/85: metadata.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 8 | РАЗМЕР: 310 Б

---

```
{
 "name": "Remix Remix: ExamPrep Reader",
 "description": "Удобная web-читалка для подготовки к экзаменам",
 "requestFramePermissions": [],
 "scripts": {
 "dev": "vite dev",
 "build": "vite build",
 "preview": "vite preview"
 }
}
```

## Универсальное пособие • полный исходный код

```

 "@types/node": "22.19.17",
 "@types/pdfkit": "^0.17.6",
 "@types/react": "19.2.7",
 "@types/react-dom": "19.2.3",
 "@vitejs/plugin-react": "5.1.1",
 "esbuild": "^0.28.2",
 "tailwindcss": "4.1.17",
 "typescript": "5.9.3",
 "vite": "7.3.2",
 "vite-plugin-singlefile": "2.3.0"
},
 "description": "Интерактивное пособие по алгоритмам и
}
```

-----  
ФАЙЛ 7/85: README.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 115 | РАЗМЕР: 1.5 КБ

-----  
# algv0 – Универсальное пособие по алгоритмам и структурам данных

Интерактивная web-читалка (React + Vite + Tailwind) с подсветкой кода  
и автоматическим экспортом всего исходного кода в PDF.

## Быстрый старт

```
```bash  
npm install  
npm run dev          # предпросмотр на http://0.0.0.0:5174  
npm run build        # сборка в dist/ (single-file)  
```
```

## Экспорт кода: код → txt → png → pdf

Пайплайн собирает **весь** исходный код репозитория в один файл

| Команда | Шаг | Результат |
|---------|-----|-----------|
|---------|-----|-----------|

## Универсальное пособие • полный исходный код

- на телефоне – липкая строка «Содержание · N из M» и в
- \* **\*\*Переходы между темами\*\*** – кнопки «Назад / Вперёд» в «Предыдущая / Следующая тема» под текстом; на клавиатуре
- \* **\*\*Всплывающие подсказки по терминам\*\*** – текст главы и известные термины (BFS, куча, бор, суффиксная ссылка, подчёркиваются, а по наведению/тапу показывается карта мини-анимацией и кнопкой «Показать симулятор» (симулятор Словарь – `src/data/glossary.ts`, мини-анимации – `src/`
- \* **\*\*Реестр интерактивов\*\*** – `src/components/vizRegistry` и для панели под главой, и для модалки из подсказки.

### ## Структура

...

src/

- App.tsx – оболочка, привязка глав к визуализации
- components/ – интерактивные симуляторы (Действия)
- data/ – контент пособия (главы/билеты и т.д.)
- scripts/export/ – пайплайн код → txt → png → pdf
- public/export/ – сгенерированные TXT и PDF (обновляемые)
- .github/workflows/ – автоматизация

...

---

### # Структура приложения и парсинг элементов

Если вы хотите парсить HTML конспект внешними инструментами, используйте теги и классы.

### ## Заголовки

- \* Заголовки секций: Используют стандартный тег `

## ` и
- \* Подзаголовки: Используют тег `

### `.

### ## Текст и параграфы

- \* Обычный текст содержится в тегах `

`.
- \* Блоки "Шиза" (Аналогии) и другая текстовая инфографика: параграфы `

`.

Универсальное пособие • полный исходный код

>  
> Всего глав в пособии: **\*\*25\*\***.

## Уровень 1. Нет вообще ничего: ни аналогии, ни 2D

| Глава              | id        | Что делать           |
|--------------------|-----------|----------------------|
| Алгоритмы на карте | `alg-map` | Страница-заглушка (8 |

## Уровень 2. Темы, которых в пособии НЕТ ВООБЩЕ (дыры)

Это самая большая проблема: визуализаторы для них написаны, но соответствующих глав в `src/data/content.ts` нет — и

| Билет         | Тема                           | Статус                                |
|---------------|--------------------------------|---------------------------------------|
| <b>**1**</b>  | Дерево отрезков (Segment Tree) | Главы нет. Книжка читается вхолостую. |
| <b>**7**</b>  | Планарность и раскраска графов | Номерной главы нет; аналогии          |
| <b>**11**</b> | Мосты в графах                 | Номерной главы нет; есть без аналогии |
| <b>**13**</b> | Эйлеровы пути и циклы          | Номерной главы нет; есть без аналогии |

Дополнительно — «осиротевшие» визуализаторы без глав (книжки)

- \* `stack-dfs` → `StackViz.tsx` — **\*\*Стек и DFS\*\***
- \* `queue-bfs` → `QueueViz.tsx` + `BfsViz.tsx` — **\*\*Очередь и BFS\*\***
- \* `heap-beam-search` → `HeapViz.tsx` — **\*\*Куча и лучевой поиск\*\***
- \* `segment-trees` → `SegmentTreeVisualizer.tsx` — **\*\*Дерево отрезков\*\***
- \* `dynamic-programming` → `DPVisualizer.tsx` — **\*\*Динамическое программирование\*\***

## Уровень 3. Есть 2D, но НЕТ бытовой аналогии («суха

|    |                                          |   |   |   |
|----|------------------------------------------|---|---|---|
| 12 | 12. Графы. Точки сочленения              | 1 | - | - |
| 14 | 14. Графы. Кратчайший путь. Дейкстра     | 3 | - | - |
| 15 | 15. Графы. Кратчайший путь. Форд–Беллман | 1 | - | - |
| 16 | 16. Графы. Кратчайший путь. Флойд        | - | - | - |
| 17 | 17. Графы. Алгоритм Джонсона             | - | - | - |
| 18 | 18. Графы. Остовное дерево. Краскал      | 1 | - | - |
| 19 | 19. Графы. Остовное дерево. Прима        | 1 | - | - |
| 20 | 20. Графы. Остовное дерево. Борувка      | 1 | - | - |
| 21 | 21. Префикс-функция (π), КМП             | 4 | - | - |
| 22 | 22. Z-функция                            | 4 | - | - |
| 23 | 23. Алгоритм Ахо–Корасик                 | 2 | - | - |
| 24 | 24. Классы сложности, сведение задач     | 4 | - | - |
| -  | Обзор: Кратчайшие пути и Графы           | - | - | - |
| -  | Алгоритмы на карте                       | - | - | - |
| -  | Эйлер: Путь ≠ Цикл                       | - | - | - |
| -  | Мосты в графах: код + визуализация       | 1 | - | - |

## ## Рекомендуемый порядок работ

1. **\*\*Вернуть потерянные билеты 1, 7, 11, 13\*\*** и подключить ``segment-trees``, ``stack-dfs``, ``queue-bfs``, ``heap-be`` — это 5 готовых компонентов, которые сейчас просто не
2. **\*\*Дописать аналогии\*\*** в 5 главах из «Уровня 3» (по ш
3. **\*\*Добавить 2D\*\*** к 4 главам «Уровня 4» — минимум `inli`
4. **\*\*Решить судьбу `alg-mar`\*\*** — раскрыть или удалить.

ФАЙЛ 9/85: tsconfig.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 32 | РАЗМЕР: 6

```
{
 "compilerOptions": {
 "target": "ES2020",
 "useDefineForClassFields": true,
```



```
import { defineConfig } from "vite";
import { viteSingleFile } from "vite-plugin-singlefile"

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
 // На GitHub Pages приложение живёт в /algv0/, локально
 // VITE_BASE можно переопределить, если репозиторий б
 base: process.env.VITE_BASE || "/",
 plugins: [react(), tailwindcss(), viteSingleFile()],
 resolve: {
 alias: {
 "@": path.resolve(__dirname, "src"),
 },
 },
 server: {
 host: "0.0.0.0",
 port: 5173,
 strictPort: true,
 // предпросмотр может открываться через внешний про
 allowedHosts: true,
 },
 preview: {
 host: "0.0.0.0",
 port: 4173,
 strictPort: true,
 allowedHosts: true,
 },
});
```

```

 if (e.metaKey || e.ctrlKey || e.altKey) return;
 if (e.key === "ArrowRight" && next) goTo(next.id);
 if (e.key === "ArrowLeft" && prev) goTo(prev.id);
 };
 window.addEventListener("keydown", onKey);
 return () => window.removeEventListener("keydown", onKey);
}, [goTo, next, prev]);

```

```
const progress = useMemo(() => ((index + 1) / chapter
```

[illegible]

```
 <PythonCompiler
 chapterId={activeChapter.id}
 chapterTitle={activeChapter.title}
 onOpenGuide={() => setActiveTab("guide")}
 />
 }}

 <SimulatorModal vizId={modalVizId} onClose={() =>

 <footer className="p-8 text-center text-slate-600"
 © 2026 Universal Educational Guide. Интерактивн
 </footer>
 </div>
);
}
```

---

ФАЙЛ 12/85: src/hooks/useTermHints.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 124 | РАЗМЕР: 5.0 КБ

---

```
import { useEffect } from "react";
import { GLOSSARY_LABELS } from "../data/glossary";

/**
 * Ограничители, чтобы текст не рябил, но и чтобы подсказки
 *
 * Считаем не «сколько раз встретился термин», а «сколько
 * написание». Иначе в главе про splay первые же «Splay
 * лимит, и слова «Zig-Zag», «вращений», «AVL-дереве» о
 */
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const SKIP_SELECTOR = "code, pre, script, style, a, tex

const escapeRe = (s: string) => s.replace(/[/\.*\+\?\^\$\{\}\(\)|\
```

```

for (const textNode of targets) {
 const text = textNode.nodeValue ?? "";
 re.lastIndex = 0;
 if (!re.test(text)) continue;
 re.lastIndex = 0;

 const frag = document.createDocumentFragment();
 let last = 0;
 let m: RegExpExecArray | null;

 while ((m = re.exec(text)) !== null) {
 const surface = m[1].toLowerCase();
 const id = labelToId.get(surface);
 if (!id) continue;

 const usedTerm = perTerm.get(id) ?? 0;
 const usedSurface = perSurface.get(surface) ?? 0;
 if (usedTerm >= MAX_PER_TERM || usedSurface >= MAX_PER_SURFACE) {
 perTerm.set(id, usedTerm + 1);
 perSurface.set(surface, usedSurface + 1);
 }

 if (m.index > last) frag.appendChild(document.createElement("span"));
 const span = document.createElement("span");
 span.className = "term-hint";
 span.dataset.termId = id;
 span.tabIndex = 0;
 span.setAttribute("role", "button");
 span.setAttribute("aria-label", `Подсказка: ${m[1]}`);
 span.textContent = m[1];
 frag.appendChild(span);
 last = m.index + m[1].length;
 }

 if (last === 0) continue;
 if (last < text.length) frag.appendChild(document.createElement("span"));
 textNode.parentNode?.replaceChild(frag, textNode);
}

```

```
@import "tailwindcss";

@layer utilities {
 .neon-glow-blue {
 box-shadow: 0 0 15px rgba(59, 130, 246, 0.5);
 }
 .neon-glow-emerald {
 box-shadow: 0 0 15px rgba(16, 185, 129, 0.5);
 }
 .neon-glow-rose {
 box-shadow: 0 0 15px rgba(244, 63, 94, 0.5);
 }
 .neon-glow-yellow {
 box-shadow: 0 0 20px rgba(234, 179, 8, 0.8);
 }
 .no-scrollbar::-webkit-scrollbar {
 display: none;
 }
 .no-scrollbar {
 scrollbar-width: none;
 }
}

@keyframes pulse-gold {
 0%,
 100% {
 stroke: #eab308;
 stroke-width: 5;
 filter: drop-shadow(0 0 2px rgba(234, 179, 8, 0.5))
 }
 50% {
 stroke: #fef08a;
 stroke-width: 8;
 filter: drop-shadow(0 0 12px rgba(234, 179, 8, 0.9))
 }
}

.animate-pulse-gold {
```

```
.term-hint:hover,
.term-hint:focus-visible {
 background: rgba(99, 102, 241, 0.22);
 color: #fff;
 outline: none;
}
@media (hover: none) {
 .term-hint {
 border-bottom-style: solid;
 background: rgba(99, 102, 241, 0.12);
 }
}
```

/\* Контент главы приходит готовым HTML — чиним его под

```
.chapter-body {
 max-width: 100%;
 overflow-wrap: anywhere;
}

.chapter-body :where(section, div, p, li) {
 min-width: 0;
}
```

```
.chapter-body :where(pre, code) {
 white-space: pre-wrap;
 word-break: break-word;
}
```

```
.chapter-body pre {
 overflow-x: auto;
 max-width: 100%;
 font-size: clamp(11px, 2.9vw, 13px);
}
```

```
.chapter-body table {
 display: block;
 width: 100%;
 overflow-x: auto;
```

```
 height: 8px;
 }
 ::-webkit-scrollbar-track {
 background: #0f172a;
 }
 ::-webkit-scrollbar-thumb {
 background: #334155;
 border-radius: 4px;
 }
 ::-webkit-scrollbar-thumb:hover {
 background: #475569;
 }
```

/\* Индикатор длительности кадра в пошаговых визуализаци

```
@keyframes demoProgress {
 from {
 width: 0%;
 }
 to {
 width: 100%;
 }
}
```

---

ФАЙЛ 14/85: src/main.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 11 | РАЗМЕР: 230 В

---

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "./index.css";
import App from "./App";

createRoot(document.getElementById("root")!).render(
 <StrictMode>
 <App />
 </StrictMode>
```

```
const src: string;
export default src;
}
```

## РАЗДЕЛ: КОНТЕНТ И ДАННЫЕ ПОСОБИЯ

ФАЙЛ 17/85: src/data/additionalTickets.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 934 | РАЗМ

```
import { Chapter } from "../types";
```

```
export const additionalTickets: Chapter[] = [
 {
 id: "sparse-table",
 title: "2. Двумерная разреженная матрица (Sparse Table)",
 type: "html",
 description: "Разреженная таблица для идемпотентных операций",
 category: "Продвинутые структуры",
 content: `
<section id="sparse-table" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Сложность
 <h2 class="text-2xl sm:text-3xl font-bold text-indigo-600">2. Двумерная разреженная матрица (Sparse Table)
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-700">
 <h3 class="text-xl font-bold text-blue-400">Определение
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">Пусть ST — двумерная таблица размером $L \times R$, где L — количество строк, а R — количество столбцов.
 <p class="text-xl font-mono text-white">Для любого $k \geq 0$ и любых l, r (таких, что $l \leq r$) определим $ST[l][r]$ как минимальный элемент на отрезке $[l, r]$ по столбцу k .
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">

```



```

id: "treap",
title: "3. Декартово дерево (Treap)",
type: "html",
description: "Дерево поиска по ключу и Куча по приоритету",
category: "Продвинутые структуры",
content: `
<section id="treap" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 3
 <h2 class="text-2xl sm:text-3xl font-bold text-slate-900">Декартово дерево (Treap)
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
 <h3 class="text-xl font-bold text-blue-400">Описание
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">Декартово дерево — это бинарное дерево поиска, в котором каждый узел имеет два значения: ключ (key) и приоритет (priority). Ключ используется для поддержания структуры дерева (порядка ин-order), а приоритет используется для поддержания свойства кучи (максимум/минимум в корне).
 <p class="text-xl font-mono text-white">Алгоритмы: Insert, Delete, Find, Traverse.
 метода: Split и Merge.</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Удар катаной (Split)
 </div>
 <p class="text-slate-300 text-sm mb-2">Представьте, что вы ударяете катаной по дереву в точке X. Все деревья, которые находятся левее X, остаются на месте. Все деревья, которые находятся правее X, разламываются и перемещаются вправо.
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Аналогия 2: Слияние капель (Merge)
 </div>
 <p class="text-slate-300 text-sm mb-2">Представьте, что у вас есть два потока воды (два дерева). Вы соединяете их в один поток. Вы берете элементы из первого потока и добавляете их во второй поток, пока первый поток не закончится.
 </div>
 </div>
 </div>
 </div>
`

```

```
<div class="space-y-8">
```

```
<!-- 0. Определение -->
```

```
<div class="bg-slate-700/50 p-6 rounded-xl border
```

```
<h3 class="text-xl font-bold text-blue-400 m
```

```
<div class="bg-slate-900 p-4 rounded-lg mb-4" >
```

```
<p class="text-lg text-blue-300 italic m
```

```
<p class="text-lg sm:text-xl font-mono m
```

```
поиска без
```

```
: серия поворотов, которая поднимает v в ко
```

```
<p class="text-sm text-slate-400 text-c
```

```
т – поворот.</p>
```

```
</div>
```

```
<div class="grid grid-cols-1 lg:grid-cols-3" >
```

```
<div class="bg-slate-900 rounded-lg p-4" >
```

```
<p class="text-xs uppercase tracking
```

```
<p class="text-white font-bold">мож
```

```
<p class="text-slate-400 text-xs mt
```

```
</div>
```

```
<div class="bg-slate-900 rounded-lg p-4" >
```

```
<p class="text-xs uppercase tracking
```

```
<p class="text-emerald-300 font-bol
```

```
<p class="text-slate-400 text-xs mt
```

```
</div>
```

```
<div class="bg-slate-900 rounded-lg p-4" >
```

```
<p class="text-xs uppercase tracking
```

```
<p class="text-white font-bold">0(n
```

```
<p class="text-slate-400 text-xs mt
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<!-- 1. Зачем -->
```

```
<div class="bg-slate-800/60 p-6 rounded-xl border
```

```
<h3 class="text-lg font-bold text-white mb-4" >
```

```
<p class="text-slate-300 text-sm mb-4">В об
```

```
ючи по возрастанию – и дерево вырождается в
```

## Аналогия 2: тропинка через газон

```

</div>
<p class="text-slate-300 text-sm">Ты ид
ода дорожка сама укорачивалась вдвое: чем ч
оплачивает все следующие. Пойдѐшь один раз
</div>
</div>

```

<!-- 3. Поворот -->

```

<div class="bg-slate-800/60 p-6 rounded-xl bord
<h3 class="text-lg font-bold text-white mb-3
<p class="text-slate-300 text-sm mb-4">Повор
движение, три перевешенные ссылки и
<pre class="bg-slate-950 p-4 rounded-lg ove
ading-relaxed">

```

```

 / \ \ поворот 20 / \ \
 20 C -----> A 40
 / \ \ <-----< / \ \
A B поворот 40 B C

```

обход слева направо ДО: A 20 B 40 C

обход слева направо ПОСЛЕ: A 20 B 40 C ← тот же сам

```

<div class="grid grid-cols-1 md:grid-cols-2
<div class="bg-slate-900 rounded-lg p-4
<p class="text-emerald-400 font-bol
<p class="text-slate-300 text-xs">П
сла поворотов – сломать его поворотами нево
</div>

```

```

<div class="bg-slate-900 rounded-lg p-4
<p class="text-amber-400 font-bold
<p class="text-slate-300 text-xs">П
одно и то же место, поэтому B просто перев
</div>

```

```

</div>
</div>

```

```

</div>

```

<!-- 4. Три случая -->

```

<div class="bg-slate-800/60 p-6 rounded-xl bord

```

```
<p class="text-slate-300 text-sm mt-4">И та
нова смотрим на тройку. Zig может случиться
<p class="text-slate-400 text-xs mt-3"> Ни
вороты можно поделат руками.</p>
```

```
</div>
```

```
<!-- 5. Главная ловушка -->
```

```
<div class="bg-rose-950/30 p-6 rounded-xl borde
```

```
<h3 class="text-lg font-bold text-rose-300
```

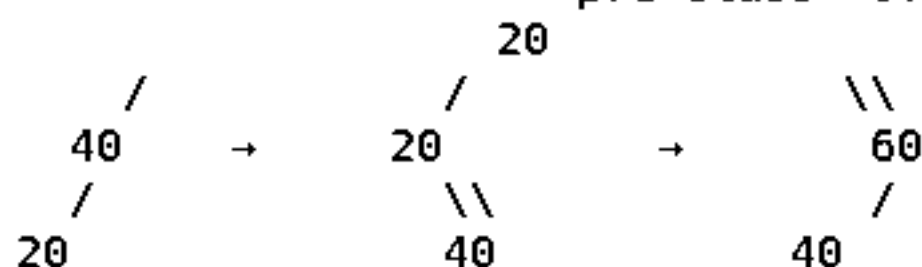
```
<p class="text-slate-300 text-sm mb-4">Это
ми.</p>
```

```
<div class="grid grid-cols-1 lg:grid-cols-2
```

```
<div class="bg-slate-950 rounded-lg p-4
```

```
<p class="text-rose-400 font-bold t
```

```
<pre class="overflow-x-auto text-[1
```



было: бамбук влево

стало: бамбук вправо

глубина никого не сократилась!</pre>

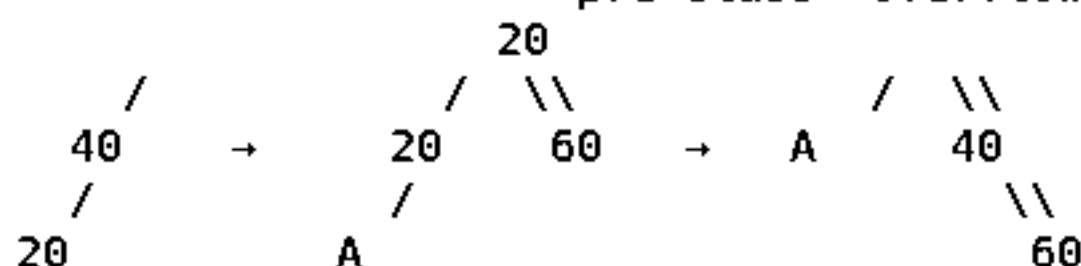
```
<p class="text-slate-400 text-xs mt
мортизации не получится.</p>
```

```
</div>
```

```
<div class="bg-slate-950 rounded-lg p-4
```

```
<p class="text-emerald-400 font-bol
```

```
<pre class="overflow-x-auto text-[1
```



было: линия из 3 узлов

ах идея такая:</p>

```
<div class="grid grid-cols-1 md:grid-cols-3">
 <div class="bg-slate-900 rounded-lg p-4">
 <p class="text-white font-bold mb-1">
 <p class="text-slate-400 text-xs">с
 </div>
 <div class="bg-slate-900 rounded-lg p-4">
 <p class="text-white font-bold mb-1">
 <p class="text-slate-400 text-xs">н
 </div>
 <div class="bg-slate-900 rounded-lg p-4">
 <p class="text-white font-bold mb-1">
 <p class="text-slate-400 text-xs">д
 </div>
```

```
</div>
<p class="text-slate-400 text-xs mt-4">Отсю
сти запросов splay-дерево не хуже (с точнос
дерево. Оно как бы само подстраивается под
```

```
</div>
```

```
<!-- 8. Операции -->
```

```
<div class="bg-slate-800/60 p-6 rounded-xl bord
 <h3 class="text-lg font-bold text-white mb-3">
 <p class="text-slate-300 text-sm mb-4">Крас
 лается в корне.</p>
 <div class="space-y-2 text-sm">
 <div class="bg-slate-900 rounded-lg p-3">
 — спус
 маем последний узел на пути (соседний по зн
 <div class="bg-slate-900 rounded-lg p-3">
) — Spl
 росто отрезаем ссылки.</div>
 <div class="bg-slate-900 rounded-lg p-3">
 k) — sp
 /div>
 <div class="bg-slate-900 rounded-lg p-3">
) — Spl
 него не будет правого сына — туда и вешаем l
```

```

 rotate(x);
 } else if ((g.left === p) === (p.left === x)) {
 rotate(p); // ZIG-ZIG: сн
 rotate(x);
 } else {
 rotate(x); // ZIG-ZAG: сн
 rotate(x);
 }
}
return x; // теперь x –
}

```

// поиск: спустились и подняли найденное наверх

```

function find(root, key) {
 let cur = root, last = null;
 while (cur) {
 last = cur;
 if (key === cur.key) break;
 cur = key < cur.key ? cur.left : cur.right;
 }
 return last ? splay(last) : null; // splay делает
}

```

<p class="text-xs text-slate-400">Вся р  
>ate(x);</span> против <span class="font-monospace">теперь x –  
 т свою оценку.</p>

</div>

</details>

<!-- 10. Плюсы/минусы -->

<div class="grid grid-cols-1 md:grid-cols-2 gap-

<div class="bg-emerald-950/30 rounded-xl p-

<p class="text-emerald-300 font-bold mb

<ul class="list-disc list-inside text-s

<li>Простой код: нет высот, цветов,

<li>Меньше памяти, чем у AVL и крас

<li>Сам подстраивается под «горячие

<li>split / merge получаются почти

</ul>

```

 <p class="text-slate-400">Треар дер
play не хранит вообще ничего и держит балан
p>
 </div>
 </div>
</details>

</div>
</section>`,
 },
 {
 id: "prefix-sums-2d",
 title: "5. Двумерная матрица префиксных сумм",
 type: "html",
 description: "Сжатие суммы подматрицы за O(1)",
 category: "Алгоритмы матрицы",
 content: `
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 , y1...y2) = P[x2][y2] - P[x1-1][y2] - P[x2
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 rder border-emerald-500 shadow-md">
 ≈ Аналогия 1: Вырезание прямоу
 </div>
 <p class="text-slate-300 text-sm mb">
 з него маленький целевой прямоугольник в це

```

```

 </div>
 </div>
</section>`,
 },
 {
 id: "top-sort",
 title: "9. Графы. Топологическая сортировка",
 type: "html",
 description: "Разложение задач по порядку выполнения",
 category: "Графы",
 content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
перед.</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Учеба в вузе
 </div>
 <p class="text-slate-300 text-sm mb-4">
екая сортировка – это расписание, которое
>
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Ограничения: Циклы
 </div>
 </div>
 </div>
 </div>
 </div>
</section>`
 }
]

```



```

 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "graph-bridges",
 title: "11. Графы. Мосты",
 type: "html",
 description: "Рёбра, удаление которых расхреначивает",
 category: "Графы. Продвинутое",
 content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-rose-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">
 <p class="text-xl font-mono text-white">
и fup[v] > tin[u].</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
 Аналогия 1: Единственный мост
 </div>
 <p class="text-slate-300 text-sm mb-4">
 этот мост, экономикой обоих кусков придет
 </div>
 </div>
 </div>
 </div>
 </div>
</div>

```

```

 title: "13. Графы. Эйлеров цикл",
 type: "html",
 description: "Пройти по всем ребрам ровно 1 раз.",
 category: "Графы",
 content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-indigo-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-indigo-300 italic">
 <p class="text-xl font-mono text-white">
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
 Аналогия 1: Рисование не отрыв
 </div>
 <p class="text-slate-300 text-sm mb">
 е сходится нечетное число линий, значит, из
 а везде должно быть четное число (зашел-выш
 </div>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "pathfinding-accel",
 title: "17. Графы. Ускорения поиска",
 type: "html",
 description: "",
 category: "Графы. Пути",
 }

```

```


 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 (проверяем через DSU) - берем в ответ.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия: Прокладка интернет
 </div>
 <p class="text-slate-300 text-sm mb-4">
 с самых дешевых дорог в стране". Он строит
 единую сеть.</p>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "mst-prima",
 title: "19. Графы. Остовное дерево. Прима",
 type: "html",
 description: "",
 category: "Графы. Остовные",
 content: `
<section id="mst-prima" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>

```

```

<div class="bg-slate-900 p-4 rounded-lg mb-12">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 ается с соседом.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
 order border-emerald-500 shadow-md">
 Аналогия: Племена
 </div>
 <p class="text-slate-300 text-sm mb-4">
 изкому племени для объединения. К вечеру пл
 ства шлют гонцов к ближайшему чужому царств
 </div>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "string-kmp",
 title: "21. Алгоритм Кнута-Морриса-Пратта (КМП)",
 type: "html",
 description: "",
 category: "Строки",
 content: `
<section id="string-kmp" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">

```

```

 type: "html",
 description: "",
 category: "Строки",
 content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
щегося с i.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 b
rder border-emerald-500 shadow-md">
 Аналогия 1: Копипаста
 </div>
 <p class="text-slate-300 text-sm mb-4">
в тексте кричит: "Эй! Начиная с этой буквы
т "окно" [L, R] самой дальней найденной копи
 </div>
 </div>
 <details class="bg-slate-900/40 rounded-lg">
 <summary class="p-4 font-bold text-slate-300">
 -b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded">

int l = 0, r = 0;
for (int i = 1; i < n; i++) {
 if (i <= r) z[i] = min(r - i + 1, z[i - l]);

```

```
 енную сетку (ответ), он БЫСТРО (за класс Р)
 </div>
 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #4b4b9b; border-radius: 10px; box-shadow: 0 10px 30px #4b4b9b; position: relative; width: 100%; height: 100%;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-purple-500 shadow-md">
 Аналогия 2: Машина-переводчик
 </div>
 <p class="text-slate-300 text-sm mb-0">
 ь отличный переводчик на английский (Сведения о машине В, то В – это NP-Полная (сосредоточена на
 </div>
 </div>
 </div>
 </section>`,
},
];
```

---

ФАЙЛ 18/85: src/data/content\_temp.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 385 | РАЗМЕР: 10.5 КБ

---

```
export interface Chapter {
 id: string;
 title: string;
 type: "html";
 content: string;
}

export const chapters: Chapter[] = [
 {
 id: "euler-path-vs-cycle",
 title: "Эйлер: Путь ≠ Цикл (ты путаешь!)",
 type: "html",
 content: `<section id="euler-path-vs-cycle" class="text-slate-300 text-sm mb-0">
 <h3>Эйлер: Путь ≠ Цикл (ты путаешь!)</h3>
 <p>В задаче В, то В – это NP-Полная (сосредоточена на
 `
 }
];
```

```
<p class="text-slate-300 text-sm mb-4">
 Ты вышел из дома (нечётная).
 талый пришёл в бар (вторая нечётная).
 Домой вернуться не смог, потому что
</p>
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border-rose-500 shadow-md bg-slate-950">
 Цикл: Гамильтонов синдром (болезнь Гамильтона)
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
 Гамильтон – это болезнь.
 ng> ровно раз и вернуться домой.
 Ты обходишь все бары (вершины) ровно раз.
 Главное – зайти и выйти, не заходя дважды.
 Никто не знает, как решать быстро.

 лноту).
 </p>
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg border-rose-500 shadow-md bg-slate-950">
 <summary class="p-4 font-bold text-slate-400">
 Душный режим: Формальное доказательство
 </summary>
```

```
<div class="p-5 text-sm text-slate-300 space-y-4">
 <p class="text-blue-400">Теорема Эйлера
 <p>
```

```
 Связный граф содержит эйлеров цикл ⇔

Эйлеров путь (не цикл) ⇔ ровно 0 или 2 вершины имеют нечётную степень
 </p>
```

```
<p class="text-rose-400">Почему Гамильтон?
 <p>
```

```
 Задача коммивояжёра (TSP) – это взвешенный граф, где нужно найти кратчайший цикл, проходящий через все вершины.
```

```
 </p>
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg border-emerald-500 shadow-md">
 <div class="absolute -top-3 left-4 bg-slate-800 border-rose-500 shadow-md">
 Мост: Единственная веревка
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Представь, что ты альпинист. Ты спустился в пещеру.
 Из пещеры и нет других выходов — только вверх.
 Если верёвка (ребро v-u) оборвётся, ты погибнешь.
 Если бы из пещеры и был чёрный ход, ты бы вышел.
 </p>
 </div>

 <div class="bg-slate-900 p-6 rounded-lg border-emerald-500 shadow-md">
 <div class="absolute -top-3 left-4 bg-slate-900 border-rose-500 shadow-md">
 Аналогия: Кровеносный сосуд
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Граф — это кровеносная система. Ребра (сосуды) — это не мост, живём.
 Если же перерезать единственную артерию, ты погибнешь.
 Алгоритм DFS ищет такие «критические» ребра.
 </p>
 </div>
 </div>

 <details class="bg-slate-900/40 rounded-lg border-emerald-500 shadow-md">
 <summary class="p-4 font-bold text-slate-400">
 Полный код на C++ (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-4">
 <p>Классическая реализация — <code class="text-slate-400">
```



```
void find_bridges() {
 timer = 0;
 visited.assign(n, false);
 tin.assign(n, -1);
 low.assign(n, -1);

 for (int i = 0; i < n; ++i) {
 if (!visited[i]) dfs(i);
 }
}
```

```
</div>
<p class="text-xs text-slate-400">Занес
</div>
</details>
</div>
</section>`,
},
{
 id: "planarity-euler-formula",
 title: "Планарность: K5, K3,3 и формула Эйлера",
 type: "html",
 content: `<section id="planarity-euler-formula" cla
<div class="flex items-center mb-6">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-3xl font-bold text-white">Планар
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-blue-400 r

 <div class="bg-slate-900 p-4 rounded-lg mb-6">
 <p class="text-lg text-blue-300 italic r
 <p class="text-3xl font-mono text-white"
 <div class="text-sm text-slate-400 spac
 <p><span class="text-blue-400 font-l
 <p><span class="text-emerald-400 fo
 <p><span class="text-rose-400 font-l`
}
```

```

 </p>
 </div>
 </div>

 <details class="bg-slate-900/40 rounded-lg border
 <summary class="p-4 font-bold text-slate-400
 order-slate-700/50">
 Доказательство непланарности K5 (Скры
 </summary>
 <div class="p-5 text-sm text-slate-300 space
 <p class="text-blue-400">Доказательство
 <p>
 Пусть K5 планарен. $V = 5$, $E = 10$.

 $F = E - V + 2 = 10 - 5 + 2 = 7$

Каждая грань ограничена минимум

Сумма длин граней $= 2E = 20$.

Отсюда: $2E \geq 3F \Rightarrow 20 \geq 21$ — <sp

Значит, K5 не может быть планарн
 </p>
 </div>
 </details>
 </div>
</section>`,
 },
 {
 id: "graph-articulation",
 title: "12. Графы. Точки сочленения",
 type: "html",
 content: `<section id="graph-articulation" class="ml
 <div class="flex items-center mb-6">
 <span class="bg-rose-600 text-white px-4 py-1 r
 <h2 class="text-3xl font-bold text-white">Точки
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border
 <h3 class="text-xl font-bold text-rose-400`

```

```

 <p class="text-slate-300 text-sm mb-4">
 У тебя несколько компьютеров в одной
 абелем (мостом). Сами свитчи — это точки со
 </p>
 </div>
</div>

<details class="bg-slate-900/40 rounded-lg border-1
 <summary class="p-4 font-bold text-slate-400
 order-slate-700/50">
 Душный мод: Код и корень DFS (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-4">
 <p class="text-blue-400 font-bold">Особый случай
 <p>
 Для стартовой вершины обхода правил
 то выше неё прыгать некуда). Поэтому для неё
 у него больше 1 независимого ребенка
 </p>
 <div class="bg-slate-950 p-4 rounded-lg">
 <pre class="text-xs font-mono text-slate-300">
void dfs(int v, int p = -1) {
 visited[v] = true;
 tin[v] = low[v] = timer++;
 int children = 0;

 for (int to : adj[v]) {
 if (to == p) continue;
 if (visited[to]) {
 low[v] = min(low[v], tin[to]);
 } else {
 dfs(to, v);
 low[v] = min(low[v], low[to]);
 if (low[to] >= tin[v] && p != -1)
 IS_CUTPOINT(v);
 ++children;
 }
 }
}

```

```

import { tickets14to20 } from "../tickets/ticket14_20";
import { tickets21to24 } from "../tickets/ticket21_24";
import { tickets6to13 } from "../tickets/ticket6_13";
import { chapters as newChapters } from "../content_temp

// We want to combine all of them, but overwrite 7, 11,
const merged = [
 ...graphChapters,
 ...additionalTickets,
 ...tickets6to13,
 ...tickets14to20,
 ...tickets21to24
].filter(c => c && c.id);

// Remove old 7, 11, 12, 13: graph-planar-colors, graph
const toRemove = ["graph-planar-colors", "graph-bridges

const dedup = new Map<string, Chapter>();
merged.forEach(c => {
 if (!toRemove.includes(c.id)) {
 // We prefer the newer versions (like tickets14_20
 dedup.set(c.id, c);
 }
});

// Add the new ones
if (newChapters) {
 newChapters.forEach(c => dedup.set(c.id, c));
}

const finalChapters = Array.from(dedup.values());

finalChapters.sort((a, b) => {
 const numA = parseInt(a.title.match(/(\d+)/)?.[0] ||
 const numB = parseInt(b.title.match(/(\d+)/)?.[0] ||
 return numA - numB;
});

```

```

 simulator?: string;
}

export const GLOSSARY: GlossaryEntry[] = [
 {
 id: "bfs",
 labels: ["BFS", "обход в ширину", "поиск в ширину",
 title: "BFS – обход в ширину",
 short:
 "Волна от источника: сначала все соседи, потом со
 formal:
 "Очередь (FIFO). Кладём старт, достаём вершину –
 у рёбер.",
 complexity: "O(V + E)",
 demo: "bfs",
 simulator: "queue-bfs",
 },
 {
 id: "dfs",
 labels: ["DFS", "обход в глубину", "поиск в глубину",
 title: "DFS – обход в глубину",
 short:
 "Прёшь вперёд до упора, упёрся – откатываешься на
 formal:
 "Стек (или рекурсия). Из DFS растут: времена вход
 complexity: "O(V + E)",
 demo: "dfs",
 simulator: "stack-dfs",
 },
 {
 id: "heap",
 labels: ["куча", "кучу", "кучи", "куче", "кучей", "к
 title: "Куча (priority queue)",
 short:
 "Не отсортированный список, а «мешок, из которого
 formal:
 "Полное бинарное дерево в массиве: дети узла i ле
 complexity: "вставка/извлечение O(log n), «посмотре

```

```

 "Дерево, где путь от корня по буквам и есть слово
 formal: "Из корня по символам; в вершине хранится ф.
 complexity: "поиск слова — $O(|\text{слова}|)$ ",
 demo: "trie",
 simulator: "aho-corasick",
},
{
 id: "bfs-on-trie",
 labels: [
 "обход дерева",
 "обход бора",
 "обход в ширину по бору",
 "какой-то обход",
 "обходом дерева",
 "конечный автомат",
 "конечного автомата",
 "автомат",
 "автомата",
],
 title: "Обход бора в ширину (тот самый «какой-то об.
 short:
 "В Ахо–Корасике бор обходят именно BFS: суффикснул
 о есть строго по уровням.",
 formal:
 "Кладём в очередь детей корня (их ссылка = корень
 кладём с в очередь.",
 complexity: " $O(\text{суммарной длины словаря} \times \text{алфавит})$ ",
 demo: "trie-bfs",
 simulator: "aho-corasick",
},
{
 id: "suffix-link",
 labels: ["суффиксная ссылка", "суффиксные ссылки",
 title: "Суффиксная ссылка",
 short:
 "«Куда откатиться, если следующая буква не подошла
 строки.",
 formal: "link(v) = go(link(parent(v)), ch). Считает

```

```
{
 id: "sparse-table",
 labels: [
 "разреженная таблица", "разреженная таблица", "sp",
 "разреженная", "двоичные подьёмы", "двоичный подь",
],
 title: "Разреженная таблица (метод двоичных подьёмов)",
 short:
 "Заранее считаем ответ для всех отрезков длины 1,
 М.",
 formal: "ST[i][j] = op(ST[i][j-1], ST[i+2^(j-1)][j-1])",
 complexity: "препроцессинг $O(n \log n)$, запрос $O(1)$ ",
 demo: "sparse-table",
 simulator: "sparse-table",
},
{
 id: "segment-tree",
 labels: ["дерево отрезков", "дереве отрезков", "дер"],
 title: "Дерево отрезков",
 short:
 "Матрёшка из отрезков: корень отвечает за весь ма
 товых кусков.",
 formal: "Строится за $O(n)$, запрос и обновление — $O(\log n)$ ",
 demo: "segment-tree",
 simulator: "segment-trees",
},
{
 id: "bridge",
 labels: ["мост", "моста", "мосты", "мостов", "мостов"],
 title: "Мост",
 short:
 "Ребро, после удаления которого граф разваливается",
 formal: "Ребро (u,v) — мост $\iff \text{low}[v] > \text{tin}[u]$, где
 ребро в родителя — нет).",
 complexity: " $O(V + E)$ одним DFS",
 demo: "bridge",
 simulator: "bridges-code",
},
```

```

 id: "np",
 labels: [
 "NP-полная", "NP-полнота", "NP-трудная", "NP-полн",
 "класс P", "полином", "полиномиальное",
],
 title: "Класс NP и NP-полнота",
 short:
 "NP — «решение проверить легко, найти трудно» (судно",
 "е в NP.",
 formal: "Задача NP-полна, если она в NP и к ней полн",
 demo: "np",
},
{
 id: "potential",
 labels: ["потенциал", "потенциалы", "потенциалов",
 title: "Потенциалы (перевзвешивание Джонсона)",
 short:
 "Трюк «поднять все дороги на одинаковую высоту»: ",
 formal: " $w'(u,v) = w(u,v) + h(u) - h(v)$, где h — ра",
 "к путей сохраняется.",
 demo: "relax",
 simulator: "johnson-algo",
},
{
 id: "scc",
 labels: ["компонента сильной связности", "компоненты",
 title: "Компоненты сильной связности",
 short:
 "Группы вершин, где из каждой можно дойти до кажд",
 formal: "Косарайю: DFS с сохранением порядка выхода",
 complexity: " $O(V + E)$ ",
 demo: "scc",
 simulator: "scc-kosaraju",
},
{
 id: "prefix-function",
 labels: [
 "префикс-функция", "префикс-функции", "префикс-фу
```



```

 ция).",
 demo: "dp",
 simulator: "dynamic-programming",
},
{
 id: "shortest-path",
 labels: ["кратчайший путь", "кратчайшие пути", "кра
 title: "Кратчайший путь",
 short:
 "Самый дешёвый маршрут из А в В. «Дешёвый» – по с
 formal:
 "Дейкстра – неотрицательные веса, $O(E \log V)$. Фор
 ы, $O(V^3)$.",
 demo: "relax",
 simulator: "dijkstra",
},
{
 id: "negative-cycle",
 labels: ["отрицательный цикл", "отрицательного цикл
 title: "Отрицательный цикл",
 short:
 "Круг, пройдя по которому вы становитесь «богаче»
 formal:
 "Форд–Беллман: если на V -й итерации ещё удастся с
 demo: "relax",
 simulator: "bellman-ford",
},
{
 id: "greedy",
 labels: [
 "жадный алгоритм", "жадный", "жадно", "жадная",
 "самое дешевое ребро", "самое дешёвое ребро", "са
],
 title: "Жадный алгоритм",
 short:
 "На каждом шаге хватаем то, что лучше прямо сейчас
 formal:
 "Корректность обычно доказывается «леммой о разре

```

```

 },
 {
 id: "adjacency",
 labels: ["матрица смежности", "список смежности", "список смежности"],
 title: "Способы хранить граф",
 short:
 "Матрица смежности – таблица «есть ли ребро», быстрая

 и экономно для разреженных графов.",
 formal: "Матрица: проверка $O(1)$, память $O(V^2)$. Список:",
 },
 {
 id: "dijkstra",
 labels: ["Дейкстра", "Дейкстры", "Дейкстре", "Дейкстры"],
 title: "Алгоритм Дейкстры",
 short:
 "Жадный «навигатор»: всегда раскрываем ближайший

 путь.",
 formal:
 "Куча хранит пары (расстояние, вершина). Достали

 элемент с минимальным расстоянием.",
 complexity: " $O(E \log V)$ с бинарной кучей",
 demo: "relax",
 simulator: "dijkstra",
 },
 {
 id: "bellman-ford",
 labels: ["Форд-Беллман", "Форда-Беллмана", "Беллман-Форд"],
 title: "Алгоритм Форда-Беллмана",
 short:
 "Тупой, но честный: $V-1$ раз проходим по ВСЕМ рёбрам.",
 formal:
 "После k -й итерации гарантированно верны все кратчайшие

 пути, если нет отрицательного цикла.",
 complexity: " $O(V \cdot E)$ ",
 demo: "relax",
 simulator: "bellman-ford",
 },
],
 {

```

```

 demo: "mst",
 simulator: "mst-boruvka",
},
{
 id: "kruskal",
 labels: ["Краскал", "Краскала", "Краскалу", "Kruska",
 title: "Алгоритм Краскала",
 short:
 "Сортируем все рёбра по весу и берём подряд те, ч
 formal:
 "Сортировка $O(E \log E) + E$ операций union/find. С
 complexity: "O(E log E)",
 demo: "mst",
 simulator: "mst-kruskal",
},
{
 id: "components",
 labels: [
 "компонента связности", "компоненты связности", "
 "компоненте связности", "связности", "связный", "
],
 title: "Компоненты связности",
 short:
 "Граф-архипелаг: острова, между которыми нет мост
 бой.",
 formal:
 "Считаются одним проходом: пока есть непосещённая
 complexity: "O(V + E)",
 demo: "components",
 simulator: "graph-dfs-bfs",
},
{
 id: "graph",
 labels: [
 "граф", "графа", "графе", "графы", "графов", "гра
 "вершины", "вершина", "вершин", "ребра с весом",
],
 title: "Граф",

```

```

 "Слева всё меньше корня, справа — всё больше. Поиск
formal:
 "Без балансировки дерево может вырождаться в список
complexity: "O(высоты): от $O(\log n)$ до $O(n)$ ",
demo: "segment-tree",
},
{
 id: "inclusion-exclusion",
 labels: [
 "включений-исключений", "включения-исключения", "вырезание
 прямоугольников",
],
 title: "Включения-исключения на префиксных суммах",
 short:
 "Чтобы получить сумму прямоугольника, берём больший
 отрезали дважды.",
 formal:
 " $\text{Sum}(x1..x2, y1..y2) = P[x2][y2] - P[x1-1][y2] - P[x2][y1-1] + P[x1-1][y1-1]$ ",
 complexity: "запрос $O(1)$ после $O(nm)$ предподсчёта",
 demo: "prefix-sum",
 simulator: "prefix-sums-2d",
},
{
 id: "reduction",
 labels: ["сведение", "сведения", "сведении", "сведёние"],
 title: "Сведение задач",
 short:
 "«Я не умею решать A, зато умею B». Если A можно решить,
 то и B можно.",
 formal:
 " $A \leq_p B$: есть полиномиальная функция, переводящая
 NP-полную задачу к новой.",
 demo: "np",
},
{
 id: "range",
 labels: ["отрезок", "отрезка", "отрезке", "отрезки"],
 title: "Запрос на отрезке",
 short:

```

```

 "Самый простой случай: родитель x уже сидит в корн
 канчивается, если глубина была нечётной.",
 formal: "Применяется ровно один раз за всю операцию
 demo: "zig",
 simulator: "splay-rotations",
 },
 {
 id: "zig-zig",
 labels: ["Zig-Zig", "зиг-зиг", "ZigZig"],
 title: "Zig-Zig – x и родитель с одной стороны",
 short:
 "Дерево вытянулось в «бамбук»: g → p → x идут в о
 м (p, x). Именно это вдвое сокращает глубину вс
 formal:
 "Если крутить наивно (два раза поднимать x), бамб
 demo: "zig-zig",
 simulator: "splay-rotations",
 },
 {
 id: "zig-zag",
 labels: ["Zig-Zag", "зиг-заг", "ZigZag", "зигзаг"],
 title: "Zig-Zag – x и родитель с разных сторон",
 short:
 "Узлы идут «змейкой»: p – левый сын g, а x – прав
 p и g становятся двумя детьми x.",
 formal: "Эквивалентно двойному повороту AVL-дерева
 demo: "zig-zag",
 simulator: "splay-rotations",
 },
];

```

/\*\* Быстрый доступ по id. \*/

export const GLOSSARY\_BY\_ID = new Map(GLOSSARY.map((e) => {

/\*\* Все метки, отсортированные от длинных к коротким (ч

export const GLOSSARY\_LABELS: { label: string; id: string; }[] =

GLOSSARY.map((e) => ({ label: e.labels[0], id: e.id }));

).sort((a, b) => b.label.length - a.label.length);

```
<div class="flex items-center mb-6">

 <h2 class="text-3xl font-bold text-white">Дейкстра
 </div>
```

```
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 </div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 border border-emerald-500 shadow-md">
 Аналогия 1: Позитивное планирование
 </div>
 <p class="text-slate-300 text-sm mb-4">
 (нельзя поехать и заработать). Он всегда в
 <div id="slot-chapter-image-dijkstra">
 </div>
```

```
<!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 border border-rose-500 shadow-md">
 Ограничения
 </div>
 <p class="text-slate-300 text-sm mb-4">
 охождение улицы), Дейкстра сломается, так как
 </div>
 </div>
```

```
<details class="bg-slate-900/40 rounded-lg">
```

```

<div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 ♂ Аналогия 1: Параноик
</div>
<p class="text-slate-300 text-sm mb
еряет ВСЕ возможные дороги V-1 раз подряд.
<div id="slot-chapter-image-bellman
</div>

<!-- Аналогия 2 -->
<div class="bg-slate-900 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
 Отрицательный цикл
 </div>
 <p class="text-slate-300 text-sm mb
г станет ЕЩЕ короче — значит мы попали во в
 </div>
</div>

<div id="slot-bellman-viz" class="mt-8"></d
</div>
</div>
</section>`
},
{
 id: "floyd",
 title: "Билет 16. Флойд-Уоршелл",
 type: "html",
 category: "Графы",
 content: `<section id="floyd-content" class="mb-12
<div class="flex items-center mb-6">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-3xl font-bold text-white">Флойд
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord

```

```

type: "html",
category: "Строки",
content: `<section id="aho-corasick" class="mb-12 s
<div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-indigo-600 text-white px-4 py-1 round
 <h2 class="text-2xl sm:text-3xl font-bold text-white
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-l
 <h3 class="text-xl font-bold text-blue-400 mb-4">

 <div class="bg-slate-900 p-4 rounded-lg mb-6 text
 <p class="text-sm text-blue-300 italic mb-2">0с
 <p class="text-lg font-mono text-white">Бор (Тр
 </div>
 <p class="text-slate-300 text-sm">Позволяет найти
 -mono text-amber-300 bg-black/30 px-1 rounded">
 y.</p>
 </div>

 <!-- Аналогии -->
 <div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg border bo
 <div class="absolute -top-3 left-4 bg-slate-700
 emerald-500 shadow-md">
 Аналогия 1: Паутина (Бор)
 </div>
 <p class="text-slate-300 text-sm mb-4">Представ
 мы движемся по веткам. Если нужного продолж
 ("нити страховки") в самый длинный из извест
 </div>

 <div class="bg-slate-900 p-6 rounded-lg border-2
 <div class="absolute -top-3 left-4 bg-rose-900
 -500 shadow-md">
 Аналогия 2: Матёшка (Терминальные)
 </div>

```



```
// Суффиксная ссылка ребёнка – это переход к следующему ребру
nodes[child].link = nodes[nodes[v].link].go_to_child(v);

// Если суффиксная ссылка сама является словом, то она не имеет терминальной ссылки
// Иначе наследуем terminal_link
if (nodes[nodes[child].link].is_terminal) {
 nodes[child].term_link = nodes[child].link;
} else {
 nodes[child].term_link = nodes[nodes[child].link].term_link;
}

q.push(child);
}
}
}

</div>
</div>
</details>
</div>
</section>`
},
{
 id: "alg-map",
 title: "Алгоритмы на карте",
 type: "html",
 category: "Графы",
 content: `<section id="alg-map-content" class="mb-1">
<div class="flex items-center mb-6">
 Алгоритмы на карте
 <h2 class="text-3xl font-bold text-white">Алгоритмы на карте
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-purple-400">Алгоритмы на карте
 <p class="text-slate-300 text-sm mb-4">Когда мы имеем карту с городами и дорогами между ними, то нам нужно найти кратчайший путь от одного города до другого. Для этого можно использовать алгоритм Дейкстры. Он работает так же, как алгоритм Брайда-Форда, но вместо того чтобы перебирать все возможные пути, он использует приоритетную очередь. Это позволяет находить кратчайший путь за время O(N^2), где N — количество городов. Алгоритм работает следующим образом:
 1. Выбираем начальный город s.
 2. Помещаем его в очередь.
 3. Пока очередь не пуста:
 - Извлекаем из очереди город v с минимальным расстоянием от s.
 - Если v — целевой город t, то выходим.
 - Для каждого соседа u города v обновляем расстояние от s до u, если оно больше расстояния от s до v плюс вес ребра (v,u).
 - Добавляем u в очередь.
 4. Выходим.
 </div>
 </div>
</div>`
}
```

```

],
 correctIndex: 1,
 explanation: "Для эйлерова цикла работает критерий
ого критерия. Это NP-полная задача, которую не
},
{
 question: "Куда ведёт эйлеров путь, если нечётных
options: [
 "Всё ещё можно пройти по всем рёбрам ровно раз",
 "Такого графа не существует",
 "Эйлерова пути нет – нужно удалять лишние рёбра",
],
 correctIndex: 2,
 explanation: "Если нечётных вершин больше 2, эйлер
ровно 2 нечётные или 0."
}
],
"bridges-code": [
{
 question: "После DFS у нас $low[u] = 4$, а $tin[v] =$
options: [
 "Да, так как $low[u] (4) > tin[v] (2)$ ",
 "Нет, $low[u]$ должно быть меньше или равно",
 "Только если граф связный и неориентированный"
],
 correctIndex: 0,
 explanation: "Условие моста: $low[u] > tin[v]$. В н
},
{
 question: "Для чего нужен массив $low[v]$?",
 options: [
 "Для того чтобы было больше переменных в коде",
 "Чтобы знать минимальный tin предка, до которого",
 "Чтобы хранить глубину рекурсии"
],
 correctIndex: 1,
 explanation: " $low[v]$ хранит минимальное время вхо
поиска мостов и точек сочленения."
}
]
```

```
 "Потому что Эйлер был против",
 "Потому что 5 вершин — несчастливое число"
],
 correctIndex: 0,
 explanation: "У K5: V=5, E=10, 3V - 6 = 9. 10 > 9"
 }
]
};
```

---

ФАЙЛ 23/85: src/data/vizSync.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 322 | РАЗМ

---

```
/**
 * Захардкоженная «карта синхронизации» компилятора с ви
 *
 * Для каждой страницы (главы) здесь вручную перечислен
 * по которым идёт пошаговая подсветка в её интерактивн
 * в первую очередь индексы циклов i, j, k и размеры n,
 *
 * Компилятор НЕ показывает код алгоритма — только комм
 * переменными, чтобы студент повторял шаги руками, гля
 */
```

```
export interface SyncVariable {
 /** Имя переменной, как в коде визуализации: i, j, k,
 name: string;
 /** Роль в алгоритме (показывается во всплывающей под
 role: string;
 /** Характерный диапазон/значение на демонстрации. */
 range?: string;
}
```

```
export interface PageSync {
 /** Заголовок демонстрации на странице (если интеракт
 vizTitle?: string;
```

```

 { name: "p", role: "родитель узла x перед поворотом", range: 1 },
 { name: "g", role: "дедушка узла x — нужен в случае поворота", range: 1 },
],
},
"sparse-table": {
 vizTitle: "Разреженная таблица (1D и 2D)",
 stepNote: "Шаг построения = заполнение ячейки уровня",
 variables: [
 { name: "n", role: "длина массива / сторона квадрата", range: 1000 },
 { name: "k", role: "уровень таблицы: ячейка хранит сумму", range: 1000 },
 { name: "i", role: "начало блока в строке уровня", range: 1000 },
 { name: "j", role: "номер уровня при построении (от 0 до k)", range: 1000 },
 { name: "r, c", role: "строка и столбец ячейки в матрице", range: 1000 },
 { name: "kx, ky", role: "уровни по вертикали и горизонтали", range: 1000 },
],
},
"prefix-sums-2d": {
 vizTitle: "Префиксные суммы (1D и 2D)",
 stepNote: "Шаг = вычисление $S[i][j]$ по формуле включения-исключения",
 variables: [
 { name: "n", role: "число строк матрицы", range: 1000 },
 { name: "m", role: "число столбцов матрицы", range: 1000 },
 { name: "i", role: "индекс текущей строки (таблицы)", range: 1000 },
 { name: "j", role: "индекс текущего столбца (столбца)", range: 1000 },
 { name: "S[i][j]", role: "сумма прямоугольника (1D и 2D) на каждом шаге" },
],
},
"dynamic-programming": {
 vizTitle: "Динамическое программирование (мемоизация)",
 stepNote: "Шаг = вызов fibМемо(n): попадание в кэш и вычисление",
 variables: [
 { name: "n", role: "номер числа Фибоначчи, которое нужно вычислить", range: 1000 },
 { name: "мемо[n]", role: "кэш уже посчитанных значений" },
],
},
"heap-beam-search": {
 vizTitle: "Куча (priority queue)",

```

```

 { name: "v", role: "текущая вершина обхода", range:
 { name: "to / u", role: "очередной сосед вершины",
 { name: "visited", role: "множество уже посещённых",
 { name: "order", role: "порядок, в котором обход",
 },
 },
 "graph-components": {
 // отдельного виджета нет, тема опирается на DFS/BFS
 variables: [],
 },
 "top-sort": {
 vizTitle: "Топологическая сортировка",
 stepNote: "Шаг = выход рекурсии из вершины v: она д
 variables: [
 { name: "v", role: "текущая вершина DFS", range:
 { name: "to", role: "вершина, куда ведёт ребро из
 { name: "i", role: "перебор стартовых вершин во в
 { name: "order", role: "список выхода из рекурсии
],
 },
 "scc-kosaraju": {
 vizTitle: "Компоненты сильной связности (Косарайю)"
 stepNote: "Шаг = вершина из order (в обратном поряд
 variables: [
 { name: "v", role: "текущая вершина DFS (первый пр
 { name: "to", role: "сосед по ребру (на 2-м прохо
 { name: "i", role: "индекс прохода по order на вто
 { name: "order", role: "порядок выхода из DFS пер
 { name: "comp", role: "номер компоненты сильной с
],
 },
 "graph-articulation": {
 vizTitle: "Точки сочленения",
 stepNote: "Шаг = обновление low[v] по ребру; вершин
 variables: [
 { name: "v", role: "текущая вершина DFS", range:
 { name: "to", role: "сосед: либо ребёнок в DFS-дер
 { name: "tin[v]", role: "время входа в вершину (п

```



```

 { name: "π[i]", role: "префикс-функция: длина наи
 ица" },
],
 },
 "string-z-func": {
 vizTitle: "Z-функция",
 stepNote: "Шаг = вычисление z[i]: внутри Z-блока [l
 variables: [
 { name: "n", role: "длина строки s", range: "n =
 { name: "i", role: "позиция, для которой считаем
 { name: "l, r", role: "правый крайний Z-блок: отр
 { name: "z[i]", role: "длина наибольшего общего п
],
 },
 "aho-corasick": {
 vizTitle: "Ахо-Корасик",
 stepNote: "Шаг = построение бора по образцам, затем
 variables: [
 { name: "state", role: "текущее состояние автомата
 { name: "c", role: "очередной символ текста – пер
 { name: "go(v, c)", role: "функция перехода автом
],
 },
 "complexity-classes": {
 variables: [],
 },
 intro: {
 // мнемодкарточки без шагов алгоритма
 variables: [],
 },
 "alg-map": {
 variables: [],
 },
 };

```

```

/** Безопасный доступ: страница без записи получает пус
export const getPageSync = (chapterId?: string | null):
 (chapterId && PAGE_SYNC[chapterId]) || { variables: [

```

```
<div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
order border-emerald-500 shadow-md">
```



```

 <div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4
border-rose-500 shadow-md">
 Аналогия 2: Ступени
 </div>
 <p class="text-slate-300 text-sm mb-2">
 нужно покрыть отрезок, мы берем две самые бо
 </div>
 </div>
 <details class="bg-slate-900/40 rounded-lg"
 <summary class="p-4 font-bold text-slate-300"
 -b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300"
 <pre class="bg-slate-950 p-4 rounded"
int k = log2(R - L + 1);
int minimum = min(st[L][k], st[R - (1 << k) + 1][k]);</pre>
 </div>
 </details>
 </div>
</div>
</section>`
 },
 {
 id: "treap",
 title: "3. Декартово дерево (Treap)",
 type: "html",
 description: "Дерево поиска по ключу и Куча по приоритету",
 category: "Продвинутые структуры",
 content: `
<section id="treap" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"

```

```

 } else {
 auto [L, R] = split(t->left, x);
 t->left = R;
 return {L, t};
 }
 }</pre>
</div>
</details>
</div>
</div>
</section>`
},
{
 id: "splay-tree",
 title: "4. Splay-дерево",
 type: "html",
 description: "Дерево поиска, которое самобалансируется",
 category: "Продвинутые структуры",
 content: `
<section id="splay-tree" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 (Zig, Zig-Zig, Zig-Zag), гарантируя амортизацию
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
 border-emerald-500 shadow-md">
 Аналогия 1: VIP-лифт
 </div>
 </div>
 </div>
 </div>
 </div>
</section>`

```

```

<div class="space-y-4 text-sm text-slate-500">
 <div>
 <strong class="text-white block">
 корень?
 <p>Оно поднимет узел, на котором
 ска. За счет сдвига этого листа в корень, д
 </p>
 </div>

 <div>
 <strong class="text-white block">
 <p>Если агент постоянно переключо
 находящимися на разных концах дерева, его
 ащая дерево в вытянутый бамбук для другого.
 . Постоянное свинг-переключение превратит пр
 </p>
 </div>

 <div>
 <strong class="text-white block">
 <p>Хотя один поиск может стоять
 емах обладают прекрасным свойством: они не
 по пути. "Палочное" дерево прессуется в сб
 осов.</p>
 </div>
</div>
</div>
</div>
</div>
</section>`
 },
 {
 id: "prefix-sums-2d",
 title: "5. Двумерная матрица префиксных сумм",
 type: "html",
 description: "Сжатие суммы подматрицы за $O(1)$ ",
 category: "Алгоритмы матрицы",
 content: `
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">

```

```

{
 id: "dijkstra",
 title: "14. Графы. Поиск кратчайшего пути. Дейкстра",
 type: "html",
 category: "Графы. Пути",
 content: `
<section id="dijkstra-content" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">14. Графы. Поиск кратчайшего пути. Дейкстра
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
 <h3 class="text-xl font-bold text-emerald-400">Аналогия 1: Позитивное планирование
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">Вспомогательная задача: найти кратчайший путь от города А до города В, не заезжая в город Б.
 <p class="text-xl font-mono text-white">Аналогия 1: Позитивное планирование
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Позитивное планирование
 </div>
 <p class="text-slate-300 text-sm mb-4">Позитивное планирование: поиск кратчайшего пути от города А до города В, не заезжая в город Б. (нельзя поехать и заработать). Он всегда работает.
 </div>
 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Ограничения
 </div>
 <p class="text-slate-300 text-sm mb-4">Ограничения: поиск кратчайшего пути от города А до города В, не заезжая в город Б. Он всегда работает и не умеет пересматривать уже "зафиксированные" пути.
 </div>
 </div>
 </div>
 </div>
`

```

```

 опали во временную петлю.</p>
 </div>
 </div>
 <div id="slot-bellman-viz" class="mt-8"></div>
 </div>
</section>`
 },
 {
 id: "floyd",
 title: "16. Графы. Поиск кратчайшего пути. Флойд",
 type: "html",
 category: "Графы. Пути",
 content: `
<section id="floyd-content" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-indigo-400">Поиск кратчайшего пути. Флойд
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-indigo-400">Внешний цикл
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-indigo-300 italic">Внешний цикл — это цикл, который проходит через все вершины графа, начиная с вершины k, и возвращаясь к ней.
 <p class="text-xl font-mono text-white">
 graph LR
 k((k)) --> v1((v1))
 v1 --> v2((v2))
 v2 --> k

```

Дейкстра (Билет 14) быстрая, но  
ает очень медленно. Мы хотим сделать **Джон**  
е пути остались теми же.

</p>

<ol class="text-sm text-slate-300 s  
<li>Добавляем **фиктивную** верш  
<li>Запускаем от неё **медленн  
b>«потенциалы»</b> <code class="text-pink-4  
<li>Меняем старые веса: новое р  
<li>**Магия!**</b> Все веса тепе  
ренные потенциалы сокращаются).</li>  
<li>Теперь смело запускаем **б****

</ol>

</div>

</div>

<div id="slot-johnson-viz" class="mt-8"></div>

</div>

</div>

</section>

},

{

id: "mst-kruskal",

title: "18. Графы. Остовное дерево. Краскал",

type: "html",

category: "Графы. Остовные",

content: `

<section id="mst-kruskal" class="mb-12 scroll-mt-10">

<div class="flex items-center mb-6 flex-wrap gap-3">

<span class="bg-blue-600 text-white px-4 py-1 r

<h2 class="text-2xl sm:text-3xl font-bold text-r

</div>

<div class="space-y-8">

<div class="bg-slate-700/50 p-6 rounded-xl bord

<h3 class="text-xl font-bold text-emerald-4

<div class="bg-slate-900 p-4 rounded-lg mb-r

<p class="text-lg text-emerald-300 ital

<p class="text-xl font-mono text-white"

(проверяем через DSU) - берем в ответ.</p>

```

 order border-emerald-500 shadow-md">
 ✱ Аналогия: Заражение вирусом
 </div>
 <p class="text-slate-300 text-sm mb-4">
 ли вирус). Мы стартуем из одного города, и
 Сеть растет только из одного связного куска
 </div>
 </div>
</div>
</section>`
},
{
 id: "mst-boruvka",
 title: "20. Графы. Остовное дерево. Борувка",
 type: "html",
 category: "Графы. Остовные",
 content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 ается с соседом.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 order border-emerald-500 shadow-md">
 Аналогия: Племена
 </div>
 <p class="text-slate-300 text-sm mb-4">

```

```

<div class="grid grid-cols-1 xl:grid-cols-2"
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 ♂ Аналогия 1: Поиск без отка
 </div>
 <p class="text-slate-300 text-sm mb
 Мы идём по строке слева направо.
строки СЕЙЧАС закончился под моим пальцем?»
 Представь, ты ищешь в тексте слов
х</code>. Тупой алгоритм начнет искать зано
</code>, а это начало нашего слова! Не надо
 </p>
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
 Аналогия 2: LLM стриминг
 </div>
 <p class="text-slate-300 text-sm mb
 Для LLM, которая стримит токены
каждом шаге. Если мы частично совпали с зап
акого места этого слова мы можем продолжить
 </p>
 </div>
</div>

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Пример вычисления (Скрыто)
</summary>
 <div class="p-5 text-sm text-slate-300
 <p>Тот же пример: abcabacd</p>

```



```
title: "22. Z-функция — Сдвиг и наложение",
type: "html",
category: "Строки",
content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 са 0) и её суффиксом, начинающимся с i.</p>
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Линейка-шаблон
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Тут мы берём всю строку и «прикладываем» её к строке.
 Я начну читать строку отсюда, насколько длинная строка.
 Это самый быстрый способ найти Z-функцию.
 >, считаешь Z-функцию, и там, где <code>Z[i]</code>,
 </p>
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Аналогия 2: LLM Самокопираст
 </div>
 </div>
 </div>
 </div>
 </div>
`
```

```

while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z
if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}

```

```

 </div>
 </details>
</div>
</div>
</section>`
},
{
 id: "aho-corasick",
 title: "23. Алгоритм Ахо-Корасик",
 type: "html",
 category: "Строки",
 content: `
<section id="aho-corasick" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-l">
 <h3 class="text-xl font-bold text-blue-400 mb-4">

 <div class="bg-slate-900 p-4 rounded-lg mb-6 text
 <p class="text-sm text-blue-300 italic mb-2">Осн
 <p class="text-lg font-mono text-white">Бор (Tr
 </div>
 <p class="text-slate-300 text-sm">Позволяет найти
 го один проход.</p>
 </div>

 <!-- Аналогии -->
 <div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg border bo
 <div class="absolute -top-3 left-4 bg-slate-700
 emerald-500 shadow-md">

```

```

 </div>
 </details>
 </div>
 </section>`
 },
 {
 id: "complexity-classes",
 title: "24. Классы сложности, сведение задач",
 type: "html",
 category: "Теория сложности",
 content: `
<section id="complexity-classes" class="mb-12 scroll-mt
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border
 <h3 class="text-xl font-bold text-purple-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-purple-300 italic">
 <p class="text-xl font-mono text-white">
Сведение (Reduction) $A \rightarrow B$: Если я умею реша
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 rder border-emerald-500 shadow-md">
 Аналогия 1: Судoku (P vs NP)
 </div>
 <p class="text-slate-300 text-sm mb-4">
пример сортировка чисел). Класс NP –
енную сетку (ответ), он БЫСТРО (за класс P)
 </div>
 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l

```

```

<h3 class="text-xl font-bold text-slate-300">
<div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-slate-300 italic">
 <p class="text-xl font-mono text-white">
(V + E).</p>
</div>
<div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия: Матрица = Таблица
 </div>
 <p class="text-slate-300 text-sm mb-4">
гупо, но проверка "знает ли Вася Петю" мгр
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
 Аналогия: Списки = Контакты
 </div>
 <p class="text-slate-300 text-sm mb-4">
т. Оптимально для почти всех задач.</p>
 </div>
 </div>
</div>

{/* DFS vs BFS */}
<div class="bg-slate-700/50 p-6 rounded-xl bord
<h3 class="text-xl font-bold text-indigo-400">

<div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия DFS -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
rder border-indigo-500 shadow-md">
 ♂ DFS (Поиск в глубину) = Ж
 </div>
 <p class="text-slate-300 text-sm mb-4">

```

```

<div class="grid grid-cols-1 md:grid-co
 <pre class="bg-slate-950 p-3 rounded
// DFS (Рекурсия)
vector<bool> vis;
vector<vector<int>>> adj;

void dfs(int v) {
 vis[v] = true;
 for (int u : adj[v]) {
 if (!vis[u]) dfs(u);
 }
}</pre>
 <pre class="bg-slate-950 p-3 rounded
// BFS (Очередь)
queue<int> q;
q.push(start_node);
vis[start_node] = true;

while(!q.empty()) {
 int v = q.front(); q.pop();
 for (int u : adj[v]) {
 if (!vis[u]) {
 vis[u] = true;
 q.push(u);
 }
 }
}</pre>
 </div>
</details>
</div>
</div>
</section>`,
 },
 {
 id: "graph-planar-colors",
 title: "7. Графы. Планарные. Покраска",
 type: "html",
 description: "Формула Эйлера и теорема о 4 красках.
```

```

<div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 ество компонент.</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Острова
 </div>
 <p class="text-slate-300 text-sm mb-4">
 карту — остались ли серые (неисследованные)
 ь через океан — столько и компонент.</p>
 </div>
 </div>
 </div>
</div>
</section>`,
 },
 {
 id: "top-sort",
 title: "9. Графы. Топологическая сортировка",
 type: "html",
 description: "Разложение задач по порядку выполнения",
 category: "Графы",
 content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">

```



```

 <div class="absolute -top-3 left-4 border-rose-500 shadow-md">
 Связь с Билетом 12
 </div>
 <p class="text-slate-300 text-sm mb">
 SCC (Билет 10) склеивает
 </p>

 Точки сочленения (Билет 12)
 </p>
 </div>
 </div>

 <details class="bg-slate-900/40 rounded-lg">
 <summary class="p-4 font-bold text-slate-300 border-slate-700/50">
 Душный мод: Код Алгоритм Косарайю
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <p class="text-emerald-400 font-bold">
 <ol class="list-decimal list-inside">
 Запускаем DFS на исходном графе

 Разворачиваем этот список

 Переворачиваем все ребра в обратном направлении

 Идем по <code>order</code>

 </p>
 </div>
 </details>
</div>
</div>
</section>`,
 },
 {
 id: "graph-bridges",
 title: "11. Графы. Мосты",
 type: "html",
 },

```



```
<div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
```

```
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-rose-400">

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">
 <div class="text-left font-mono text-sm">
 <p><strong class="text-white">Точка
рой увеличивает число компонент связности.</p>
 <div class="p-3 bg-slate-950 rounded">

 <p class="text-xl font-bold text">
 <p class="text-xs text-slate-400">
 </div>
 </div>
 </div>
</div>
```

```
<p class="text-slate-300 text-sm">
 Важно: Это работает то
еются к точкам сочленения. То есть,
ег - это тупик со степенью 1).
</p>
```

```
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap">
 <div class="bg-slate-800 p-6 rounded-lg border">
 <div class="absolute -top-3 left-4 bg-slate-800
order-rose-500 shadow-md">
```

Аналогия: Главный перекресток

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

Представь город, где два района соединены  
её перекроют (удалят вершину) – районы будут

```

 low[v] = min(low[v], tin[to]);
 } else {
 dfs(to, v);
 low[v] = min(low[v], low[to]);
 if (low[to] >= tin[v] && p != -1)
 IS_CUTPOINT(v);
 ++children;
 }
}
if (p == -1 && children > 1)
 IS_CUTPOINT(v);
}</pre>

```

</div>

</div>

</details>

<div class="bg-indigo-950/60 p-6 rounded-xl border">

<h4 class="text-lg font-bold text-indigo-400">

<p class="text-slate-300 text-sm mb-4">

Это глубокий дуальный мост между ориент

</p>

<ul class="list-disc list-inside text-sm te

<li><strong class="text-rose-400">Точки

ong> и показывают физическую уязвимость свя

<li><strong class="text-emerald-400">SCC

ависимые "конгломераты".</li>

<li><strong>Как точка сочленения рушит

Косарайю, свернув каждую SCC в одну супер-в

(сделаем неориентированным), то любая <strong

о и есть критическая SCC! Если удалить эту

. Одно слабое звено изолирует целые касты S

</ul>

</div>

</div>

</section>`,

},

{

id: "graph-euler",

---

ФАЙЛ 28/85: src/components/ArticulationPointsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useState, useEffect } from "react";
import { Play, Pause, RotateCcw, Info } from "lucide-react";
```

```
interface Node {
 id: number;
 x: number;
 y: number;
 label: string;
}
```

```
interface Edge {
 u: number;
 v: number;
}
```

```
const nodes: Node[] = [
 { id: 0, x: 250, y: 50, label: "0" },
 { id: 1, x: 100, y: 150, label: "1" },
 { id: 2, x: 250, y: 250, label: "2" },
 { id: 3, x: 400, y: 150, label: "3" },
 { id: 4, x: 550, y: 150, label: "4" },
 { id: 5, x: 700, y: 50, label: "5" },
 { id: 6, x: 700, y: 250, label: "6" },
];
```

```
const edges: Edge[] = [
 { u: 0, v: 1 },
 { u: 1, v: 2 },
 { u: 2, v: 0 },
 { u: 2, v: 3 },
 { u: 3, v: 4 }, // 3 and 4 are articulation points (3
 { u: 4, v: 5 },
```

```

 desc,
 visited: new Set(visited),
 ap: new Set(ap),
 currentNode: u,
 tin: [...currentTin],
 low: [...currentLow],
 });
};

```

recordStep("Начало DFS (Тарьян) для поиска точек со

```

const dfs = (v: number, p: number = -1) => {
 visited.add(v);
 currentTin[v] = currentLow[v] = timer++;
 let children = 0;

 recordStep(`Заходим в вершину ${v}. tin[${v}]=${c

 for (const to of adj[v]) {
 if (to === p) continue;

 if (visited.has(to)) {
 currentLow[v] = Math.min(currentLow[v], curren
 recordStep(
 `Обратное ребро ${v}-${to}. Обновляем low[${v}
 v,
);
 } else {
 children++;
 dfs(to, v);
 currentLow[v] = Math.min(currentLow[v], curren

 recordStep(
 `Возврат в ${v} из ${to}. Обновляем low[${v}
 v,
);

 if (currentLow[to] >= currentTin[v] && p !==

```

```

 tin: [],
 low: [],
 };

 useEffect(() => {
 let interval: NodeJS.Timeout;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 interval = setInterval(() => {
 setCurrentStepIndex((prev) => prev + 1);
 }, 1500);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearInterval(interval);
 }, [isPlaying, currentStepIndex, steps.length]);

 const togglePlay = () => setIsPlaying(!isPlaying);
 const reset = () => {
 setIsPlaying(false);
 setCurrentStepIndex(0);
 };

 return (
 <div className="bg-slate-900 rounded-xl border border-
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <Info className="w-5 h-5 text-rose-400" />
 Моделирование поиска точек сочленения
 </h3>

 <div className="flex items-center gap-2 bg-slate-
 <button
 onClick={togglePlay}
 className="flex items-center gap-2 px-3 py-
 text-white"
 >
 {isPlaying ? (
 <Pause className="w-4 h-4 ml-1" />

```

```

 />
);
 }}}

 { /* Nodes */
 {nodes.map({node} => {
 const isCurrent = currentStep.currentNode === node.id
 const isVisited = currentStep.visited.has(node.id)
 const isAp = currentStep.ap.has(node.id);

 const fill = isCurrent
 ? "#3b82f6"
 : isAp
 ? "#e11d48"
 : isVisited
 ? "#10b981"
 : "#1e293b";
 const stroke = isCurrent
 ? "#60a5fa"
 : isAp
 ? "#f43f5e"
 : isVisited
 ? "#34d399"
 : "#334155";
 const r = isAp ? 24 : 20;

 return (
 <g key={node.id}>
 <circle
 cx={node.x}
 cy={node.y}
 r={r}
 fill={fill}
 stroke={stroke}
 strokeWidth="3"
 className="transition-all duration-300"
 />
 <text

```

```

 </>
 })
 </g>
);
 }}}
</svg>
</div>

<div className="bg-slate-950 p-6 border-t lg:bo
 <h4 className="text-sm font-bold text-slate-4
 Статус алгоритма
 </h4>

 <div className="bg-slate-900 border border-sl
 <p className="text-white font-medium min-h-
 {currentStep.desc}
 </p>
 </div>

 <div className="space-y-4 flex-1 overflow-y-a
 <div className="mb-4">
 <h5 className="text-xs text-slate-500 fon
 <div className="flex items-center gap-2 t
 <div className="w-3 h-3 rounded-full bg
 Точка сочленения
 </div>
 <div className="flex items-center gap-2 t
 <div className="w-3 h-3 rounded-full bg
 Посещена
 </div>
 <div className="flex items-center gap-2 t
 <div className="w-3 h-3 rounded-full bg
 Текущая
 </div>
 <div className="flex items-center gap-2 t
 <div className="w-8 h-1 bg-rose-500 bor
 Мост
 </div>
 </div>
 </div>

```

ФАЙЛ 29/85: src/components/BellmanFordViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState, useEffect } from 'react';

interface Node { id: string; x: number; y: number; name: string; }
interface Edge { u: string; v: string; w: number; }
interface Step {
 iteration: number;
 checkingEdge: { u: string; v: string; w: number } | null;
 distances: { [key: string]: number };
 previous: { [key: string]: string | null };
 log: string;
 hasNegativeCycle?: boolean;
 activeCodeLine: number;
}

export default function BellmanFordViz() {
 const [hasCycle, setHasCycle] = useState(false);
 const [steps, setSteps] = useState<Step[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useState(0);
 const [isPlaying, setIsPlaying] = useState(false);

 const nodes: Node[] = [
 { id: 'S', x: 60, y: 150, name: 'База (S)' },
 { id: 'A', x: 160, y: 60, name: 'Застава (A)' },
 { id: 'B', x: 160, y: 240, name: 'Топь 1 (B)' },
 { id: 'C', x: 260, y: 150, name: 'Мост (C)' },
 { id: 'D', x: 350, y: 60, name: 'Топь 2 (D)' },
 { id: 'E', x: 350, y: 240, name: 'Лес (E)' },
 { id: 'F', x: 450, y: 150, name: 'Склад (F)' },
];

 const getEdges = (cycle: boolean): Edge[] => {
 const defaultEdges = [
```



```

useEffect(() => {
 const simulationSteps: Step[] = [];
 let dist: { [key: string]: number } = { S: 0, A: 999 };
 let prev: { [key: string]: string | null } = { S: null };

 simulationSteps.push({
 iteration: 0, checkingEdge: null,
 distances: { ...dist }, previous: { ...prev },
 log: ' Фуря заведена. Начинаем с Базы (S). Расстояние до А: 999',
 activeCodeLine: 2,
 });

 const vCount = nodes.length;
 for (let iter = 1; iter < vCount; iter++) {
 let anyUpdate = false;
 for (const edge of edges) {
 const uDist = dist[edge.u];
 const vDist = dist[edge.v];

 let updated = false;
 if (uDist !== 999 && uDist + edge.w < vDist) {
 dist = { ...dist, [edge.v]: uDist + edge.w };
 prev = { ...prev, [edge.v]: edge.u };
 updated = true;
 anyUpdate = true;
 }

 simulationSteps.push({
 iteration: iter, checkingEdge: { u: edge.u, v: edge.v },
 distances: { ...dist }, previous: { ...prev },
 log: `Итерация ${iter}: Проверяем ${edge.u} → ${edge.v}. ${
 updated ? `Релаксация успешна! dist[${edge.v}] = ${dist[edge.v]} ` : ''
 }`,
 activeCodeLine: updated ? 6 : 5,
 });
 }
 }
}

```



```

const isNeg = edge.w < 0;
const isCyclePart = hasCycle && ((edge.u === 'D' || edge.v === 'D'));

let strokeColor = '#475569';
let marker = 'url(#arrow-bf-normal)';
if (isChecking) { strokeColor = '#f59e0b'; }
else if (currentStep.hasNegativeCycle && isCyclePart) { strokeColor = '#f43f5e'; }

return (
 <g key={idx}>
 <line x1={uNode.x} y1={uNode.y} x2={vNode.x} y2={vNode.y} stroke={strokeColor} strokeWidth={isChecking ? 4 : 2} marker={marker} />
 <circle cx={(uNode.x + vNode.x) / 2} cy={(uNode.y + vNode.y) / 2} r={isChecking ? 4 : 2} stroke={strokeColor} />
 <text x={(uNode.x + vNode.x) / 2} y={(uNode.y + vNode.y) / 2} text="Checking" fontColor={isNeg ? '#f43f5e' : '#475569'} fontSize="11" fontWeight="bold" />
 </g>
);
}}}

{nodes.map((node) => {
 const dist = currentStep.distances[node.id];
 const isCheckingNode = currentStep.checkingNode === node.id;

 return (
 <g key={node.id} className="transition-appear">
 <circle cx={node.x} cy={node.y} r={isCheckingNode ? 4 : 2} stroke={isCheckingNode ? '#f59e0b' : '#64748b'} strokeWidth={isCheckingNode ? 4 : 2} />
 <text x={node.x} y={node.y + 5} fill={isCheckingNode ? '#f59e0b' : '#64748b'} fontColor={isCheckingNode ? '#f59e0b' : '#64748b'} fontStyle="italic" fontWeight="bold" textAnchor="middle" className="label" />
 <text x={node.x} y={node.y - 28} fill={isCheckingNode ? '#f59e0b' : '#64748b'} fontColor={isCheckingNode ? '#f59e0b' : '#64748b'} fontStyle="italic" fontWeight="bold" textAnchor="middle" className="label" />
 <text x={node.x} y={node.y + 35} fill={isCheckingNode ? '#f59e0b' : '#64748b'} fontColor={isCheckingNode ? '#f59e0b' : '#64748b'} fontStyle="italic" fontWeight="bold" textAnchor="middle" className="label" />
 </g>
);
})}

```



```
const visited = new Set<number>();

// Init
steps.push({
 activeLine: 1,
 queue: [],
 visited: [],
 currentNode: null,
 logs: "Инициализация: начинаем с корня (0). "
});

queue.push(0);
visited.add(0);
steps.push({
 activeLine: 2,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: null,
 logs: "Добавляем стартовую вершину 0 в очередь и по"
});

while (queue.length > 0) {
 steps.push({
 activeLine: 4,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: null,
 logs: `Очередь не пуста, элементов: ${queue.length}
 });

 const curr = queue.shift()!;
 steps.push({
 activeLine: 5,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Достаем из очереди (${curr}).`
 });
```

```

 queue: [],
 visited: Array.from(visited),
 currentNode: null,
 logs: "Очередь пуста. Алгоритм завершен!"
 });

 return steps;
};

const STEPS = generateBfsSteps();

export default function BfsViz() {
 const [stepIdx, setStepIdx] = useState(0);
 const [isPlaying, setIsPlaying] = useState(false);

 const step = STEPS[stepIdx];

 const handleNext = useCallback(() => {
 setStepIdx(prev => Math.min(prev + 1, STEPS.length - 1));
 }, []);

 // @ts-expect-error зарезервировано под кнопку «назад»
 const handlePrev = useCallback(() => {
 setStepIdx(prev => Math.max(prev - 1, 0));
 }, []);

 useEffect(() => {
 if (!isPlaying) return;
 if (stepIdx >= STEPS.length - 1) {
 setIsPlaying(false);
 return;
 }
 const timer = setTimeout(handleNext, 1200);
 return () => clearTimeout(timer);
 }, [isPlaying, stepIdx, handleNext]);

 return (
 <div className="bg-slate-900 border-2 border-slate-100" data-bbox="100 970 1000 1000">

```

```

<svg className="w-full h-full min-h-[350px]"
 {/* Рёбра */}
 {EDGES.map((e, idx) => {
 const n1 = NODES.find(n => n.id === e.from)
 const n2 = NODES.find(n => n.id === e.to)
 const isVisited = step.visited.includes(n1)
 return (
 <line
 key={idx}
 x1={n1.x} y1={n1.y} x2={n2.x} y2={n2.y}
 className={`transition-all duration-500 ${
 isVisited ? 'stroke-slate-900' : 'stroke-slate-600'
 }`}
 />
)
 })}

```

```

{/* Вершины */}
{NODES.map(n => {
 const isCurrent = step.currentNode === n.id
 const isInQueue = step.queue.includes(n.id)
 const isVisited = step.visited.includes(n.id)

```

```

 let fill = '#1e293b'; // slate-800
 let stroke = '#475569'; // slate-600
 let scale = 1;
 void scale;

```

```

 if (isCurrent) {
 fill = '#3b82f6'; // blue-500
 stroke = '#60a5fa'; // blue-400
 scale = 1.3;
 } else if (isInQueue) {
 fill = '#059669'; // emerald-600
 stroke = '#34d399'; // emerald-400
 scale = 1.1;
 } else if (isVisited) {
 fill = '#0f172a'; // slate-900
 stroke = '#3b82f6'; // blue-500
 }
 })}

```

```

<div className="p-4">
 {[
 { line: 1, code: 'queue = Queue()' },
 { line: 2, code: 'queue.push(start); vi
 { line: 3, code: '' },
 { line: 4, code: 'while not queue.isEmp
 { line: 5, code: ' node = queue.pop()'
 { line: 6, code: ' for neighbor in gra
 { line: 7, code: ' if neighbor not in
 { line: 8, code: ' visited.add(ne
 { line: 9, code: ' queue.push(nei
]}.map((cl) => {
 const isActive = step.activeLine === cl
 return (
 <div key={cl.line} className={`px-2 p
d border-l-2 border-blue-400 -ml-0.5` : 'te
 <span className="opacity-40 w-6 inl
 {c
 </div>
);
])}
</div>
</div>

```

```

<div className="bg-slate-800/80 border border
 <div className="text-sm font-bold text-eme
 Состояние Очереди (Queue):
 <span className="text-xs text-slate-500
 </div>
 <div className="flex items-center gap-2 mi
 {step.queue.length === 0 ? (
 <span className="text-slate-500 text-x
) : (
 step.queue.map((qId, i) => (
 <span key={`${qId}-${i}`} className=
enter justify-center rounded-md shadow-lg s
 {qId}


```



```
 },
 floyd: {
 src: floydImg,
 alt: 'Все ко Всем Флойд',
 caption: ' Все связаны со всеми!',
 borderColor: 'border-emerald-500/30',
 captionColor: 'text-emerald-400',
 },
 };

export default function ChapterImage({ vizType }: { vizType: string }) {
 const info = imageMap[vizType];
 if (!info) return null;

 return (
 <div>
 <div className={`rounded-2xl overflow-hidden border-2 border-emerald-500`} >

 </div>
 <p className={`text-center text-xs mt-2 font-bold text-emerald-400`} >
 {info.caption}
 </p>
 </div>
);
}
```

---

ФАЙЛ 32/85: src/components/ChapterNav.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React from "react";
import { ChevronLeft, ChevronRight } from "lucide-react";
import type { Chapter } from "../types";
```

```
interface Props {
 prev?: Chapter;
 next?: Chapter;
 index: number;
```

```

 }

 return (
 <nav className="grid grid-cols-1 sm:grid-cols-2 gap-4">
 {prev ? (
 <button
 onClick={() => onGo(prev.id)}
 className="group text-left p-4 rounded-xl bg-white border border-gray-200 transition-all"
 >
 <div className="flex items-center gap-1.5 text-indigo-400 mb-1">
 <ChevronLeft className="w-3.5 h-3.5" /> Предыдущая тема
 </div>
 <div className="text-sm font-bold text-slate-700">
 {prev.title}
 </div>
 </button>
) : (
 <div className="hidden sm:block" />
)}

 {next && (
 <button
 onClick={() => onGo(next.id)}
 className="group text-right p-4 rounded-xl bg-white border border-gray-200 transition-all sm:col-start-2"
 >
 <div className="flex items-center justify-end text-indigo-400 mb-1">
 Следующая тема <ChevronRight className="w-3.5 h-3.5" />
 </div>
 <div className="text-sm font-bold text-slate-700">
 {next.title}
 </div>
 </button>
)}
 </nav>
);
 };
};

```

```

 window.clearTimeout(hideTimer.current);
 hideTimer.current = null;
 }
}, []));

const scheduleHide = useCallback(() => {
 cancelHide();
 hideTimer.current = window.setTimeout(() => setAnchor(
}, [cancelHide]);

useEffect(() => {
 setAnchor(null);
 setSaving("idle");
}, [chapter.id]);

const saveHtml = useCallback(async () => {
 setSaving("work");
 try {
 await downloadChapterHtml(chapter);
 setSaving("done");
 window.setTimeout(() => setSaving("idle"), 2500);
 } catch {
 setSaving("idle");
 }
}, [chapter]);

const open = useCallback(
 (el: HTMLElement) => {
 const termId = el.dataset.termId;
 if (!termId) return;
 cancelHide();
 setAnchor({ termId, rect: el.getBoundingClientRect()
 },
 [cancelHide]
);

const handlePointerOver = (e: React.MouseEvent) => {
 if (isTouch()) return;

```

```

 <Loader2 className="w-3.5 h-3.5 animate-s
) : saving === "done" ? (
 <Check className="w-3.5 h-3.5 text-emeral
) : (
 <Download className="w-3.5 h-3.5" />
)}
 {saving === "done" ? "Файл сохранён" : "Ска
</button>
<button
 onClick={() => window.print()}
 title="Распечатать или сохранить в PDF сред
 className="flex items-center gap-1.5 text-x
 300 hover:text-white hover:border-indigo-50
>
 <Printer className="w-3.5 h-3.5" /> Печать
</button>

</div>

<div
 ref={contentRef}
 className="prose prose-invert max-w-none"
 onMouseOver={handlePointerOver}
 onMouseOut={handlePointerOut}
 onClick={handleClick}
 onFocus={handleFocus}
 dangerouslySetInnerHTML={{ __html: chapter.co
/>

<p className="mt-6 flex items-start gap-2 text-
ed-lg px-3 py-2">
 <MousePointerClick className="w-4 h-4 shrink-

 Подчёркнутые термины – интерактивные: навед
 объяснение на пальцах, мини-анимацию и кноп

</p>

```

```

 onClose={() => setAnchor(null)}
 onOpenSimulator={() => {
 setAnchor(null);
 onOpenSimulator(id);
 }}
 onCardEnter={cancelHide}
 onCardLeave={scheduleHide}
 />
</>
});
};

```

ФАЙЛ 34/85: src/components/DfsBridgesSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import { useState, useEffect, useRef } from "react";
import { Undo, Play, Pause, SkipForward, RefreshCw } from "lucide-react";

```

```

interface CodeLine {
 line: string;
 highlight: string;
}

```

```

const CODE_LINES: CodeLine[] = [
 { line: "void dfs(int v, int p = -1) {", highlight: "normal" },
 { line: " visited[v] = true;", highlight: "normal" },
 { line: " tin[v] = low[v] = timer++;", highlight: "normal" },
 { line: " for (int to : adj[v]) {", highlight: "normal" },
 { line: " if (to == p) continue;", highlight: "normal" },
 { line: " if (visited[to]) {", highlight: "normal" },
 { line: " low[v] = min(low[v], tin[to]);", highlight: "normal" },
 { line: " } else {", highlight: "normal" },
 { line: " dfs(to, v);", highlight: "normal" },

```

```

 { u: 3, v: 4, key: "3-4" },
 { u: 4, v: 5, key: "4-5" },
 { u: 5, v: 3, key: "3-5" },
 { u: 1, v: 6, key: "1-6" }
];

const DFS_STEPS: DfsStep[] = [
 {
 currentNode: 0, visitedIndices: [0],
 tin: {0:1}, low: {0:1}, stack: [0], bridges: [],
 activeEdge: null, backEdge: null,
 log: "Входим в A. tin[A]=1, low[A]=1.",
 activeCodeLine: [0,1,2]
 },
 {
 currentNode: 1, visitedIndices: [0,1],
 tin: {0:1,1:2}, low: {0:1,1:2}, stack: [0,1], bridges: [],
 activeEdge: [0,1], backEdge: null,
 log: "Идём (A,B). В впервые. tin[B]=2, low[B]=2.",
 activeCodeLine: [4,5,7,10,11]
 },
 {
 currentNode: 6, visitedIndices: [0,1,6],
 tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1,6],
 activeEdge: [1,6], backEdge: null,
 log: "Из B идём в G. tin[G]=3, low[G]=3.",
 activeCodeLine: [4,5,7,10,11]
 },
 {
 currentNode: 1, visitedIndices: [0,1,6],
 tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1,6],
 activeEdge: null, backEdge: null,
 log: "У G нет соседей (кроме B). Возврат. low[G]=3 : ",
 activeCodeLine: [13,14,15]
 },
 {
 currentNode: 2, visitedIndices: [0,1,6,2],
 tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:4}, stack: [0,1,6,2],
 activeEdge: null, backEdge: null,
 log: "Идём (A,C). В впервые. tin[C]=4, low[C]=4.",
 activeCodeLine: [17,18,20,23,24]
 },
 {
 currentNode: 3, visitedIndices: [0,1,6,2,3],
 tin: {0:1,1:2,6:3,2:4,3:5}, low: {0:1,1:2,6:3,2:4,3:5}, stack: [0,1,6,2,3],
 activeEdge: [2,3], backEdge: null,
 log: "Идём (A,D). В впервые. tin[D]=5, low[D]=5.",
 activeCodeLine: [26,27,29,32,33]
 },
 {
 currentNode: 4, visitedIndices: [0,1,6,2,3,4],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6}, low: {0:1,1:2,6:3,2:4,3:5,4:6}, stack: [0,1,6,2,3,4],
 activeEdge: [3,4], backEdge: null,
 log: "Идём (A,E). В впервые. tin[E]=6, low[E]=6.",
 activeCodeLine: [35,36,38,41,42]
 },
 {
 currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, stack: [0,1,6,2,3,4,5],
 activeEdge: [4,5], backEdge: null,
 log: "Идём (A,F). В впервые. tin[F]=7, low[F]=7.",
 activeCodeLine: [44,45,47,50,51]
 },
 {
 currentNode: 6, visitedIndices: [0,1,6,2,3,4,5,6],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7,6:8}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7,6:8}, stack: [0,1,6,2,3,4,5,6],
 activeEdge: [5,6], backEdge: null,
 log: "Идём (A,G). В впервые. tin[G]=8, low[G]=8.",
 activeCodeLine: [53,54,56,59,60]
 },
 {
 currentNode: 5, visitedIndices: [0,1,6,2,3,4,5,6],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7,6:8}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7,6:8}, stack: [0,1,6,2,3,4,5,6],
 activeEdge: null, backEdge: [5,6],
 log: "Выходим из G. low[G]=8.",
 activeCodeLine: [62,63,65,68,69]
 },
 {
 currentNode: 4, visitedIndices: [0,1,6,2,3,4,5,6],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7,6:8}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7,6:8}, stack: [0,1,6,2,3,4,5,6],
 activeEdge: null, backEdge: null,
 log: "Выходим из F. low[F]=7.",
 activeCodeLine: [71,72,74,77,78]
 },
 {
 currentNode: 3, visitedIndices: [0,1,6,2,3,4,5,6],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7,6:8}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7,6:8}, stack: [0,1,6,2,3,4,5,6],
 activeEdge: null, backEdge: null,
 log: "Выходим из E. low[E]=6.",
 activeCodeLine: [80,81,83,86,87]
 },
 {
 currentNode: 2, visitedIndices: [0,1,6,2,3,4,5,6],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7,6:8}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7,6:8}, stack: [0,1,6,2,3,4,5,6],
 activeEdge: null, backEdge: null,
 log: "Выходим из D. low[D]=5.",
 activeCodeLine: [89,90,92,95,96]
 },
 {
 currentNode: 1, visitedIndices: [0,1,6,2,3,4,5,6],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7,6:8}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7,6:8}, stack: [0,1,6,2,3,4,5,6],
 activeEdge: null, backEdge: null,
 log: "Выходим из C. low[C]=4.",
 activeCodeLine: [98,99,101,104,105]
 },
 {
 currentNode: 0, visitedIndices: [0,1,6,2,3,4,5,6],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7,6:8}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7,6:8}, stack: [0,1,6,2,3,4,5,6],
 activeEdge: null, backEdge: null,
 log: "Выходим из A. low[A]=1.",
 activeCodeLine: [107,108,110,113,114]
 }
];

```

```

 },
 {
 currentNode: 4, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
 activeEdge: null, backEdge: null,
 log: "Возврат F→E. low[E]=min(6,5)=5. (E,F) – не мост,
 activeCodeLine: [11,13,16,17]
 },
 {
 currentNode: 3, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
 activeEdge: null, backEdge: null,
 log: "Возврат E→D. low[D]=min(6,5)=5. (D,E) – не мост,
 activeCodeLine: [11,13,16,17]
 },
 {
 currentNode: 2, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
 activeEdge: null, backEdge: null,
 log: "Возврат D→C. low[5] = 5 > tin[2] = 4! (C,D) – мост,
 activeCodeLine: [13,14,15,16,17]
 },
 {
 currentNode: 1, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:3,2:4,3:5,4:6,5:7},
 activeEdge: null, backEdge: null,
 log: "Возврат C→B. low[B]=min(low[B],low[C])=1. (B,C) – мост,
 activeCodeLine: [11,13,16,17]
 },
 {
 currentNode: 0, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:3,2:4,3:5,4:6,5:7},
 activeEdge: null, backEdge: null,
 log: "Возврат B→A. DFS завершён! Мосты: (B,G) и (C,F),
 activeCodeLine: [11,13,16,17,18]
 },
 {
 currentNode: null, visitedIndices: [0,1,6,2,3,4,5],

```

```

 dfsAuto ? "bg-rose-600 text-white" : "bg-
 }` }>
 {dfsAuto ? <><Pause className="h-3 w-3" />
 </button>
 <button onClick={() => { setDfsAuto(false); s
 disabled={dfsIdx === DFS_STEPS.length-1}
 className="p-1.5 hover:bg-slate-700 rounded
 <SkipForward className="h-3.5 w-3.5" />
 </button>
 <button onClick={() => { setDfsAuto(false); s
 className="p-1.5 hover:bg-slate-700 rounded
 <RefreshCw className="h-3.5 w-3.5" />
 </button>
</div>
</div>

<div className="grid grid-cols-1 md:grid-cols-5 g
 { /* Graph visualization */ }
 <div className="md:col-span-3 bg-slate-950 roun
 <div className="absolute inset-0 bg-[radial-g
 >
 <svg className="w-full h-[280px] z-10 relativ
 {GRAPH_EDGES.map(e => {
 const u = GRAPH_NODES.find(n => n.id ===
 const v = GRAPH_NODES.find(n => n.id ===
 const bridge = step.bridges.includes(e.ke
 const activeTree = step.activeEdge && (
 (step.activeEdge[0]===e.u && step.activ
 (step.activeEdge[0]===e.v && step.activ
));
 const activeBack = step.backEdge && (
 (step.backEdge[0]===e.u && step.backEdg
 (step.backEdge[0]===e.v && step.backEdg
));
 let sc = "#475569", sw = 2, dash = "none"
 if (bridge) { sc = "#eab308"; sw = 4; }
 else if (activeTree) { sc = "#10b981"; sw
 else if (activeBack) { sc = "#f43f5e"; sw

```



```

 <div>
 </div>
 </div>
</div>

{/* Code panel + stack */}
<div className="md:col-span-2 space-y-3">
 {/* Code panel */}
 <div className="bg-slate-950 rounded-lg border" style={{borderColor: 'black', borderStyle: 'solid', borderWidth: 1px}}>
 <div className="bg-slate-900 px-3 py-1.5 text-slate-400 font-mono text-sm">
 >
 <div className="p-2 overflow-x-auto">
 <pre className="text-[10px] font-mono leading-relaxed">
 {CODE_LINES.map(({cl, i}) => {
 const hl = step.activeCodeLine.includes(cl);
 return <div key={i}
 className={`px-1 transition-all duration-200 ${
 hl ? "bg-indigo-600/20 text-indigo-400" : "text-slate-400"
 }`}
 style={{border: 1px solid transparent, borderImage: step.activeCodeLine === cl ? "linear-gradient(to right, transparent 49%, #000000 49%, #000000 51%, #ffffff 51%) 1" : "none"}}>
 {cl.line} || ' '

 </div>;
 })}
 </pre>
 </div>
 </div>
 </div>

 {/* Stack */}
 <div className="bg-slate-900/50 rounded-xl border" style={{borderColor: 'black', borderStyle: 'solid', borderWidth: 1px}}>
 <div className="text-[10px] font-bold text-slate-400">
 Cтек DFS
 {step.stack.length}
 </div>
 <div className="flex flex-col-reverse gap-1">
 {step.stack.length === 0
 ? <div className="text-[10px] text-slate-400">Пустой стек
 : step.stack.map((id, i) => {
 <div key={id} className={`px-2 py-0.5 text-[10px] text-slate-400 font-mono font-normal">
 {id}
 </div>
 })
 }
 </div>
 </div>

```

```

 currentNode: string | null;
 checkingEdge: { u: string; v: string } | null;
 distances: { [key: string]: number };
 visited: { [key: string]: boolean };
 previous: { [key: string]: string | null };
 log: string;
 activeCodeLine: number;
}

export default function DijkstraViz() {
 const nodes: Node[] = [
 { id: 'A', x: 60, y: 150, name: 'Пиццерия (A)' },
 { id: 'B', x: 160, y: 60, name: 'Дом 1 (B)' },
 { id: 'C', x: 160, y: 240, name: 'Парк (C)' },
 { id: 'D', x: 340, y: 60, name: 'Офис (D)' },
 { id: 'E', x: 340, y: 240, name: 'Дом 2 (E)' },
 { id: 'F', x: 450, y: 150, name: 'ТЦ (F)' },
 { id: 'G', x: 250, y: 150, name: 'Метро (G)' },
];

 const edges: Edge[] = [
 { u: 'A', v: 'B', w: 4 },
 { u: 'A', v: 'C', w: 2 },
 { u: 'C', v: 'B', w: 1 },
 { u: 'C', v: 'G', w: 3 },
 { u: 'C', v: 'E', w: 8 },
 { u: 'B', v: 'G', w: 4 },
 { u: 'B', v: 'D', w: 5 },
 { u: 'G', v: 'D', w: 1 },
 { u: 'G', v: 'E', w: 2 },
 { u: 'D', v: 'F', w: 3 },
 { u: 'E', v: 'F', w: 1 },
];

 const adjList: { [key: string]: { v: string; w: number } } = {};
 nodes.forEach(n => (adjList[n.id] = []));
 edges.forEach(e => {
 adjList[e.u].push({ v: e.v, w: e.w });
 });
}

```

```

 minD = dist[n.id];
 u = n.id;
 }
}

if (!u) break;

visited[u] = true;
simulationSteps.push({
 currentNode: u, checkingEdge: null,
 distances: { ...dist }, visited: { ...visited },
 log: ` Выбираем вершину ${u} с мин. непосещённой
 activeCodeLine: 5,
});

const neighbors = edges.filter(e => e.u === u);
for (const edge of neighbors) {
 const v = edge.v;
 if (visited[v]) continue;

 simulationSteps.push({
 currentNode: u, checkingEdge: { u, v },
 distances: { ...dist }, visited: { ...visited },
 log: ` Смотрим соседа ${v}. Текущее расстояние
 ${dist[u] + edge.w}.`,
 activeCodeLine: 8,
 });

 if (dist[u] + edge.w < dist[v]) {
 dist[v] = dist[u] + edge.w;
 prev[v] = u;
 simulationSteps.push({
 currentNode: u, checkingEdge: { u, v },
 distances: { ...dist }, visited: { ...visited },
 log: ` Улучшение (Релаксация)! Новое кратчайшее
 activeCodeLine: 9,
 });
 }
}

```

```

 <h3 className="text-2xl font-bold text-white">
 <p className="text-sm text-indigo-300">Полн
 </div>
</div>

<div className="flex items-center gap-2">
 <button
 onClick={() => { setIsPlaying(false); setCu
 className="flex items-center gap-1 bg-slate
 ion-colors"
 >
 Сброс
 </button>
 <button
 onClick={() => setIsPlaying(!isPlaying)}
 className={`flex items-center gap-1 px-4 py-2
 isPlaying ? 'bg-amber-600 hover:bg-amber-500' : 'bg-indigo-600 hover:bg-indigo-500'
 }`
 >
 {isPlaying ? 'Пауза' : '▶ Пуск'}
 </button>
 <button
 onClick={() => { setIsPlaying(false); if (c
 disabled={currentStepIndex >= steps.length
 className="flex items-center gap-1 bg-indigo-600
 px-3 py-2 rounded-lg text-sm font-medium tr
 >
 ⏮
 </button>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6 ml-6">
 { /* Граф */ }
 <div className="w-full lg:w-2/3 bg-indigo-950/20
 stify-center min-h-[400px]">
 <div className="absolute top-3 left-3 bg-indigo-950/20
 ont-bold shadow-sm">

```

```

 strokeDasharray={isChecking ? '4,4'
 />
 <circle
 cx={({uNode.x + vNode.x) / 2} cy={({uNode.y + vNode.y) / 2}
 r="12" fill="#1e293b"
 stroke={isChecking ? '#f59e0b' : '#f87192'}
 />
 <text
 x={({uNode.x + vNode.x) / 2} y={({uNode.y + vNode.y) / 2}
 fill={isChecking ? '#f59e0b' : '#f87192'}
 fontSize="11" fontWeight="bold" text={edge.w}
 >
 {edge.w}
 </text>
 </g>
);
}})

{/* Вершины */}
{nodes.map((node) => {
 const isCurrent = currentStep.currentNode === node.id
 const isVisited = currentStep.visited[node.id]
 const dist = currentStep.distances[node.id]

 return (
 <g key={node.id} className="transition-all duration-300">
 <circle
 cx={node.x} cy={node.y}
 r={isCurrent ? 22 : 18}
 fill={isCurrent ? '#4f46e5' : isVisited ? '#f87192' : '#1e293b'}
 stroke={isCurrent ? '#a5b4fc' : isVisited ? '#f59e0b' : '#f87192'}
 strokeWidth={isCurrent ? 4 : 2}
 className="transition-all duration-300"
 />
 <text x={node.x} y={node.y + 5} fill="#1e293b">
 {node.id}
 </text>
 <text x={node.x} y={node.y - 28} fill="#1e293b">
 {node.dist}
 </text>
 </g>
)
})}
```

```

 <span key={idx} className={`px-3 py-2
 isCheckingThis ? 'bg-amber-500' : 'bg-white'
 600 text-slate-300'
 }`}>
 {edge.v}<span cl
 <span className={isCheckingTh
 an>

 });
 }}}
 </div>
</div>
);
}}
</div>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
 /* Таблица расстояний */
 <div className="w-full lg:w-1/2 bg-rose-950/10">
 <div>
 <h4 className="text-lg font-bold text-rose-950">
 <div className="overflow-x-auto">
 <table className="w-full text-left border-collapse">
 <thead>
 <tr className="border-b border-rose-950">
 <th className="py-2 px-3">Вершина</th>
 <th className="py-2 px-3">Расстояние</th>
 <th className="py-2 px-3">Из (пред.
 </tr>
 </thead>
 <tbody className="text-sm">
 {nodes.map((n) => {
 const dist = currentStep.distances[n.i]
 const prev = currentStep.previous[n.i]
 const vis = currentStep.visited[n.i]
 const isCurr = currentStep.currentN

```

```
 }
 </div>
);
 }}}
 </div>
</div>
</div>
</div>
</div>
);
}
```

---

ФАЙЛ 36/85: src/components/DPVisualizer.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import { useState, useEffect } from 'react';
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRight } from 'lucide-react';
```

```
interface DPStep {
 id: number;
 n: number;
 action: 'call' | 'memo_hit' | 'base_case' | 'calc' | 'end';
 desc: string;
 codeLine: number;
 memoState: { [key: number]: number };
}
```

```
const DP_CODE_LINES = [
 /* 1 */ "function fibMemo(n, memo = {}) {",
 /* 2 */ " if (n in memo) return memo[n];",
 /* 3 */ " if (n <= 2) return 1;",
 /* 4 */ " memo[n] = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);",
 /* 5 */ " return memo[n];",
 /* 6 */ "}"
];
```

```

 id: stepId++,
 n,
 action: 'base_case',
 desc: `Базовый случай: n <= 2 (n=${n}). Возвр
 codeLine: 3,
 memoState: { ...memoMap }
 });
 return 1;
}

generatedSteps.push({
 id: stepId++,
 n,
 action: 'call',
 desc: `Вычисляем рекурсивно: fibМемо(${n - 1})
 codeLine: 4,
 memoState: { ...memoMap }
});

const res1 = simulateFib(n - 1);
const res2 = simulateFib(n - 2);
memoMap[n] = res1 + res2;

generatedSteps.push({
 id: stepId++,
 n,
 action: 'calc',
 desc: `Сохраняем в кэш: мемо[${n}] = ${res1} +
 codeLine: 5,
 memoState: { ...memoMap }
});

return memoMap[n];
}

const finalRes = simulateFib(targetN);

generatedSteps.push({
```



```

 </div>
 <p className="text-sm text-slate-400">
 Смотри, как таблица DP кэширует результаты
 </p>
 </div>

 <div className="flex flex-wrap items-center gap-2">
 <div className="flex items-center gap-2 bg-slate-950">
 N:
 <select
 value={targetN}
 onChange={(e) => setTargetN(Number(e.target.value))}
 disabled={isPlaying}
 className="bg-transparent text-white font-mono"
 >
 <option value={4}>N = 4</option>
 <option value={5}>N = 5</option>
 <option value={6}>N = 6</option>
 <option value={7}>N = 7</option>
 </select>
 </div>

 <div className="flex items-center bg-slate-950">
 <button
 onClick={() => { setIsPlaying(false); setTargetN(4); }}
 className="p-2 text-slate-400 hover:text-white"
 title="Сбросить"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
 <button
 onClick={() => { setIsPlaying(false); setTargetN(4); setStepIndex(0); }}
 disabled={currentStepIndex === 0}
 className="p-2 text-slate-400 hover:text-white"
 title="Шаг назад"
 >
 <ChevronLeft className="w-5 h-5" />
 </button>
 </div>
 </div>

```



```


 </div>
 <pre className="p-6 font-mono text-sm space-x-2">
 {DP_CODE_LINES.map((line, idx) => {
 const lineNum = idx + 1;
 const isActive = currentStep?.codeLine === lineNum;
 return (
 <div
 key={lineNum}
 className={`flex items-center px-4 py-2 ${
 isActive
 ? 'bg-emerald-600/20 border-l-4 border-emerald-500'
 : 'text-slate-300 hover:bg-slate-800'
 }`}
 >
 {lineNum}
 {line}
 </div>
);
 })}
 </pre>
 <div className="p-6 bg-slate-900/50 border-t border-slate-800">

 {currentStep?.desc}
 </div>
</div>
)}
</div>

```

```

{/* Progress Bar */}
<div className="mt-8 pt-6 border-t border-slate-800">
 <div className="flex items-center justify-between">
 Прогресс выполнения
 Шаг {currentStepIndex + 1} из {steps.length}
 </div>
 <div className="h-2 bg-slate-950 rounded-full overflow-hidden">
 <div

```

```

const resetC1 = () => {
 setC1Path([]); setC1VisitedEdges([]); setC1Msg("Кли");
};

const clickC1 = (vId: number) => {
 if (c1Done && c1Path.length > 0) return;
 if (c1Path.length === 0) {
 setC1Path([vId]);
 setC1Msg("Старт! Кликай соседние вершины, чтобы пр");
 return;
 }
 const last = c1Path[c1Path.length - 1];
 const edge = C1_EDGES.find(e => (e.u === last && e.v === vId));
 if (!edge) {
 setC1Msg("Нет ребра между этими вершинами! Выбери");
 return;
 }
 const key = `${Math.min(last, vId)}-${Math.max(last, vId)}`;
 if (c1VisitedEdges.includes(key)) {
 setC1Msg("Это ребро уже пройдено! Эйлер не прощает");
 setC1Done(true); return;
 }
 const nextEdges = [...c1VisitedEdges, key];
 const nextPath = [...c1Path, vId];
 setC1VisitedEdges(nextEdges);
 setC1Path(nextPath);

 if (nextEdges.length === C1_EDGES.length) {
 const startDegree = C1_EDGES.filter(e => e.u === nextPath[0]).length;
 const endDegree = C1_EDGES.filter(e => e.v === nextPath[nextPath.length - 1]).length;
 const isCycle = startDegree % 2 === 0 && endDegree % 2 === 0;
 if (isCycle && nextPath[0] === vId) {
 setC1Msg(" ЭЙЛЕРОВ ЦИКЛ! Все рёбра пройдены, ф");
 } else {
 setC1Msg(`    ЭЙЛЕРОВ ПУТЬ! Все рёбра пройдены! С`);
 }
 setC1Done(true);
 }
};

```

```

 className="transition-all duration-200"
 <text x={n.x} y={n.y+4} textAnchor="middle"
 bel}</text>
 </g>;
 }
 </svg>
 <div className="absolute top-2 left-2 bg-slate-500
 Pebep: {c1VisitedEdges.length}/{C1_EDGES.length}
 </div>
</div>

<div className={`p-3 rounded-xl border text-sm ${
 c1Done && c1VisitedEdges.length === C1_EDGES.length
 ? "bg-emerald-950/40 border-emerald-500/30 text-emerald-500"
 : c1Done
 ? "bg-rose-950/40 border-rose-500/30 text-rose-500"
 : "bg-slate-900/50 border-slate-700 text-slate-300"}
 >
 {c1Msg}
</div>
</div>
);
}

```

ФАЙЛ 38/85: src/components/ExpressQuiz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import React, { useState } from 'react';
import { CheckCircle2, XCircle, Award, RotateCcw, AlertTriangle } from 'lucide-react';

```

```

interface Question {
 id: number;
 question: string;
 options: string[];
 correctAnswer: number;
}

```

```

 'Он позволяет выполнять шаги слияния колхозов (колхозы),
 'Он был разработан в Токио и поэтому самый быстрый
],
 correctAnswer: 2,
 explanation: 'Абсолютно верно! В Боровке каждая компания
 делить вычисления на GPU или многопроцессорных кластерах.
},
{
 id: 4,
 question: 'В чем главная суть Динамического программирования?
 options: [
 'Писать код очень быстро, динамично нажимая на клавиши.
 'Запоминать (кешировать) результаты уже решенных задач.
 'Использовать динамическую типизацию как в Python.
 'Постоянно менять требования к проекту в процессе разработки.
],
 correctAnswer: 1,
 explanation: 'Точно! Главный принцип ДП – мемоизация.
},
{
 id: 5,
 question: 'Какая структура данных идеально помогает реализовать
 options: [
 'СНМ (Система непересекающихся множеств / Disjoint Set).
 'Стек (LIFO).',
 'Красно-черное дерево.',
 'Хэш-таблица со списками коллизий.'
],
 correctAnswer: 0,
 explanation: 'Блестяще! СНМ (DSU) позволяет за практический
 время решить задачу.
 },
];

```

```

export const ExpressQuiz: React.FC = () => {
 const [currentQIndex, setCurrentQIndex] = useState<number>(0);
 const [selectedOption, setSelectedOption] = useState<string>('');
 const [isAnswered, setIsAnswered] = useState<boolean>(false);

```

```

 <Award className="w-10 h-10 text-white" />
 </div>
 <h3 className="text-3xl font-bold text-white mb-6">Ты проделал большую работу!</h3>
 <p className="text-slate-400 text-sm mb-6">Ты набрал {score} из {quizQuestions.length} баллов.
 </p>
 <div className="bg-slate-950 border border-slate-800 p-6">
 <p className="text-slate-400 text-sm uppercase font-medium">Результаты теста</p>
 <p className="text-5xl font-black text-indigo-300">{score}</p>
 <p className="text-slate-300 text-sm">
 {score === quizQuestions.length
 ? ' Идеально! Ты настоящий сеньор-раскраш!'
 : score >= 3
 ? ' Отличный результат! Главные концепции ты освоил!'
 : ' Стоит повторить главы пособия и еще раз пройти тест.'
 }
 </p>
 </div>
 <button
 onClick={handleRestart}
 className="inline-flex items-center gap-2 px-4 py-2 rounded-xl shadow-lg shadow-indigo-600/30"
 >
 <RotateCcw className="w-5 h-5" />
 Пройти тест заново
 </button>
 </div>
);
}

return (
 <div className="bg-slate-900/90 border border-slate-800 p-6">
 <div className="flex items-center justify-between">
 <div className="flex items-center gap-3">
 Экспресс-тест
 <h3 className="text-xl font-bold text-white">Ты проделал большую работу!</h3>
 </div>
 </div>
 </div>
);
}

```

```

) : {
 <div className={"w-6 h-6 rounded-full
order-indigo-500 bg-indigo-500 text-white"
 {String.fromCharCode(65 + idx)}}
 </div>
 }}
 </div>
 <div className="flex-1 text-sm md:text-l
 {option}
 </div>
</div>
);
 }}}
</div>
</div>

{isAnswered && (
 <div className="bg-slate-950 border border-slate-
 <div className="flex items-center gap-2 mb-2
 <AlertCircle className="w-4 h-4" />
 Объяснение:
 </div>
 <p className="text-slate-300 text-sm leading-
 {currentQ.explanation}
 </p>
 </div>
)}

<div className="flex justify-end pt-4 border-t bo
 <button
 onClick={handleNext}
 disabled={!isAnswered}
 className={
 "px-8 py-3.5 font-bold rounded-xl transition
 (!isAnswered
 ? "bg-slate-800 text-slate-500 border bor
 : "bg-indigo-600 hover:bg-indigo-500 activi
 }
 </button>

```



```

const initialMatrix = [
 [0, 4, 999, 2, 999, 999, 999],
 [999, 0, 3, 999, 999, 3, 999],
 [999, 999, 0, 999, 2, 999, 999],
 [999, 999, 999, 0, 4, 2, 999],
 [999, 999, 999, 999, 0, 999, 1],
 [999, 999, 999, 999, 999, 0, 5],
 [999, 1, 999, 999, 999, 999, 0]
];

const adjList: { [key: number]: { v: number; w: number } } = {
 0: [{ v: 1, w: 4 }, { v: 3, w: 2 }],
 1: [{ v: 2, w: 3 }, { v: 5, w: 3 }],
 2: [{ v: 4, w: 2 }],
 3: [{ v: 4, w: 4 }, { v: 5, w: 2 }],
 4: [{ v: 6, w: 1 }],
 5: [{ v: 6, w: 5 }],
 6: [{ v: 1, w: 1 }],
};

const codeLines = [
 { line: 1, text: 'function floyd_warshall(matrix, V' },
 { line: 2, text: ' dist = copy(matrix)' },
 { line: 3, text: ' for k from 0 to V - 1: // Пос' },
 { line: 4, text: ' for i from 0 to V - 1:' },
 { line: 5, text: ' for j from 0 to V - 1' },
 { line: 6, text: ' if dist[i][k] + d' },
 { line: 7, text: ' dist[i][j] = ' },
 { line: 8, text: ' return dist' },
];

useEffect(() => {
 const simulationSteps: Step[] = [];
 const dist = initialMatrix.map(row => [...row]);

 simulationSteps.push({
 k: -1, i: -1, j: -1, matrix: dist.map(r => [...r]),
 log: ' Инициализация матрицы прямых путей, ∞ (999)'
 });
});

```

```

 setIsPlaying(false);
 }, []));

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => setCurrentStepIndex((p) => p + 1), 1000);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const currentStep = steps[currentStepIndex] || null;
if (!currentStep) return <div>Загрузка...</div>;

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border border-slate-700">
 <div className="flex flex-wrap items-center justify-between">
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-emerald-600 text-white rounded">

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white">Загрузка...</h3>
 <p className="text-sm text-emerald-300">Получите доступ к этому контенту</p>
 </div>
 </div>

 <div className="flex items-center gap-2">
 <button onClick={() => { setIsPlaying(false); }} className="text-sm"> Остановить</button>
 <button onClick={() => setIsPlaying(!isPlaying)} className="text-sm bg-amber-600 text-white"> {isPlaying ? 'Пауза' : 'Воспроизвести'}</button>
 <button onClick={() => { setIsPlaying(false); }} className="px-3 py-2 bg-emerald-600 text-white"> Остановить</button>
 </div>
 </div>
 </div>
);

```

```

 });
 });
 })
 {nodesSvg.map((node) => {
 const isK = currentStep.k === node.id;
 const isI = currentStep.i === node.id;
 const isJ = currentStep.j === node.id;
 let fill = '#1e293b'; let stroke = '#64748f';
 if (isK) { fill = '#4f46e5'; stroke = '#a8a2be'; }
 else if (isI) { fill = '#b45309'; stroke = '#c9a04d'; }
 else if (isJ) { fill = '#047857'; stroke = '#2e8b57'; }
 return (
 <g key={node.id} className="transition-colors">
 <circle cx={node.x} cy={node.y} r={isJ ? 4 : 2} className="transition-colors duration-200">
 <text x={node.x} y={node.y + 5} fill="white">{node.k}</text>
 <text x={node.x} y={node.y + 32} fill="white">{node.i}</text>
 </g>
);
 });
 })
 </svg>

 <div className="w-full bg-slate-950/80 p-4 rounded">
 <p className="font-semibold text-amber-400">{currentStep.k}</p>
 <p className="min-h-[40px] flex items-center">{currentStep.i}</p>
 </div>
</div>

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/80 p-4 rounded">
 <h4 className="text-lg font-bold text-emerald-400">Смежность</h4>
 <div className="space-y-2 text-sm font-mono">
 {Object.keys(adjList).map((uStr) => {
 const u = parseInt(uStr);
 const isI = currentStep.i === u;
 const isK = currentStep.k === u;
 return (

```

```

 {currentStep.matrix.map((row, i) => (
 <tr key={i} className="border-b borde
 <td className={`p-2 bg-rose-950/40
tStep.k === i ? 'text-amber-400' : ''}`}>
 {nodeNames[i].split(' ')[0]} {i}
 </td>
 {row.map((val, j) => {
 const isUpd = currentStep.i === i
 const isVia = currentStep.k !== -
tep.j === j});
 let bg = 'bg-slate-900/40 text-sl
 if (isUpd) bg = 'bg-emerald-900/8
 else if (isVia) bg = 'bg-indigo-9
 else if (i === j) bg = 'bg-slate-
 return (<td key={j} className={`p
 /td>});
 }}}
 </tr>
 }}}
</tbody>
</table>
</div>
</div>

```

```

{/* Код алгоритма */}
<div className="w-full lg:w-1/2 bg-slate-900/60
 <div>
 <h4 className="text-lg font-bold text-slate
 <div className="bg-slate-950/80 rounded-lg p
 {codeLines.map(item => (
 <div key={item.line} className={`py-1.5
e === item.line ? 'bg-blue-900/80 text-ambe
 <span className="text-slate-600 w-5 s
 <span className="whitespace-pre flex-
 </div>
)}}
 </div>
</div>

```

```

 'D': ['B'],
 'E': ['B'],
 'F': ['C'],
 'G': ['C'],
};

type Action = {
 type: 'visit' | 'process' | 'backtrack';
 node: string;
 desc: string;
 queueOrStack?: string[];
 visitedNodes?: string[];
};

function generateDFSSteps(start: string) {
 const steps: Action[] = [];
 const vis = new Set<string>();
 const stack: string[] = []; // for visualization, j

 function dfs(v: string) {
 vis.add(v);
 stack.push(v);
 steps.push({ type: 'visit', node: v, desc: `Замечено: ${v}`,
 visitedNodes: Array.from(vis) });

 for (const u of adj[v]) {
 if (!vis.has(u)) {
 steps.push({ type: 'process', node: v, desc: `Обработка: ${v}`,
 queueOrStack: [...stack], visitedNodes: Array.from(vis) });
 dfs(u);
 }
 }

 stack.pop();
 steps.push({ type: 'backtrack', node: v, desc: `Откат: ${v}`,
 queueOrStack: [...stack], visitedNodes: Array.from(vis) });
 }
}

```

```

const [mode, setMode] = useState<"dfs" | "bfs">("dfs");
const [autoPlay, setAutoPlay] = useState(false);
const [stepIdx, setStepIdx] = useState(0);

const steps = useMemo(() => mode === 'dfs' ? generateDfsSteps(
 graph, startNode, endNode, visitedNodes, stepIdx) : generateBfsSteps(
 graph, startNode, endNode, visitedNodes, stepIdx), [mode]);

useEffect(() => {
 setStepIdx(0);
 setAutoPlay(false);
}, [mode]);

useEffect(() => {
 if (autoPlay) {
 const t = setInterval(() => {
 setStepIdx(p => {
 if (p >= steps.length - 1) {
 setAutoPlay(false);
 return p;
 }
 return p + 1;
 });
 }, 1200);
 return () => clearInterval(t);
 }
}, [autoPlay, steps.length]);

const step = steps[stepIdx] || steps[0];

const isVisited = (nodeId: string) => step.visitedNodes.includes(nodeId);
const isCurrent = (nodeId: string) => step.node === nodeId;

return (
 <div className="space-y-4">
 <div className="flex bg-slate-900 border border-slate-800 p-4">
 <button onClick={() => setMode("dfs")}
 className={`px-4 py-2 text-xs font-medium rounded-md
 text-slate-400 hover:text-slate-300`} >
 DFS (В глубину)
 </button>
 </div>
 </div>
);

```

```

 fillColor = mode === 'dfs' ? 'blue' : 'red';
 strokeColor = mode === 'dfs' ? 'blue' : 'red';
 textColor = "fill-white";
 }

 if (current && step.type !== 'edge') {
 strokeColor = "stroke-amber";
 textColor = "fill-amber";
 radius = 22;
 }

 return (
 <g key={n.id} className={className} >
 <circle cx={n.x} cy={n.y}
 className={`stroke-${strokeColor} fill-${fillColor} stroke-width=2`}/>
 <text x={n.x} y={n.y}
 className={`text-${textColor} font-size=14`}/>
 {n.label}
 </text>

 {
 /* Ripple for BFS */
 current && mode === 'bfs' && step.type === 'edge' &&
 <circle cx={n.x} cy={n.y}
 className="stroke-amber stroke-width=2 fill-transparent"
 >
)
 </g>
)
 }
 }
 </svg>
</div>

{
 /* Structure View (Stack / Queue) */
 <div className="bg-slate-900 border border-emerald-500 p-10" >
 <div className="font-size=14 font-weight=bold text-white" >
 <div className={`absolute -top-3 left-3 right-3 bottom-3
 ${mode === 'dfs' ? 'border border-emerald-500' : 'border border-emerald-500'}
 >
 {mode === 'dfs' ? 'Стек вызовов' : 'Очередь'}
 </div>
 </div>
 </div>
}

```

```
 <SkipForward strokeWidth={3} size={20} />
 </button>
 <button onClick={() => {setAutoPlay(!autoPlay)}}
 bg-slate-800 rounded-lg ml-2 border-1 border-slate-700
 <RefreshCw strokeWidth={3} size={20} />
 </button>
 </div>

 { /* Description Log */ }
 <div className="flex-1 bg-slate-900/50 p-4 overflow-hidden">
 <div className="text-[10px] text-slate-400">
 {steps.length}</div>
 <div className="text-sm text-slate-400">
 {step.desc}
 </div>
 </div>
 </div>
</div>
);
}
```

---

ФАЙЛ 41/85: src/components/HeartViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useState, useCallback } from 'react';
import { Heart } from 'lucide-react';
```

```
interface Patient {
 id: string;
 name: string;
 emoji: string;
 symptom: string;
 priority: number; // 1 to 99 (Severity)
 animState: 'in' | 'idle' | 'winner' | 'out';
}
```



```

// Position in tree layout
function nodePos(index: number): { x: number; y: number } {
 const level = Math.floor(Math.log2(index + 1));
 const posInLevel = index - (Math.pow(2, level) - 1);
 const count = Math.pow(2, level);
 const x = ((posInLevel + 0.5) / count) * 100;
 const y = 10 + level * 24;
 return { x, y };
}

// Priority presets for realistic ER scenarios
const PATIENT_PRESETS = [
 { name: 'Иван (Инфаркт)', symptom: ' Боли в сердце', emoji: '🫀' },
 { name: 'Маша (Перелом)', symptom: ' Открытый перелом', emoji: '🦴' },
 { name: 'Кот Барсик (Укус)', symptom: ' Укусил шмеля', emoji: '🐈' },
 { name: 'Семён (Температура)', symptom: ' Жар 39.5', emoji: '🌡️' },
 { name: 'Оля (Порез)', symptom: ' Глубокий порез', emoji: '🔪' },
 { name: 'Пётр (Кашель)', symptom: ' Простуда', emoji: '🤧' },
];

const INITIAL_PATIENTS: Patient[] = (() => {
 let arr: Patient[] = [];
 const start = [
 { name: 'Иван (Инфаркт)', symptom: ' Боли в сердце', emoji: '🫀' },
 { name: 'Маша (Перелом)', symptom: ' Открытый перелом', emoji: '🦴' },
 { name: 'Семён (Температура)', symptom: ' Жар 39.5', emoji: '🌡️' },
 { name: 'Кот Барсик (Укус)', symptom: ' Укусил шмеля', emoji: '🐈' },
 { name: 'Оля (Порез)', symptom: ' Глубокий порез', emoji: '🔪' },
 { name: 'Пётр (Кашель)', symptom: ' Простуда', emoji: '🤧' },
];
 start.forEach((p, i) => {
 arr = heapInsert(arr, { id: `init${i}`, ...p, animS: 0 });
 });
 return arr;
})();

let uid = 300;

```

```

const marked = nextHeap.map(x => x.id === (target?.id || 0))

setHeap(marked);
addLog(` Поступил пациент: ${finalName} с тяжестью ${finalSeverity}

setTimeout(() => {
 setHeap(nextHeap);
 setBusy(false);
}, 500);
}, [busy, heap]);

const handleRandomInsert = () => {
 const preset = PATIENT_PRESETS[Math.floor(Math.random() * PATIENT_PRESETS.length)];
 const randomShift = Math.floor(Math.random() * 10);
 const prio = Math.max(10, Math.min(99, preset.priority + randomShift));
 insertPatient(preset.name, prio);
};

const handleCustomInsert = (e: React.FormEvent) => {
 e.preventDefault();
 if (!customName.trim()) return;
 insertPatient(customName, customPriority);
 setCustomName('');
};

// --- EXTRACT MAX: Направить самого тяжелого в реанимацию
const extractMax = useCallback(() => {
 if (busy || heap.length === 0) {
 setShake(true);
 addLog('⚠ Нет пациентов для экстренной госпитализации');
 setTimeout(() => setShake(false), 400);
 return;
 }
 setBusy(true);
 const topPatient = heap[0];
 setHealing(topPatient);

 addLog(` СРОЧНО! Забираем самого тяжелого: ${topPatient.name} с тяжестью ${topPatient.severity}

```

```
// --- Render binary tree lines ---
const edges = heap.flatMap((_, idx) => {
 const res: React.ReactElement[] = [];
 const p = nodePos(idx);
 const l = 2 * idx + 1;
 const r = 2 * idx + 2;
 if (l < heap.length) {
 const lp = nodePos(l);
 res.push(
 <line key={`el${idx}`}
 x1={` ${p.x}%`} y1={` ${p.y + 4}%`}
 x2={` ${lp.x}%`} y2={` ${lp.y - 4}%`}
 stroke="#475569" strokeWidth="2"
 />
);
 }
 if (r < heap.length) {
 const rp = nodePos(r);
 res.push(
 <line key={`er${idx}`}
 x1={` ${p.x}%`} y1={` ${p.y + 4}%`}
 x2={` ${rp.x}%`} y2={` ${rp.y - 4}%`}
 stroke="#475569" strokeWidth="2"
 />
);
 }
 return res;
});

return (
 <div className="flex flex-col gap-6">
 {/* МНЕМОНИКА / ПРАВИЛО */}
 <div className="bg-emerald-950/40 border border-emerald-400">
 <div className="text-3xl"> </div>
 <div>
 <h3 className="font-black text-emerald-400 text-4xl">
 Queue</h3>
 </div>
 </div>
 </div>
);
```

```

 min="10"
 max="99"
 value={customPriority}
 onChange={e => setCustomPriority(Number(e
 className="w-20 accent-emerald-500 cursor
 />
</div>
<button
 type="submit"
 disabled={busy || !customName.trim() || heap
 className="bg-teal-600 hover:bg-teal-500 di
 px-4 rounded-xl shadow-lg active:scale-95 t
>
 Везти
</button>
</form>

{/* Забрать самого тяжелого */}
<button
 onClick={extractMax}
 disabled={busy || heap.length === 0}
 className="flex-1 min-w-[170px] bg-gradient-to
 40 disabled:cursor-not-allowed text-white f
 transition-all cursor-pointer flex items-c
>
 В реанимацию (EXTRACT MAX)
</button>

<button
 onClick={reset}
 className="px-5 py-3.5 rounded-2xl bg-slate-8
 r-pointer border border-slate-700"
>
 Сброс
</button>
</div>

{/* ИНТЕРАКТИВНОЕ ОТДЕЛЕНИЕ */}

```



---

ФАЙЛ 42/85: src/components/hints/demoKit.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { createContext, useContext, useEffect, use
```

```
/** Общие примитивы для мини-визуализаций подсказок. */
```

```
export const W = 260;
```

```
export const H = 130;
```

```
/**
```

```
 * Общий множитель скорости. Кадры были слишком короткими
```

```
 * заметить, что именно переехало, и анимация читалась
```

```
 */
```

```
export const SPEED = 2.1;
```

```
/** Длительность переезда узлов. Должна быть заметно ме
```

```
export const MOVE_MS = 800;
```

```
export const EASE = "cubic-bezier(.34,.01,.2,1)";
```

```
export const MOVE = `all ${MOVE_MS}ms ${EASE}`;
```

```
/** Наведение на карточку ставит анимацию на паузу – мо
```

```
export const DemoPauseContext = createContext(false);
```

```
export function useLoop(steps: number, ms = 900) {
```

```
 const [step, setStep] = useState(0);
```

```
 const paused = useContext(DemoPauseContext);
```

```
 const period = Math.round(ms * SPEED);
```

```
 useEffect(() => {
```

```
 if (steps <= 1 || paused) return;
```

```
 const t = setInterval(() => setStep((s) => (s + 1) %
```

```
 return () => clearInterval(t);
```

```
 }, [steps, period, paused]);
```

```
 return step;
```

```

 label?: string | number;
 stroke?: string;
 }> = ({ x, y, r = 13, fill, label, stroke }) => (
 <g style={{ transition: MOVE }}>
 <circle cx={x} cy={y} r={r} fill={fill} stroke={stroke} />
 {label !== undefined && (
 <text x={x} y={y} textAnchor="middle" dominantBaseline="middle">
 {label}
 </text>
)}
 </g>
);

export const Edge: React.FC<{
 a: [number, number];
 b: [number, number];
 color?: string;
 width?: number;
 dashed?: boolean;
}> = ({ a, b, color = C.edge, width = 2, dashed }) => (
 <line
 x1={a[0]}
 y1={a[1]}
 x2={b[0]}
 y2={b[1]}
 stroke={color}
 strokeWidth={width}
 strokeDasharray={dashed ? "4 3" : undefined}
 style={{ transition: MOVE }}
 />
);

```

```

 });
};

/** Компоненты связности: несколько «островов», которые
export const ComponentsDemo: React.FC = () => {
 const comps: [string, string][][] = [
 ["A", "B"], ["B", "C"]],
 ["D", "E"]],
 ["F", "G"], ["G", "H"], ["F", "H"]],
];
 const pos: Record<string, [number, number]> = {
 A: [26, 40], B: [62, 92], C: [26, 92],
 D: [120, 40], E: [120, 92],
 F: [190, 34], G: [232, 78], H: [178, 96],
 };
 const colors = [C.active, C.warn, C.done];
 const step = useLoop(comps.length + 1, 900);
 const compOf = (v: string) => comps.findIndex((e) => e[0] === v);

 return (
 <Svg caption={step === 0 ? "Граф распался на острова" : ""}>
 {comps.flatMap((edges, ci) =>
 edges.map(([u, v], i) => (
 <Edge key={` ${ci} - ${i}`} a={pos[u]} b={pos[v]} />
))
)}
 {Object.entries(pos).map(([k, [x, y]]) => {
 const ci = compOf(k);
 return <Node key={k} x={x} y={y} r={12} label={k} color={colors[ci]} />
 })}
 </Svg>
);
};

```

```

/** Что такое граф: вершины, рёбра, направление и вес.
export const GraphBasicsDemo: React.FC = () => {
 const step = useLoop(3, 1300);
 const pos: Record<string, [number, number]> = { A: [50, 50], B: [150, 50], C: [250, 50],

```



```

/** Сжатие координат: разрежённые числа → плотные индексы
export const CoordCompressDemo: React.FC = () => {
 const raw = [1000, 5, 74000, 5, 300];
 const sorted = [5, 300, 1000, 74000];
 const step = useLoop(3, 1200);
 const boxW = 46;
 const startX = (260 - raw.length * (boxW + 4)) / 2;

 return (
 <Svg
 caption={
 step === 0
 ? "Исходные координаты – огромные и с дырами"
 : step === 1
 ? "Сортируем уникальные значения"
 : "Каждое число заменяем его номером → плотные индексы"
 }
 >
 {raw.map((v, i) => {
 <g key={i}>
 <rect x={startX + i * (boxW + 4)} y={20} width={boxW} height={20} />
 <text x={startX + i * (boxW + 4) + boxW / 2} y={35} textAnchor="middle">
 {v}
 </text>
 </g>
 })}

 {step >= 1 &&
 sorted.map((v, i) => {
 <g key={v}>
 <rect x={30 + i * 52} y={62} width={46} height={20} />
 <text x={53 + i * 52} y={77} textAnchor="middle">
 {v}
 </text>
 </g>
 })}
)
)
}

```

```

 style={{ transition: MOVE }}
 />
)))
 {Object.entries(f.pos).map(([id, [x, y]]) => (
 <g key={id} style={{ transform: `translate(${x}, ${y})` }}>
 <circle r={13} fill={color(id)} stroke={C.idl} />
 <text textAnchor="middle" dominantBaseline="middle">
 {id}
 </text>
 </g>
)))
 </Svg>
);
};

```

```

/** Zig – один поворот, когда родитель уже корень. */
export const ZigDemo: React.FC = () => (
 <SplayFrames
 captions={["p – корень, x под ним", "Один поворот: p → x"]}
 frames={[
 { pos: { p: [130, 32], x: [78, 92] }, edges: [{"p": "p", "x": "x"}] },
 { pos: { x: [130, 32], p: [182, 92] }, edges: [{"x": "x", "p": "p"}] }
]}
 />
);

```

```

/** Zig-Zig – x и p с одной стороны, крутим сначала вершину g. */
export const ZigZigDemo: React.FC = () => (
 <SplayFrames
 captions={["Линия g → p → x («бамбук»)", "Поворот В: g → p → x"]}
 frames={[
 { pos: { g: [176, 24], p: [126, 70], x: [76, 112] }, edges: [{"g": "g", "p": "p"}, {"p": "p", "x": "x"}] },
 { pos: { p: [130, 26], x: [78, 78], g: [186, 78] }, edges: [{"p": "p", "x": "x"}, {"g": "g", "p": "p"}] },
 { pos: { x: [102, 24], p: [152, 70], g: [202, 112] }, edges: [{"x": "x", "p": "p"}, {"g": "g", "x": "x"}] }
]}
 />
);

```

```

 {Object.entries(GRAPH_POS).map(([k, [x, y]]) => (
 <Node key={k} x={x} y={y} label={k}
 fill={wave.includes(k) ? C.active : visited.h
 stroke={wave.includes(k) ? "#a5b4fc" : undefi
))}
 </Svg>
);
};

export const DfsDemo: React.FC = () => {
 const path = ["A", "B", "D", "E", "F", "C"];
 const step = useLoop(path.length + 1, 750);
 const visited = path.slice(0, step);
 const head = visited[visited.length - 1];
 return (
 <Svg caption={head ? `Идём вглубь: ${visited.join("
 {GRAPH_EDGES.map(([u, v], i) => {
 const iu = visited.indexOf(u);
 const iv = visited.indexOf(v);
 const onPath = iu >= 0 && iv >= 0 && Math.abs(i
 return <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_
 }}}
 {Object.entries(GRAPH_POS).map(([k, [x, y]]) => (
 <Node key={k} x={x} y={y} label={k} fill={k ===
))}
 </Svg>
);
};
};

```

```

export const ToposortDemo: React.FC = () => {
 const nodes = [
 { id: "трусы", x: 52, y: 32 }, { id: "штаны", x: 148, y: 32 },
 { id: "носки", x: 52, y: 98 }, { id: "боты", x: 148, y: 98 }
];
 const edges: [string, string][] = [
 ["трусы", "штаны"],
 ["носки", "боты"],
 ["штаны", "боты"]
];
 const order = ["трусы", "носки", "штаны", "боты"];
 const step = useLoop(order.length + 1, 800);
 const placed = order.slice(0, step);

```

```

 <text x={200} y={106} textAnchor="middle" fontSize=
 <text x={130} y={22} textAnchor="middle" fontSize=
 </Svg>
);
};

export const BridgeDemo: React.FC = () => {
 const step = useLoop(2, 1300);
 const pos: Record<string, [number, number]> = {
 A: [40, 40], B: [40, 95], C: [95, 68], D: [170, 68]
 };
 const edges: [string, string][] = [
 ["A", "B"], ["A", "C"], ["B", "C"], ["C", "D"], ["D", "A"]
];
 const cut = step === 1;
 return (
 <Svg caption={cut ? "Убрали C-D → граф развалился на 2 компонента" : ""}>
 {edges.map(([u, v], i) => {
 const isBridge = u === "C" && v === "D";
 if (isBridge && cut) return null;
 return <Edge key={i} a={pos[u]} b={pos[v]} color={isBridge ? "red" : "black"} />
 })}
 {Object.entries(pos).map(([k, [x, y]]) => (
 <Node key={k} x={x} y={y} label={k} fill={cut ? "red" : "black"} />
))}
 </Svg>
);
};
};

```

```

export const ArticulationDemo: React.FC = () => {
 const step = useLoop(2, 1300);
 const pos: Record<string, [number, number]> = {
 A: [36, 40], B: [36, 96], C: [130, 68], D: [224, 40]
 };
 const edges: [string, string][] = [
 ["A", "B"], ["A", "C"], ["B", "C"], ["C", "D"], ["D", "A"]
];
 const cut = step === 1;
 return (
 <Svg caption={cut ? "Убрали вершину C → две отдельные компоненты" : ""}>
 {edges.map(([u, v], i) => {
 const isArticulation = u === "C" || v === "C";
 if (isArticulation && cut) return null;
 return <Edge key={i} a={pos[u]} b={pos[v]} color={isArticulation ? "red" : "black"} />
 })}
 {Object.entries(pos).map(([k, [x, y]]) => (
 <Node key={k} x={x} y={y} label={k} fill={cut ? "red" : "black"} />
))}
 </Svg>
);
};

```



```

 <marker id="mdArrow" markerWidth="6" markerHeight="6"
 <path d="M0,0 L6,3 L0,6 Z" fill={C.done} />
 </marker>
 </defs>
 <text x={12} y={44} fontSize={9} fill={C.dim}>ВЫХОД</text>
 <text x={206} y={44} fontSize={9} fill={C.dim}>ВХОД</text>
 {items.map((v, i) => (
 <g key={v} style={{ transition: "all .3s" }}>
 <rect x={30 + i * 54} y={56} width={46} height={20} />
 <text x={53 + i * 54} y={71} textAnchor="middle">{v}</text>
 </g>
))}
 <line x1={24} y1={71} x2={10} y2={71} stroke={C.dim} />
</Svg>
);
};

export const DsuDemo: React.FC = () => {
 const parents = [[0, 1, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3]];
 const step = useLoop(parents.length, 950);
 const p = parents[step];
 const pos: [number, number][] = [[50, 95], [105, 95], [160, 95], [215, 95]];
 const rootColor = ["#6366f1", "#10b981", "#f59e0b", "#ff7f0e"];
 const root = (i: number): number => (p[i] === i ? i : root(p[i]));
 return (
 <Svg caption="find = «кто твой староста», union = 0" >
 {p.map((par, i) =>
 par !== i ? (
 <path key={i} d={`M ${pos[i][0]} ${pos[i][1]} L ${pos[root(i)][0]} ${pos[root(i)][1]}`
 fill="none" stroke={rootColor[root(i)]} strokeDash={[]} />
) : null
)}
 {pos.map(([x, y], i) => <Node key={i} x={x} y={y} />)}
 </Svg>
);
};
};

```

```

 </Svg>
);
};

export const SuffixLinkDemo: React.FC = () => {
 const links: [number, number][] = [[1, 0], [2, 0], [3, 0]];
 const step = useLoop(links.length + 1, 900);
 return (
 <Svg caption="Суффиксная ссылка: «куда откатиться,»"
 {TRIE_NODES.filter((n) => n.parent !== null).map(
 (n) => {
 const p = TRIE_NODES[n.parent as number];
 return <Edge key={n.id} a={[p.x, p.y]} b={[n.x, n.y]} />
 }
)}
 {links.slice(0, step).map(([from, to], i) => {
 const a = TRIE_NODES[from];
 const b = TRIE_NODES[to];
 return (
 <path key={i} d={`M ${a.x} ${a.y} Q ${(a.x + b.x) / 2} ${(a.y + b.y) / 2} ${b.x} ${b.y}`}
 fill="none" stroke={C.warn} strokeWidth={1.5} />
);
 })}
 {TRIE_NODES.map((n) => {
 return <Node key={n.id} x={n.x} y={n.y} label={n.ch} fill={C.warn} />
 })}
 </Svg>
);
};

export const SegmentTreeDemo: React.FC = () => {
 const step = useLoop(3, 1000);
 const nodes = [
 { x: 130, y: 24, w: 226, label: "[0..7]", lvl: 0 },
 { x: 72, y: 60, w: 110, label: "[0..3]", lvl: 1 },
 { x: 188, y: 60, w: 110, label: "[4..7]", lvl: 1 },
 { x: 44, y: 96, w: 52, label: "[0..1]", lvl: 2 },
 { x: 100, y: 96, w: 52, label: "[2..3]", lvl: 2 },
 { x: 160, y: 96, w: 52, label: "[4..5]", lvl: 2 },
 { x: 216, y: 96, w: 52, label: "[6..7]", lvl: 2 },
];

```

```

 <text x={34 + i * 33} y={90} textAnchor="middle">
 >
 </g>
)})
 </Svg>
);
};

```

```

export const SparseTableDemo: React.FC = () => {
 const step = useLoop(3, 1000);
 const len = [1, 2, 4][step];
 const count = 8 - len + 1;
 return (
 <Svg caption={`Уровень j=${step}: готовые ответы для`}>
 {Array.from({ length: 8 }).map((_, i) => (
 <g key={i}>
 <rect x={12 + i * 30} y={18} width={26} height={10} />
 <text x={25 + i * 30} y={29} textAnchor="middle">
 </g>
)})}
 {Array.from({ length: count }).map((_, i) => (
 <rect key={i} x={12 + i * 30} y={52 + (i % 4) * 10} width={26} height={10}
 fill={C.active} opacity={i === 0 ? 1 : 0.4} stroke={C.done} />
)})}
 </Svg>
);
};

```

```

export const AmortizedDemo: React.FC = () => {
 const costs = [1, 1, 1, 8, 1, 1, 1, 1];
 const step = useLoop(costs.length + 1, 550);
 const total = costs.slice(0, step).reduce((a, b) => a + b, 0);
 const avg = step ? (total / step).toFixed(2) : "-";
 return (
 <Svg caption={`Сделано ${step} операций, суммарно $`}>
 {costs.map((c, i) => (
 <rect key={i} x={14 + i * 30} y={104 - c * 10} width={26} height={10}
 fill={i < step ? (c > 2 ? C.bad : C.done) : C.done} stroke={C.done} />
)})}
 </Svg>
);
};

```



```

 <text x={32 + idx * 33} y={70} textAnchor="middle"
 fill={idx <= i && step > 0 ? C.warn : C.idle}
 </g>
)))
 <text x={4} y={40} fontSize={8} fill={C.dim}>s</text>
 <text x={4} y={70} fontSize={8} fill={C.dim}>π</text>
 <text x={130} y={106} textAnchor="middle" fontSize={10}
 π[i] — длина самого длинного «префикс = суффикс»
 </text>
</Svg>
);
};

export const DpDemo: React.FC = () => {
 const rows = 3, cols = 5;
 const step = useLoop(rows * cols + 1, 320);
 return (
 <Svg caption={`Заполняем таблицу подзадач: готово $
 {Array.from({ length: rows }).map((_, r) =>
 Array.from({ length: cols }).map((_, c) => {
 const idx = r * cols + c;
 const filled = idx < step;
 return (
 <g key={idx}>
 <rect x={40 + c * 38} y={22 + r * 32} width={38}
 fill={idx === step - 1 ? C.active : filled}
 {filled && (
 <text x={57 + c * 38} y={36 + r * 32} textAnchor="middle"
 fontSize={10} fontWeight="700" fill={C.dim}>
 {π[idx]}
 </text>
)}
 </g>
);
 })
 </Svg>
);
)}
 <text x={130} y={122} textAnchor="middle" fontSize={10}
 каждая клетка считается один раз и переиспользуется
 </text>
</Svg>

```

```

toposort: ToposortDemo,
relax: RelaxDemo,
"prefix-sum": PrefixSumDemo,
"sparse-table": SparseTableDemo,
"segment-tree": SegmentTreeDemo,
bridge: BridgeDemo,
articulation: ArticulationDemo,
mst: MstDemo,
amortized: AmortizedDemo,
np: NpDemo,
scc: SccDemo,
"prefix-function": PrefixFunctionDemo,
dp: DpDemo,
floyd: FloydDemo,
components: ComponentsDemo,
graph: GraphBasicsDemo,
"coord-compress": CoordCompressDemo,
zig: ZigDemo,
"zig-zig": ZigZigDemo,
"zig-zag": ZigZagDemo,
};

export const MiniDemo: React.FC<{ kind: DemoKind }> = (
 const [paused, setPaused] = useState(false);
 const Cmp = REGISTRY[kind];
 if (!Cmp) return null;
 return (
 <DemoPauseContext.Provider value={paused}>
 <div
 className="bg-slate-950/70 rounded-lg border bo
 onMouseEnter={() => setPaused(true)}
 onMouseLeave={() => setPaused(false)}
 >
 <Cmp />
 </div>
 </DemoPauseContext.Provider>
);
);
};

```

```

 <div className="absolute inset-0 bg-slate-950/80" />
 <div className="relative w-full sm:max-w-5xl max-h-
 ded-2xl shadow-2xl flex flex-col">
 <div className="flex items-center gap-3 px-4 py-4">
 <div className="min-w-0 flex-1">
 <h3 className="font-bold text-white text-sm">
 {viz.hint} && <p className="text-[11px] text-white">
 </p>
 </div>
 <button
 onClick={onClose}
 className="p-2 rounded-lg bg-slate-800 hover:bg-slate-700"
 aria-label="Заккрыть симулятор"
 >
 <X className="w-5 h-5" />
 </button>
 </div>
 <div className="overflow-y-auto overscroll-contain">
 <Component />
 </div>
 </div>
 </div>,
 document.body
);
};

```

ФАЙЛ 48/85: src/components/hints/TermPopover.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import React, { useEffect, useLayoutEffect, useRef, useState } from "react";
import { createPortal } from "react-dom";
import { Play, X } from "lucide-react";
import { GLOSSARY_BY_ID } from "../../data/glossary";
import { MiniDemo } from "../MiniDemo";
import { getViz } from "../vizRegistry";

```

```

useEffect(() => {
 if (!anchor) return;
 const onKey = (e: KeyboardEvent) => {
 if (e.key === "Escape") onClose();
 };
 window.addEventListener("keydown", onKey);
 window.addEventListener("scroll", onClose, true);
 return () => {
 window.removeEventListener("keydown", onKey);
 window.removeEventListener("scroll", onClose, true);
 };
}, [anchor, onClose]);

if (!anchor) return null;
const entry = GLOSSARY_BY_ID.get(anchor.termId);
if (!entry) return null;
const viz = getViz(entry.simulator);
const simulatorId = entry.simulator;

return createPortal(
 <div
 ref={cardRef}
 onMouseEnter={onCardEnter}
 onMouseLeave={onCardLeave}
 className="fixed z-[100] term-popover"
 style={{
 top: pos ? pos.top : -9999,
 left: pos ? pos.left : -9999,
 width: Math.min(CARD_W, typeof window !== "undefined" ? window.innerWidth : 1000),
 visibility: pos ? "visible" : "hidden",
 }}
 role="dialog"
 >
 <div className="bg-slate-900 border border-indigo-500 p-4"
 <div className="flex items-start gap-2 px-3 pt-3"
 <h4 className="text-sm font-bold text-indigo-400"
 <button

```



```

 if (step === 2) return "url(#arrow-pink)";
 return "url(#arrow-slate-dim)";
 }
 if (e.id === 'AB' && step === 4) return "url(#a
 if (e.id === 'AB' && step > 4) return "url(#arr
 if (e.id === 'BC' && step === 5) return "url(#a
 if (e.id === 'BC' && step > 5) return "url(#arr
 if (e.id === 'CA' && step === 6) return "url(#a
 if (e.id === 'CA' && step > 6) return "url(#arr

 if (step >= 4) return "url(#arrow-slate-dim)";
 return "url(#arrow-slate)";
};

const getEdgeContent = (e: any) => {
 if (e.u === 'S') return e.weight;

 if (step < 4) return e.weight;

 if (e.id === 'AB') {
 if (step === 4) return `${e.weight} + (${0}
 return e.newWeight;
 }
 if (e.id === 'BC') {
 if (step < 5) return e.weight;
 if (step === 5) return `${e.weight} + (${-2}
 return e.newWeight;
 }
 if (e.id === 'CA') {
 if (step < 6) return e.weight;
 if (step === 6) return `${e.weight} + (${-1}
 return e.newWeight;
 }
 return e.weight;
};

const isEdgeTextHighlighted = (e: any) => {
 if (e.id === 'AB' && step === 4) return true;

```

```

 Шаг 3: Магическая формула.Шаг 3.1: Пересчет ребра A → B.
 Всем ребрам графа назначаем новый вес.
 <div className="text-center font-monospace">
 Это повышает или понижает вес ребра?
 </div>
);
 case 4: return (
 <div>
 Шаг 3.1: Пересчет ребра A → B.
 Старый вес: -2
 Вычисляем: $-2 + 0 - (-2) =$ 0
 </div>
);
 case 5: return (
 <div>
 Шаг 3.2: Пересчет ребра B → C.
 Старый вес: 1.

 Вычисляем: $1 + (-2) - (-1) =$ 0
 </div>
);
 case 6: return (
 <div>
 Шаг 3.3: Пересчет ребра C → A.
 Старый вес: 2.

 Вычисляем: $2 + (-1) - 0 =$ 1
 </div>
);
 case 7: return (
 <div className="text-emerald-300">
 Готово! Фиктивную вершину S мы добавили в граф.
 Все новые веса ребер в графе стали меньше или равны старым.
 При этом старые кратчайшие пути остались кратчайшими!
 </div>
);
 default: return "";
}
};

```





```

 </div>

 {/* Controls */}
 <div className="flex bg-slate-900 border-1
 <button onClick={() => setStep(1)}
 className="p-3 bg-slate-800 border-1
 -slate-400 transition-colors" title="Сброси
 <RotateCcw strokeWidth={3}
 </button>
 <button onClick={() => setStep(1)}
 className="p-3 bg-slate-800 border-1
 -slate-400 transition-colors" title="Вар на
 <Undo strokeWidth={3} size=
 </button>
 <button onClick={() => setStep(
 className={`flex-1 font-bol
 ${step === MAX_STEP
 ? 'bg-emerald-600/50'
 : 'bg-indigo-600 hover:
 {step === MAX_STEP ? "Готово"
 {step !== MAX_STEP && <Skip
 </button>
 </div>

 {/* Progress Dots */}
 <div className="flex justify-center gap-1"
 {Array.from({length: MAX_STEP + 1}, (_, i) =>
 <div key={i} className={`w-10 h-10 rounded-full
 125' : i < step ? 'bg-indigo-900' : 'bg-slate-800'
 >>>
 </div>
 </div>
 </div>
</div>
</div>
</div>
);
}

```

```

 { u: 3, v: 4 },
 { u: 4, v: 3 },
];

interface AlgoStep {
 phase: "init" | "dfs1" | "transpose" | "dfs2" | "fini";
 desc: string;
 visited: Set<number>;
 currentNode: number | null;
 order: number[];
 transposed: boolean;
 sccMap: Record<number, number>; // node -> sccId
 currentSccId: number;
 activeEdges: GEdge[];
}

export const KosarajuViz: React.FC = () => {
 const [steps, setSteps] = useState<AlgoStep[]>([]);
 const [currentStepIdx, setCurrentStepIdx] = useState(0);
 const [isPlaying, setIsPlaying] = useState(false);

 useEffect(() => {
 // Generate step-by-step Kosaraju steps
 const generatedSteps: AlgoStep[] = [];
 const visited = new Set<number>();
 const order: number[] = [];
 const sccMap: Record<number, number> = {};

 const recordStep = (
 phase: AlgoStep["phase"],
 desc: string,
 currentNode: number | null,
 transposed: boolean,
 currentSccId: number,
) => {
 generatedSteps.push({
 phase,
 desc,
 currentNode,
 transposed,
 currentSccId,
 visited,
 order,
 sccMap,
 });
 };
 });
};

```

```

 }
 }

 order.push(u);
 recordStep(
 "dfs1",
 `DFS 1: вершина ${u} полностью обработана. Запи-
 u,
 false,
 -1,
);
};

// Run DFS1 on all unvisited
for (let i = 0; i < 5; i++) {
 if (!visited.has(i)) {
 dfs1(i);
 }
}

const topoOrder = [...order].reverse();
recordStep(
 "transpose",
 `Первый проход завершен! Стек выхода в топологиче-
 ебра.`,
 null,
 true,
 -1,
);

// Transpose adj
const adjT: number[][] = Array.from({ length: 5 },
initialEdges.forEach((e) => adjT[e.v].push(e.u)));

// Reset visited for phase 2
visited.clear();
let sccId = 0;

```

```

 });
 }
}

recordStep(
 "finished",
 `Алгоритм успешно завершен! Найдено ${sccId} компонент,
 null,
 true,
 sccId,
);

setSteps(generatedSteps);
setCurrentStepIdx(0);
}, []));

const step = steps[currentStepIdx] || {
 phase: "init",
 desc: "Загрузка...",
 visited: new Set(),
 currentNode: null,
 order: [],
 transposed: false,
 sccMap: {},
 currentSccId: -1,
 activeEdges: initialEdges,
};

useEffect(() => {
 let timer: NodeJS.Timeout;
 if (isPlaying && currentStepIdx < steps.length - 1) {
 timer = setInterval(() => {
 setCurrentStepIdx((prev) => prev + 1);
 }, 1500);
 } else {
 setIsPlaying(false);
 }
 return () => clearInterval(timer);
});

```

```

 className="flex items-center justify-center
 ors"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
</div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-12" >
 {/* Playfield SVG */}
 <div className="lg:col-span-8 bg-[#0B1120] relative" >
 <svg className="w-full h-full max-w-[650px]" >
 {/* Markers for directed arrows */}
 <defs>
 <marker
 id="arrow"
 viewBox="0 0 10 10"
 refX="22"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#6" />
 </marker>
 <marker
 id="arrow-active"
 viewBox="0 0 10 10"
 refX="22"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#1" />
 </marker>
 <marker
 id="arrow-transposed"

```

```

 markerId = "arrow-active";
 }
}

// Adjust slightly for bidirectional link
const isBidi =
 (edge.u === 3 && edge.v === 4) ||
 (edge.u === 4 && edge.v === 3);
const dy = isBidi ? (edge.u === 3 ? -10 : 10) : 0;

return (
 <line
 key={`edge-${idx}`}
 x1={x1}
 y1={y1 + dy}
 x2={x2}
 y2={y2 + dy}
 stroke={stroke}
 strokeWidth="2.5"
 markerEnd={`url(#${markerId})`}
 className="transition-all duration-300"
 />
);
}}}

{/* Nodes */}
{initialNodes.map((node) => {
 const isCurrent = step.currentNode === node.id;
 const isVisited =
 step.visited.has(node.id) ||
 (step.phase === "dfs1" && step.currentNode === node.id);
 const sccId = step.sccMap[node.id];

 let classes = "fill-slate-800 stroke-slate-800";
 if (sccId) {
 classes = getSccColor(sccId);
 } else if (isCurrent) {
 classes = "fill-amber-600 stroke-amber-600";
 }
 return (
 <div
 key={node.id}
 className={classes}
 style={{
 width: node.w,
 height: node.h,
 position: 'relative',
 }}
 >
 {node.label}
 </div>
);
})}
```

```

 </g>
);
 }}}
 </svg>

 {step.transposed && (
 <div className="absolute top-4 left-4 bg-in
 1 rounded">
 Граф транспонирован (Ребра обратные)
 </div>
)}
</div>

{/* Logs & Stacks */}
<div className="lg:col-span-4 bg-slate-950 p-5
00 overflow-y-auto">
 <h4 className="text-xs font-bold text-slate-4
 Порядок обхода и стек
</h4>

 <div className="bg-slate-900 border border-sl
 <p className="text-xs text-white leading-re
 {step.desc}
 </p>
 </div>

 <div className="flex-1 space-y-4 overflow-y-a
 {/* Top-Sort output / Order stack represent
 <div>
 <span className="text-[10px] text-slate-5
 СТЕК ВЫХОДА (DFS 1 ORDER):

 <div className="flex flex-wrap gap-1 bg-s
 {step.order.length === 0 ? (
 <span className="text-slate-600 text-
) : (
 [...step.order].reverse().map((val, i
 <div

```

```

 <button
 disabled={currentStepIdx === 0}
 onClick={() => setCurrentStepIdx((prev)
 className="bg-slate-900 border border-s
 >
 Назад
 </button>
 <button
 disabled={currentStepIdx >= steps.length
 onClick={() => setCurrentStepIdx((prev)
 className="bg-slate-900 border border-s
 >
 Вперед
 </button>
 </div>
</div>
</div>
</div>
</div>
);
};

```

---

ФАЙЛ 51/85: src/components/KruskalSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

---

```

import React, { useState } from 'react';
import { Play, RotateCcw, SkipForward, HelpCircle, Chec
 ide-react';

```

```

interface Vertex {
 id: number;
 label: string;
 x: number;
 y: number;
 color: string; // 'none', 'red', 'blue', 'purple', 'm

```



```

 { id: '3-6', u: 3, v: 6, weight: 8, status: 'pending'
 { id: '2-5', u: 2, v: 5, weight: 9, status: 'pending'
 { id: '6-7', u: 6, v: 7, weight: 10, status: 'pending'
 { id: '4-7', u: 4, v: 7, weight: 11, status: 'pending'
 { id: '7-8', u: 7, v: 8, weight: 12, status: 'pending'
 { id: '5-8', u: 5, v: 8, weight: 13, status: 'pending'
];
```

```
export const KruskalSimulator: React.FC = () => {
 const [activeAlgo, setActiveAlgo] = useState<'kruskal'
 const [vertices, setVertices] = useState<Vertex[]>(in
 const [edges, setEdges] = useState<Edge[]>(initialEdg
 const [stepIndex, setStepIndex] = useState<number>(0)
 const [heapQueue, setHeapQueue] = useState<string[]>(
 const [activeCheatTab, setActiveCheatTab] = useState<

 const [logs, setLogs] = useState<StepLog[]>([
 {
 title: 'Введение в раскраску (Краскал)',
 description: '«Краскал – это раскраска?». Предста
 овы начинать!',
 type: 'info',
 },
]));
```

```
// --- KRUSKAL STEPS ---
```

```
const kruskalSteps = [
 // Step 1: Inspect A-B (2)
 () => {
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 setLogs(prev => [{
 title: 'Шаг 1: Берем самое дешевое ребро A-B (в
 description: 'Оно соединяет две бесцветные вершн
 type: 'info'
 }, ...prev]));
 },
 // Step 2: Include A-B
 () => {
```

```

 setVertices(prev => prev.map(v => v.id === 2 ? {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро В-С в осто́ве',
 description: 'Вершина С перекрашена в красный ц
 type: 'success'
 }, ...prev]));
 },
// Step 7: Inspect A-D (5)
() => {
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 setLogs(prev => [{
 title: 'Шаг 4: Берем ребро A-D (вес 5)',
 description: 'ВНИМАНИЕ! Ребро соединяет КРАСНУЮ
 ю – мы перекрашиваем всё в один цвет!"',
 type: 'warning'
 }, ...prev]));
},
// Step 8: Include A-D (Merge to purple)
() => {
 setVertices(prev => prev.map(v => v.color === 'bl
 setEdges(prev => prev.map(e => e.status === 'incl
 setLogs(prev => [{
 title: 'Слияние кланов: Ребро A-D добавлено',
 description: 'Мы объединили красный и синий клан
 type: 'success'
 }, ...prev]));
},
// Step 9: Inspect B-E (6) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setLogs(prev => [{
 title: 'Шаг 5: Берем ребро B-E (вес 6)',
 description: 'Н0: это ребро соединяет вершины B
 type: 'warning'
 }, ...prev]));
},
// Step 10: Reject B-E

```

```

 setVertices(prev => prev.map(v => v.id === 6 ? {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро D-G в осто́ве',
 description: 'Вершина G окрашена в фиолетовый.',
 type: 'success'
 }, ...prev]));
 },
// Step 15: Inspect C-F (9) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setLogs(prev => [{
 title: 'Шаг 8: Берем ребро C-F (вес 9)',
 description: 'Оно соединяет вершины C и F, кото
 type: 'warning'
 }, ...prev]));
},
// Step 16: Reject C-F
() => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setLogs(prev => [{
 title: 'Отказ: Цикл обнаружен!',
 description: 'Выбрасываем ребро C-F.',
 type: 'error'
 }, ...prev]));
},
// Step 17: Inspect G-H (10)
() => {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 setLogs(prev => [{
 title: 'Шаг 9: Берем ребро G-H (вес 10)',
 description: 'Соединяет фиолетовую G и бесцветн
 type: 'info'
 }, ...prev]));
},
// Step 18: Include G-H
() => {
 setVertices(prev => prev.map(v => v.id === 7 ? {

```

```

 setLogs(prev => [{
 title: ' Успех: MST построено Краскалом!',
 description: 'Все 9 вершин окрашены в единый фи
 + 4 + 5 + 7 + 8 + 10 + 12 = 51.',
 type: 'success'
 }, ...prev]));
 },
];

// --- PRIM STEPS ---
const primSteps = [
 // Step 1: Start at E
 () => {
 setVertices(prev => prev.map(v => v.id === 4 ? {
 setHeapQueue(['D-E (3)', 'B-E (6)', 'E-F (7)', 'E
 setEdges(prev => prev.map(e => ['3-4', '1-4', '4-5
 setLogs(prev => [{
 title: 'Шаг 1: Плесень зарождается в Токио (Вер
 description: 'Мы выбрали вершину E как стартовую
 ритетную очередь (Heap): D-E(3), B-E(6), E-F(
 type: 'info'
 }, ...prev]));
 },
 // Step 2: Pop D-E (3)
 () => {
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 setLogs(prev => [{
 title: 'Шаг 2: Куча выдает самое дешевое ребро
 description: 'Плесень тянется по ребру D-E к вер
 type: 'info'
 }, ...prev]));
 },
 // Step 3: Include D-E, add D's edges to heap
 () => {
 setVertices(prev => prev.map(v => v.id === 3 ? {
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 ..e, status: 'in_heap' } : e));
 setHeapQueue(['A-D (5)', 'B-E (6)', 'E-F (7)', 'D

```

```

setVertices(prev => prev.map(v => v.id === 1 ? {
setEdges(prev => prev.map(e => e.id === '0-1' ? {
 eap' } : e));
setHeapQueue(['B-C (4)', 'B-E (6 - цикл)', 'E-F (
setLogs(prev => [{
 title: 'Успех: Вершина B захвачена плесенью',
 description: 'В кучу добавлено B-C(4). Ребро B-
 type: 'success'
}, ...prev]));
},
// Step 8: Pop B-C (4)
() => {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 setLogs(prev => [{
 title: 'Шаг 5: Куча выдает ребро B-C (вес 4)',
 description: 'Плесень тянется к вершине C.',
 type: 'info'
 }, ...prev]));
},
// Step 9: Include B-C, add C's edges
() => {
 setVertices(prev => prev.map(v => v.id === 2 ? {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 eap' } : e));
 setHeapQueue(['B-E (6 - цикл)', 'E-F (7)', 'D-G (
 setLogs(prev => [{
 title: 'Успех: Вершина C захвачена',
 description: 'Ребро C-F(9) добавлено в кучу.',
 type: 'success'
 }, ...prev]));
},
// Step 10: Pop B-E (6) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setLogs(prev => [{
 title: 'Шаг 6: Куча выдает ребро B-E (вес 6)',
 description: 'ВНИМАНИЕ! Плесень проверяет верши
 type: 'warning'

```

```

 description: 'Плесень расширяется вниз к G.',
 type: 'info'
 }, ...prev]);
},
// Step 15: Include D-G, add G's edges
() => {
 setVertices(prev => prev.map(v => v.id === 6 ? {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 eap' } : e));
 setHeapQueue(['C-F (9 - цикл)', 'G-H (10)', 'E-H
 setLogs(prev => [{
 title: 'Успех: Вершина G захвачена',
 description: 'В кучу добавлено G-H(10).',
 type: 'success'
 }, ...prev]);
},
// Step 16: Pop C-F (9) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setLogs(prev => [{
 title: 'Шаг 9: Куча выдает ребро C-F (вес 9)',
 description: 'Вершины C и F уже заражены плесенью',
 type: 'warning'
 }, ...prev]);
},
// Step 17: Reject C-F
() => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setHeapQueue(['G-H (10)', 'E-H (11)', 'F-I (13)'])
 setLogs(prev => [{
 title: 'Отказ: Игнорируем ребро C-F',
 description: 'Выбрасываем из кучи.',
 type: 'error'
 }, ...prev]);
},
// Step 18: Pop G-H (10)
() => {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {

```

```

 () => {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setLogs(prev => [{
 title: 'Шаг 12: Куча выдает H-I (вес 12)',
 description: 'Плесень тянется к последней чистой',
 type: 'info'
 }, ...prev]));
 },
 // Step 23: Include H-I
 () => {
 setVertices(prev => prev.map(v => v.id === 8 ? {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setHeapQueue(['F-I (13 - цикл)']);
 setLogs(prev => [{
 title: ' Успех: Плесень захватила весь граф
 description: 'Все 9 вершин поглощены. Итоговый
 иной – мы разрастались из одной точки с помо
 type: 'success'
 }, ...prev]));
 },
];

// --- BORUVKA STEPS ---
const boruvkaSteps = [
 // Step 1: Five-Year Plan 1 (Inspecting outgoing)
 () => {
 // Highlight the cheapest outgoing edges for EACH
 // Node 0(A) -> A-B(2)
 // Node 1(B) -> A-B(2)
 // Node 2(C) -> B-C(4)
 // Node 3(D) -> D-E(3)
 // Node 4(E) -> D-E(3)
 // Node 5(F) -> E-F(7)
 // Node 6(G) -> D-G(8)
 // Node 7(H) -> G-H(10)
 // Node 8(I) -> H-I(12)
 setEdges(prev => prev.map(e =>
 ['0-1', '3-4', '1-2', '4-5', '3-6', '6-7', '7-8

```

```

// Cheapest is A-D (5).
setEdges(prev => prev.map(e => e.id === '0-3' ? {
setLogs(prev => [{
 title: 'Вторая пятилетка: Колхозы выбирают мост
 description: 'Теперь Красный и Синий колхозы од
 -D с весом 5. Остальные мосты B-E(6) и C-F(9)
 type: 'warning'
}], ...prev]));
},
// Step 4: Concurrently merge into single Kolkhoz
() => {
 setVertices(prev => prev.map(v => ({ ...v, color:
 setEdges(prev => prev.map(e =>
 e.status === 'included' || e.id === '0-3' ? { .
 }));
 setLogs(prev => [{
 title: 'Объединение завершено: Один гигантский
 description: 'Оба колхоза объединились по мосту
 type: 'success'
 }], ...prev));
},
// Step 5: Check remaining edges & mark as rejected
() => {
 setEdges(prev => prev.map(e => e.status === 'pend
 setLogs(prev => [{
 title: ' Успех: MST построен Борувкой за 2 шаг
 description: 'Все лишние ребра (B-E, E-H, C-F,
 вес: 51. Благодаря параллельности, мы потрати
 type: 'success'
 }], ...prev));
}
];

const currentSteps =
 activeAlgo === 'kruskal' ? kruskalSteps :
 activeAlgo === 'prim' ? primSteps :
 boruvkaSteps;

```



```
 });
 }
};
```

```
const getColorClass = (color: string, id: number) =>
 switch (color) {
 case 'red': return 'bg-red-500 border-red-300 text-red-500';
 case 'blue': return 'bg-blue-500 border-blue-300 text-blue-500';
 case 'purple': return 'bg-purple-600 border-purple-300 text-purple-500';
 case 'mold': return 'bg-emerald-500 border-emerald-300 text-emerald-500';
 default:
 if (activeAlgo === 'prim' && id === 4 && stepIndex === 0)
 return 'bg-slate-700 border-emerald-500 text-emerald-500';
 return 'bg-slate-700 border-slate-500 text-slate-500';
 }
};
```

```
const getEdgeStroke = (status: string, color: string) =>
 if (status === 'inspecting') return '#f59e0b';
 if (status === 'rejected') return '#ef4444';
 if (status === 'in_heap') return '#0284c7';
 if (status === 'included') {
 if (color === 'red') return '#ef4444';
 if (color === 'blue') return '#3b82f6';
 if (color === 'purple') return '#a855f7';
 if (color === 'mold') return '#10b981';
 }
 return '#475569';
};
```

```
return (
 <div className="space-y-12">

 {/* 3-in-1 MST Simulator Box */}
 <div className="bg-slate-900/90 border border-slate-800 p-4">
 <div className="flex flex-col lg:flex-row justify-between">
 <div>
```

```

 <Layers className="w-3.5 h-3.5" />
 Прима (Билет 19)
 </button>
 <button
 onClick={() => handleReset('boruvka')}
 className={
 "flex items-center gap-1.5 px-3 py-2
+
 (activeAlgo === 'boruvka'
 ? 'bg-rose-600 text-white shadow-md
 : 'text-slate-400 hover:text-slate-
)
 >
 <Users className="w-3.5 h-3.5" />
 Борувка (Билет 20)
 </button>
 </div>

 <button
 onClick={() => handleReset(activeAlgo)}
 className="flex items-center justify-cent
-200 font-semibold text-xs rounded-xl borde
 >
 <RotateCcw className="w-3.5 h-3.5" />
 Сбросить
 </button>

 <button
 onClick={handleNextStep}
 disabled={stepIndex >= currentSteps.length}
 className={
 "flex items-center justify-center gap-2
initial cursor-pointer " +
 (stepIndex >= currentSteps.length
 ? "bg-slate-800 text-slate-500 border
 : activeAlgo === 'kruskal'
 ? "bg-indigo-600 hover:bg-indigo-500
 : activeAlgo === 'prim'

```



```

 >
 {vertex.label}
 {vertex.id === 4 && activeAlgo === 'p
 <span className="text-[8px] block -l
 }}
 </div>
)})
</div>

{stepIndex >= currentSteps.length && (
 <div className="absolute inset-x-6 bottom
 shadow-xl z-20">
 <p className={"font-bold text-base " +
 'ld-300' : 'text-rose-300'})}>
 MST построено с помощью {activeAlgo
 </p>
 <p className="text-slate-300 text-xs mt
 Общий вес минимального остова: 51. Вс
 </p>
 </div>
 })
</div>

{ /* Logs / Heap */}
<div className="bg-slate-950 rounded-2xl bord
 <div className="p-4 bg-slate-900 border-b b
 <h4 className="font-bold text-slate-200 t
 Ход выполнения
 </h4>
 </div>

 {activeAlgo === 'prim' && (
 <div className="p-3 bg-slate-900/60 borde
 <p className="text-[11px] font-bold tex
 ⚡ Приоритетная очередь (Min-Heap):
 </p>
 <div className="flex flex-wrap gap-1.5"
 {heapQueue.length === 0 ? {

```

```

 </div>
 </div>
 </div>
 </div>

 /* Cheat Sheet: Complexity & Code for all 3 algo
 <div className="bg-slate-900/80 border border-sla
 <div className="flex flex-col sm:flex-row items
 <div className="flex items-center gap-2.5">
 <BookOpen className="w-5 h-5 text-indigo-400
 <h4 className="text-xl font-bold text-white
 </div>

 <div className="flex bg-slate-950 p-1 rounded
 <button
 onClick={() => setActiveCheatTab('kruskal
 className={"px-3 py-1.5 rounded text-xs f
 == 'kruskal' ? "bg-indigo-600 text-white" : "
 >
 Краскал (Билет 18)
 </button>
 <button
 onClick={() => setActiveCheatTab('prim')}
 className={"px-3 py-1.5 rounded text-xs f
 == 'prim' ? "bg-emerald-600 text-white" : "
 >
 Прима (Билет 19)
 </button>
 <button
 onClick={() => setActiveCheatTab('boruvka
 className={"px-3 py-1.5 rounded text-xs f
 == 'boruvka' ? "bg-rose-600 text-white" : "
 >
 Борувка (Билет 20)
 </button>
 </div>
 </div>

```

```

 r_i, r_j = find(i), find(j)
 if r_i != r_j:
 parent[r_i] = r_j
 return True
 return False

```

# 2. Жадный выбор ребер

edges.sort() #  $O(E \log E)$

mst = []

for w, u, v in edges:

if union(u, v):

mst.append((u, v, w))`}

</pre>

</div>

</div>

</div>

}}

{activeCheatTab === 'prim' && {

<div className="grid grid-cols-1 lg:grid-cols-

<div className="lg:col-span-1 space-y-4">

<div className="bg-slate-950 p-4 rounded-

<p className="text-slate-400 text-xs for

<Clock className="w-3.5 h-3.5 text-em

</p>

<p className="text-2xl font-black text-r

<p className="text-slate-400 text-xs mt

</div>

<div className="bg-slate-950 p-4 rounded-

<p className="text-slate-400 text-xs for

<Award className="w-3.5 h-3.5 text-em

</p>

<p className="text-2xl font-black text-r

<p className="text-slate-400 text-xs mt

</div>

<div className="bg-slate-950 p-4 rounded-

<p className="text-slate-400 text-xs for

ложении:</p>

```

 <p className="text-slate-400 text-xs font-medium">
 <Clock className="w-3.5 h-3.5 text-rose-500" />
 </p>
 <p className="text-2xl font-black text-rose-500">
 <p className="text-slate-400 text-xs font-medium">
 <div>
 </div>
 <div className="bg-slate-950 p-4 rounded-2xl">
 <p className="text-slate-400 text-xs font-medium">
 <Award className="w-3.5 h-3.5 text-rose-500" />
 </p>
 <p className="text-2xl font-black text-rose-500">
 <p className="text-slate-400 text-xs font-medium">
 </div>
 <div className="bg-slate-950 p-4 rounded-2xl">
 <p className="text-slate-400 text-xs font-medium">
 ении:</p>
 <p className="text-xs text-slate-300 leading-5">
 , организовав массовое распараллеленное слияние
 </p>
 </div>
 </div>
 <div className="lg:col-span-2">
 <div className="bg-slate-950 p-4 rounded-2xl">
 <p className="text-slate-400 text-xs font-medium">
 <Code className="w-3.5 h-3.5 text-rose-500" />
 </p>
 <pre className="text-xs text-emerald-400 font-mono">
 80 flex-1">
 {`def boruvka(n, edges):
 parent = list(range(n))
 mst = []
 while len(mst) < n - 1:
 # Для каждого колхоза ищем самый дешёвый мост на
 cheapest = [-1] * n
 for u, v, w in edges:
 ru, rv = find(u), find(v)
 if ru != rv:
 if cheapest[ru] == -1 or cheapest[ru][2]

```

```

 badge: 'text-blue-400/70 bg-blue-950/40',
 },
 {
 img: bellmanImg,
 alt: 'Фура по болоту Форд-Беллман',
 title: ' Фура по Болоту (Ф-Б)',
 desc: 'Медленный, тяжёлый – зато тащит',
 highlight: 'отрицательные',
 highlightColor: 'text-rose-400',
 rest: 'веса и детектит отрицательные циклы.',
 complexity: 'O(V · E)',
 border: 'border-rose-500/30 hover:border-rose-400/60',
 titleColor: 'text-rose-400',
 badge: 'text-rose-400/70 bg-rose-950/40',
 },
 {
 img: floydImg,
 alt: 'Все-ко-Все Флойд',
 title: ' Все ко Все (Флойд)',
 desc: 'Матрица + три цикла {',
 highlight: 'k, i, j',
 highlightColor: 'text-emerald-400',
 rest: '}'. Ищет пути от всех ко всем.',
 complexity: 'O(V³)',
 border: 'border-emerald-500/30 hover:border-emerald-400/60',
 titleColor: 'text-emerald-400',
 badge: 'text-emerald-400/70 bg-emerald-950/40',
 },
];

```

```

export default function MnemonicCards() {
 return (
 <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
 {cards.map((c, i) => (
 <div key={i} className={`bg-slate-800 rounded-2xl p-4">
 <div className="h-48 overflow-hidden">
 <img src={c.img} alt={c.alt}
 className="w-full h-full object-cover gro

```



```

const [open, setOpen] = useState(false);

useEffect(() => {
 document.body.style.overflow = open ? "hidden" : "";
 return () => {
 document.body.style.overflow = "";
 };
}, [open]);

useEffect(() => {
 const onKey = (e: KeyboardEvent) => e.key === "Escape";
 window.addEventListener("keydown", onKey);
 return () => window.removeEventListener("keydown", onKey);
}, []);

return (
 <>
 <div className="lg:hidden sticky top-[57px] z-40">
 <button
 onClick={() => setOpen(true)}
 className="w-full flex items-center gap-2.5 p-2"
 aria-label="Открыть содержание"
 >

 <List className="w-4 h-4" />

 Содержание • {index + 1} из {total}

 {title}

 </button>
 </div>

 {open && (
 <div className="lg:hidden fixed inset-0 z-[90]">
 <div className="absolute inset-0 bg-slate-950">

```

```

 activeTab: 'guide' | 'simulator' | 'compiler';
 setActiveTab: (tab: 'guide' | 'simulator' | 'compiler'
}
```

```
export const Navbar: React.FC<NavbarProps> = ({ activeTab
 const [busy, setBusy] = useState(false);
```

```
 const saveBook = async () => {
 setBusy(true);
 try {
 await downloadBookHtml(chapters);
 } finally {
 setBusy(false);
 }
 };
};
```

```
 return (
 <header className="bg-slate-900 border-b border-slate-800">
 <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
 <div className="flex items-center gap-3 w-full">
 <div className="bg-gradient-to-tr from-indigo-500 to-indigo-600 h-6 w-6 rounded-sm">
 <Sparkles className="w-6 h-6" />
 </div>
 <div className="min-w-0 flex-1">
 <h1 className="text-xl font-extrabold text-white">
 Универсальное пособие
 </h1>
 <p className="text-xs text-indigo-400 font-medium">
 Подготовка к экзаменам: Алгоритмы, Структуры данных
 </p>
 </div>
 </div>
 </div>

 <div className="flex items-center w-full lg:w-auto">
 >
 <button
```

```

 href={` ${import.meta.env.BASE_URL}export/full_code_all_pages.pdf`
 download="full_code_all_pages.pdf"
 target="_blank"
 rel="noreferrer"
 className="flex items-center justify-center
 over:bg-emerald-600/20 border border-emerald-500/20"
 title="Скачать полный PDF сборник всех страниц"
 >
 <FileDownload className="w-4 h-4" /> Скачать PDF

 <a
 id="download-txt-btn"
 href={` ${import.meta.env.BASE_URL}export/full_code_all_pages.txt`
 download="full_code_all_pages.txt"
 target="_blank"
 rel="noreferrer"
 className="flex items-center justify-center
 :bg-sky-600/20 border border-sky-500/30 transition"
 title="Скачать полный TXT файл со всем кодом"
 >
 <FileText className="w-4 h-4" /> TXT

 <button
 id="download-html-btn"
 onClick={saveBook}
 disabled={busy}
 className="flex items-center justify-center
 er:bg-amber-600/20 border border-amber-500/20"
 title="Скачать всё пособие одним автономным файлом"
 >
 {busy ? <Loader2 className="w-4 h-4 animate-spin" /> : "Скачать"}
 </button>
</div>
</div>
</header>
);
};

```

```

 }
 return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)
 && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)
 }
 const crosses: number[] = [];
 for(let i=0;i<edges.length;i++) {
 for(let j=i+1;j<edges.length;j++) {
 if (intersect(edges[i][0],edges[i][1],
 edges[j][0],edges[j][1])) {
 if (!crosses.includes(i)) crosses.push(i);
 if (!crosses.includes(j)) crosses.push(j);
 }
 }
 }
 const lines = []; const circles = [];
 for(let i=0;i<edges.length;i++) {
 const p1=findPt(edges[i][0]),p2=findPt(edges[i][1]);
 const xing = crosses.includes(i);
 lines.push(<line key={`l${i}`} x1={p1.x} x2={p2.x}
 stroke={xing ? "#f43f5e" : "#4f46e5"} />);
 }
 const labels = ["A","B","C","D","E"];
 for(const p of pts) {
 circles.push(<g key={`c${p.id}`}>
 <circle cx={p.x} cy={p.y} r={8} fill="#4f46e5" />
 <text x={p.x} y={p.y+3} textAnchor="middle">{labels[p.id]}</text>
 </g>);
 }
 return <>{lines}{circles}</>;
 })()
</svg>
<div className="absolute bottom-2 left-2 right-2 top-20"
 z-20">
 V=5, E=10 {'>'} 3V-6 = 9 {'-->'} {'\u274C'}
</div>
</div>

{/* K3,3 */}
<div className="bg-slate-950 rounded-xl p-4 border">

```

```

const labels = ["\u04141", "\u04142", "\u04143"];
for(const p of pts) {
 circles.push(<g key={`c${p.id}`}>
 <circle cx={p.x} cy={p.y} r={8} fill="black">
 <text x={p.x} y={p.y+3} textAnchor="middle">{p.id}</text>
 </g>);
}
return <>{lines}{circles}</>;
})();
</svg>
<div className="absolute bottom-2 left-2 right-2 top-2 z-20">
 V=6, E=9 {'>'} 2V-4 = 8 (двудольный) {'-->'}
</div>
</div>
</div>

<div className="bg-amber-950/40 border border-amber-500 p-10">
 ▲ Запомни: K5 и K3,3 – два кита непланарности.
 гомеоморфный K5 или K3,3 – он непланарен. Теорема Кёрши.
</div>
</div>
);
}

```

ФАЙЛ 56/85: src/components/PythonCompiler.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useCallback, useEffect, useMemo, useRef, useState } from 'react';
import {
 ArrowRightLeft,
 BookOpen,
 CheckCircle2,
 Copy,
 Check,

```

```

function getPythonRuntime() {
 if (!runtimePromise) {
 runtimePromise = import(/* @vite-ignore */ PYODIDE_URL)
 .then((module as PyodideModule).loadPyodide({ indexURL: PYODIDE_URL }));
 }
 return runtimePromise;
}

function errorText(error: unknown) {
 if (error instanceof Error) return error.message;
 return String(error);
}

/**
 * Стартовое содержимое редактора для страницы.
 *
 * ВАЖНО: здесь намеренно НЕТ кода алгоритма — только коды
 * переменными, по которым идёт пошаговая визуализация
 * Студент сам меняет их значения, повторяя шаги демонстрации.
 */
function buildSyncTemplate(chapterId: string, chapterTitle: string) {
 const sync = getPageSync(chapterId);
 const bar = "# " + "=".repeat(56);
 const lines: string[] = [bar, "# СИНХРОНИЗАЦИЯ С ВИЗУАЛИЗАЦИЕЙ"];
 if (sync.vizTitle) lines.push(`# Демонстрация: ${sync.vizTitle}`);
 lines.push(bar, "#");

 if (sync.variables.length === 0) {
 lines.push(
 "# На этой странице нет интерактивной визуализации",
 "# компилятор работает в свободном режиме.",
 "# Откройте страницу с демонстрацией (стрелки ← и →)",
 "# и здесь появятся её переменные синхронизации."
);
 } else {
 lines.push("# Пошаговая подсветка на визуализации");
 for (const v of sync.variables) {

```

```

 savedEditor &&
 savedEditor.chapterId !== chapterId &&
 savedEditor.code !== savedEditor.template &&
 savedEditor.code.trim() !== "" &&
 savedEditor.code !== template
 ? chapterId
 : null
);
const prevTemplateRef = useRef(template);

// запоминаем состояние редактора, чтобы возврат на в
useEffect(() => {
 savedEditor = { code, template, chapterId };
}, [code, template, chapterId]);

// Смена страницы: если редактор не тронут (пуст или
// тихо подставляем переменные новой страницы. Свой к
// а предлагаем синхронизироваться кнопкой.
useEffect(() => {
 if (template === prevTemplateRef.current) return;
 const prevTemplate = prevTemplateRef.current;
 prevTemplateRef.current = template;
 if (code === prevTemplate || code.trim() === "") {
 setCode(template);
 setDesyncedFrom(null);
 } else {
 setDesyncedFrom(chapterId);
 }
 // eslint-disable-next-line react-hooks/exhaustive-
}, [template]);

const resync = useCallback(() => {
 prevTemplateRef.current = template;
 setCode(template);
 setDesyncedFrom(null);
}, [template]);

const runCode = useCallback(async () => {

```

```

 }
 }, [code]));

const hasViz = sync.variables.length > 0;

return (
 <main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
 <div className="max-w-6xl mx-auto">
 <div className="flex flex-col sm:flex-row sm:items-center">
 <div>
 <div className="inline-flex items-center gap-2">
 <Terminal className="w-4 h-4" /> Боковая панель
 </div>
 <h2 className="text-2xl sm:text-3xl font-extrabold">
 Редактор синхронизирован с текущей страницей
 </h2>
 <p className="text-slate-400 mt-2 max-w-2xl">
 Редактор синхронизирован с текущей страницей,
 по которым идёт пошаговая визуализация. На
 </p>
 </div>
 <a
 href="https://pyodide.org/"
 target="_blank"
 rel="noreferrer"
 className="inline-flex items-center gap-1.5"
 >
 Как это работает <ExternalLink className="w-4 h-4" />

 </div>

 {/* Панель синхронизации с визуализацией */}
 <div className="mb-5 rounded-2xl border border-slate-200">
 <div className="flex flex-wrap items-center gap-2">
 <Tooltip
 side="bottom"
 content={
 hasViz
 ? "Редактор и визуализация смотрят на одну и ту же страницу"
 : "На выбранной странице нет интерактивных элементов"
 }
 />
 </div>
 </div>
 </div>
 </main>
);

```



```

 >
 <Unlink className="w-3.5 h-3.5" />
 Страница сменилась – подставить её пер
 </button>
</Tooltip>
 })
</div>

{hasViz && (
 <div className="mt-3 flex flex-wrap items-c
 <span className="inline-flex items-center
 <Variable className="w-3.5 h-3.5" /> Пер

 {sync.variables.map((v) => (
 <Tooltip
 key={v.name}
 side="bottom"
 content={

 <span className="block font-bold
 <code className="font-mono text

 {v.role}<
 {v.range && <span className="bloc

 }
 }
 >
 <code
 tabIndex={0}
 className="cursor-help rounded-md b
 bold text-emerald-300 hover:bg-emerald-500/
 >
 {v.name}
 </code>
 </Tooltip>
)))
 {sync.stepNote && (
 <Tooltip side="bottom" content={`Шаг ви

```

```

 >
 <button
 type="button"
 onClick={resync}
 className="inline-flex items-center
slate-800 transition-colors"
 >
 <RotateCcw className="w-3.5 h-3.5" />
 </button>
 </Tooltip>
 <Tooltip side="bottom" content="Запустить"
 >
 <button
 type="button"
 onClick={() => void runCode()}
 disabled={running}
 className="inline-flex items-center
y-60 text-white text-xs font-bold transition"
 >
 {running ? <Loader2 className="w-3.5 h-3.5" /> : "Запустить"}
 </button>
 </Tooltip>
 </div>
 </div>

 <textarea
 value={code}
 onChange={(event) => setCode(event.target.value)}
 onKeyDown={(event) => {
 if ((event.ctrlKey || event.metaKey) && event.defaultPrevented) {
 void runCode();
 }
 }}
 spellCheck={false}
 aria-label="Код Python"
 className="block w-full min-h-[380px] resize-none focus:ring-2 focus:ring-inset focus:ring-blue-500"
 />

```

```

 {runtimeReady && <CheckCircle2 className="text-emerald-500"/>
 {runtimeReady ? "готов" : "не загружен"}

 </Tooltip>
 </div>
 <pre className="min-h-[380px] max-h-[560px] text-slate-300 leading-6 text-slate-300">
 {output}
 </pre>
 <div className="border-t border-slate-800 p-4">
 </section>
</div>

<div className="mt-5 grid md:grid-cols-3 gap-3">
 <div className="flex gap-2 rounded-xl border border-slate-800 p-4">
 <Info className="w-4 h-4 shrink-0 text-indigo-500"/>
 Ctrl или Cmd + Enter запускает код. В консоли браузера.
 </div>
 <div className="flex gap-2 rounded-xl border border-slate-800 p-4">
 <Info className="w-4 h-4 shrink-0 text-amber-500"/>
 Первый запуск скачивает примерно несколько килобайт кода.
 </div>
 <div className="flex gap-2 rounded-xl border border-slate-800 p-4">
 <Info className="w-4 h-4 shrink-0 text-emerald-500"/>
 Код выполняется в браузере и не отправляется на сервер.
 </div>
</div>
</div>
</main>
);
}

```

```

const [servedHistory, setServedHistory] = useState<st
const [isServing, setIsServing] = useState(fa
const [shakeEmpty, setShake] = useState(fa
const [log, setLog] = useState<st

const addLog = (msg: string) => setLog(prev => [msg,

// ENQUEUE: Встать в конец очереди (хвост / Tail)
const enqueue = useCallback(() => {
 if (isServing) return;
 if (queue.length >= 6) {
 addLog('⚠ Хвост очереди уже на улице, повару тяже
 setShake(true);
 setTimeout(() => setShake(false), 400);
 return;
 }

 const preset = PRESETS[counter % PRESETS.length];
 counter++;

 const newGuest: Customer = {
 id: Date.now().toString(),
 ...preset,
 animState: 'in',
 };

 setQueue(prev => [...prev, newGuest]);
 addLog(`- ENQUEUE: ${newGuest.name} ${newGuest.emoj

 setTimeout(() => {
 setQueue(prev => prev.map(c => c.id === newGuest.
 }, 450);
}, [queue, isServing]);

// DEQUEUE: Выдать суп первому (голова / Head, FIFO)
const dequeue = useCallback(() => {
 if (isServing) return;

```

```

 });
 setServedHistory([]);
 setIsServing(false);
 setLog([' Столовая сброшена. Снова голодная толпа!']);
 setTimeout(() => {
 setQueue(prev => prev.map(c => ({ ...c, animState: 'idle' })), 500);
 });
 };

 return (
 <div className="flex flex-col gap-6">
 {/* МНЕМОНИКА / ПРАВИЛО */}
 <div className="bg-indigo-950/40 border border-indigo-500 p-6">
 <div className="text-3xl"> </div>
 <div>
 <h3 className="font-black text-indigo-400 text-2xl">ПРАВИЛО</h3>
 <p className="text-xs text-slate-300 mt-1 leading-4">
 В очереди за супом всё честно. Кто первый встал, тот и ест.
 Новые гости встают строго в конец очереди.
 Здесь нет обхода и блата – только строгий порядок.
 </p>
 </div>
 </div>

 {/* УПРАВЛЕНИЕ */}
 <div className="flex items-center gap-3 flex-wrap">
 <button
 onClick={enqueue}
 disabled={!isServing}
 className="flex-1 min-w-[150px] bg-gradient-to-r from-indigo-500 to-indigo-600
 text-white opacity-40 disabled:cursor-not-allowed text-center font-medium
 active:scale-95 transition-all cursor-pointer">
 Встать в очередь (ENQUEUE)
 </button>

 <button
 onClick={dequeue}

```

```

 </div>
 <div className="text-center">
 <
 <span className="text-[10px] font-bol
ase font-mono tracking-wider">
 ПОВАР

 </div>
 </div>

```

```

 { /* РАЗДЕЛИТЕЛЬНАЯ СТРЕЛКА */ }

```

```

 <div className="flex flex-col items-cente
 <span className="text-2xl animate-pulse
 <span className="text-[8px] font-mono t
 </div>

```

```

 { /* СПИСОК ОЧЕРЕДИ */ }

```

```

 <div className="flex-1 flex items-end gap
 {queue.length === 0 ? {
 <div className="flex-1 flex flex-col
 </
 <p className="text-xs font-bold fon
 <p className="text-[10px]">Все сыты
 </div>
 } : {
 queue.map((customer, idx) => {
 const isHead = idx === 0;
 const isTail = idx === queue.length
 return (
 <div
 key={customer.id}
 className={`
 flex flex-col items-center ga
 ${customer.animState === 'in'
 ${customer.animState === 'eat
 ${customer.animState === 'out
 `}
 >

```

```
</div>
```

```
{/* Пол кухни */}
```

```
<div className="w-full h-3 bg-gradient-to-r from-blue-500 to-purple-500">
```

```
</div>
```

```
{/* ПРАВАЯ КОЛОНКА: СЫТЫЕ И ДОВОЛЬНЫЕ */}
```

```
<div className="md:col-span-3 bg-slate-900 rounded-2xl p-4">
```

```
<div className="absolute top-4 left-4 text-[12px] font-mono">
```

```
Сытые гости (Архив FIFO)
```

```
</div>
```

```
<div className="absolute top-4 right-4 flex items-center justify-center gap-2" style={{
 order: emerald-800/80, text: [9px], font: mono}}>
```

```
<Sparkles className="w-3.5 h-3.5 animate-pulse text-emerald-400">
```

```
СЫТЫ
```

```
</div>
```

```
<div className="flex-1 flex flex-col justify-between p-4">
```

```
{servedHistory.length === 0 ? (
```

```
<div className="flex-1 flex flex-col items-center justify-center gap-2">
```

```
Здесь 6 гостей
```

```
<p className="text-xs font-bold font-mono">Всего: 6</p>
```

```
<p className="text-[10px] mt-1">Здесь 6 гостей</p>
```

```
</div>
```

```
) : (
```

```
<div className="flex flex-col gap-1.5 w-full">
```

```
{servedHistory.map((item, idx) => (
```

```
<div
```

```
key={idx}
```

```
className="w-full py-1.5 px-3 rounded-2xl">
```

```
<div className="flex items-center gap-2 text-[11px] text-emerald-400">
```

```
>
```

```
{item}
```

```
</div>
```

```
)))
```

```
</div>
```

```
})
```

```

 fail: number;
 term_link: number;
 is_terminal: boolean;
 words: string[];
 depth: number;
 x?: number;
 y?: number;
}

export const SalmonAutomatonWidget: React.FC = () => {
 const [words, setWords] = useState<string[]>(['CAR',
 const [newWord, setNewWord] = useState('');
 const [nodes, setNodes] = useState<{ [key: number]: T
 const [simulationStep, setSimulationStep] = useState<
 const [bfsQueue, setBfsQueue] = useState<number[]>([]
 const [currentNode, setCurrentNode] = useState<number
 const [speechText, setSpeechText] = useState<string>('
 'Привет! Я Эхо-Лосось Сеня! Введи слова в словарь и
 буль!')
);
 const [isTalking, setIsTalking] = useState<boolean>(f
 const [isBlinking, setIsBlinking] = useState<boolean>

 // Регулярное моргание рыбы
 useEffect(() => {
 const blinkInterval = setInterval(() => {
 setIsBlinking(true);
 setTimeout(() => setIsBlinking(false), 200);
 }, 4000);
 return () => clearInterval(blinkInterval);
 }, []);

 // Эффект разговора при смене текста
 useEffect(() => {
 setIsTalking(true);
 const timer = setTimeout(() => setIsTalking(false),
 return () => clearTimeout(timer);
 }, [speechText]);

```



```

const layoutDFS = (u: number, depth: number) => {
 const childKeys = Object.values(newNodes[u].children);
 newNodes[u].y = 40 + depth * paddingY;

 if (childKeys.length === 0) {
 newNodes[u].x = 40 + currentX * paddingX;
 currentX++;
 } else {
 let leftX = -1;
 let rightX = -1;
 childKeys.forEach(v => {
 layoutDFS(v, depth + 1);
 if (leftX === -1) leftX = newNodes[v].x!;
 rightX = newNodes[v].x!;
 });
 newNodes[u].x = leftX === -1 ? 40 + currentX * paddingX : leftX + paddingX;
 if (leftX === -1) currentX++;
 }
};

layoutDFS(0, 0);

setNodes(newNodes);
setSimulationStep(0);
setBfsQueue([]);
setCurrentNode(null);
setSpeechText(
 `Отлично! Я построил базовое префиксное дерево (БПД) для слова "S", чтобы расставить fail-ссылки!`
);
};

useEffect(() => {
 buildInitialTrie(words);
}, []);

const handleAddWord = (e: React.FormEvent) => {

```

```

 оче суффикса просто нет. Жми «Следующий шаг»!'
 });
};

// Один шаг BFS
const stepBFS = () => {
 if (bfsQueue.length === 0) {
 setSimulationStep(2);
 setCurrentNode(null);
 setSpeechText(
 'Ура! Очередь пуста! Мы успешно построили полный

 ь полюбоваться таблицей переходов!'
);
 return;
 }

 const r = bfsQueue[0];
 const newQueue = bfsQueue.slice(1);
 const updatedNodes = { ...nodes };
 const rNode = updatedNodes[r];
 const childrenIds = Object.values(rNode.children);

 let logs = `Рассматриваем вершину #${r} (${rNode.ch

 for (const [char, u] of Object.entries(rNode.children)) {
 newQueue.push(u);
 let v = rNode.fail;
 while (v !== 0 && updatedNodes[v].children[char] !== undefined) {
 v = updatedNodes[v].fail;
 }
 const failTarget = updatedNodes[v].children[char];
 updatedNodes[u].fail = failTarget;

 // Рассчитываем терминальную ссылку (term_link)
 if (updatedNodes[failTarget].is_terminal) {
 updatedNodes[u].term_link = failTarget;
 } else {
 updatedNodes[u].term_link = updatedNodes[failTa

```

```

 </button>
 </div>
</div>

{/* Верхняя панель: Говорящий лосось и диалоговое
<div className="grid grid-cols-1 md:grid-cols-3 g
 {/* Аватар Лосося (Сеня) с анимацией моргания и
 <div className="flex flex-col items-center just
 <div className="w-40 h-40 relative flex items
 full border border-blue-500/30 shadow-inner
 {/* SVG Лосося */}
 <svg
 viewBox="0 0 200 200"
 className={`w-36 h-36 transition-transform
 isTalking ? 'scale-105 drop-shadow-[0_0_
]`}
 >
 {/* Тело лосося */}
 <ellipse cx="100" cy="100" rx="70" ry="45"
 <path d="M 40 85 Q 100 55 160 85 Q 100 115
 40 85"
 fill="#e67e22"
 className="origin-[35px_100px] animate-
 />

 {/* Хвост */}
 <path
 d="M 35 100 L 5 70 L 15 100 L 5 130 Z"
 fill="#be123c"
 className="origin-[35px_100px] animate-
 />

 {/* Плавники */}
 <path d="M 90 55 L 70 30 L 110 55 Z" fill="#f87171"
 <path d="M 90 145 L 70 170 L 110 145 Z" fill="#f87171"
 <path
 d="M 120 115 L 100 135 L 130 130 Z"
 fill="#f87171"
 className="origin-[120px_115px] animate-
 />

 {/* Глаз (с морганием) */}

```

```
<div className="flex items-center space-x-2">
 <Volume2 className={`w-4 h-4 ${isTalking ? 'animate-pulse' : ''}`} />
 Прямое вещание из аквариума:
</div>
```

```
<p className="text-slate-200 text-base leading-relaxed">
 {speechText}
</p>
```

```
<div className="mt-4 pt-3 border-t border-slate-200">
 <div className="text-xs text-slate-400 flex justify-between">
 <Info className="w-3.5 h-3.5 text-blue-400" />
 Статус: {simulationStep === 0 ? 'Борется с вирусом!' : 'Восстановлен!'}
 </div>
</div>
```

```
<div className="flex gap-2">
 {simulationStep === 0 && (
 <button
 onClick={startBFS}
 className="bg-emerald-600 hover:bg-emerald-500 text-white px-4 py-2 shadow-lg shadow-emerald-900/40 transition-colors"
 >
 <Play className="w-4 h-4" /> Запустить симуляцию
 </button>
)}
 {simulationStep === 1 && (
 <button
 onClick={stepBFS}
 className="bg-blue-600 hover:bg-blue-500 text-white px-4 py-2 shadow-lg shadow-blue-900/40 transition-colors"
 >
 Шаг BFS
 1/3
 </button>
)}
 {simulationStep === 2 && (
 <button
 onClick={endBFS}
 className="bg-red-600 hover:bg-red-500 text-white px-4 py-2 shadow-lg shadow-red-900/40 transition-colors"
 >
 Завершить
 </button>
)}
</div>
```

```

 { /* Draw Edges */
 {Object.values(nodes).map(node => {
 if (node.id === 0) return null;
 const parent = nodes[node.parent];
 if (!parent.x || !parent.y || !node.x || !node.y) return null;

 const isCurrent = currentNode === node.id;

 return (
 <g key={`edge-${node.id}`}>
 <line
 x1={parent.x}
 y1={parent.y + 16}
 x2={node.x}
 y2={node.y - 16}
 stroke={isCurrent ? '#60a5fa' : '#ccc'}
 strokeWidth="2"
 markerEnd="url(#arrowhead)"
 />
 <text
 x={(parent.x + node.x) / 2 - 10}
 y={(parent.y + node.y) / 2}
 fill="#94a3b8"
 fontSize="12"
 fontWeight="bold"
 >
 {node.char}
 </text>
 </g>
);
 })}

 { /* Draw Fail Links */
 {simulationStep > 0 && Object.values(nodes).map(node => {
 if (node.id === 0 || node.fail === 0) return null;
 // n draw all! But maybe wait, we can just draw fail link
 // Wait, all nodes get fail link. Let's
 })}
 }
}

```

```

 } else if (node.is_terminal) {
 fill = '#059669';
 stroke = '#34d399';
 }

 return (
 <g key={`node-${node.id}`}>
 <circle
 cx={node.x}
 cy={node.y}
 r="16"
 fill={fill}
 stroke={stroke}
 strokeWidth="2"
 />
 <text
 x={node.x}
 y={node.y + 4}
 fill="white"
 fontSize={isRoot ? "10" : "12"}
 fontWeight="bold"
 textAnchor="middle"
 >
 {isRoot ? 'R' : node.char}
 </text>
 {!isRoot && (
 <text
 x={node.x + 12}
 y={node.y - 12}
 fill="#94a3b8"
 fontSize="9"
 >
 {node.id}
 </text>
)}
 </g>
);
 }}}

```

```

 maxLength={8}
 className="bg-slate-800 border border-slate-700 outline-none focus:border-blue-500"
 />
 <button
 type="submit"
 className="bg-slate-700 hover:bg-slate-600 transition-colors"
 >
 <Plus className="w-3.5 h-3.5" /> Добавить
 </button>
 </form>
 </div>

```

```

 { /* Таблица Вершин и суффиксных ссылок */ }
 <div className="lg:col-span-2 bg-slate-900/50 p-4" >
 <h5 className="text-white font-bold mb-3 flex" >

 Таблица переходов и fail-ссылок

 Всего вершин: {Object.keys(nodes).length}

 </h5>

```

```

 <div className="max-h-[220px] overflow-y-auto" >
 <table className="w-full text-left border-collapse" >
 <thead>
 <tr className="border-b border-slate-700" >
 <th className="py-2 px-3" >ID Вершины</th>
 <th className="py-2 px-3" >Символ</th>
 <th className="py-2 px-3" >Родитель</th>
 <th className="py-2 px-3" >fail-ссылка</th>
 <th className="py-2 px-3" >term_link</th>
 <th className="py-2 px-3" >Дети</th>
 <th className="py-2 px-3" >Слова (dict)
 </tr>
 </thead>

```

```

</td>
<td className="py-2.5 px-3">
 {node.id === 0 ? (

 {node.fail}

) : (
 <span className="bg-indigo-950 text-white" style={
 0 transition-colors cursor-help" title={`Ск
 {node.term_link}

)}
 </td>
<td className="py-2.5 px-3 text-slate-500">
 {Object.keys(node.children).length === 0 ? 'Ø' : Object.entries(node.children)
 .map(([c, id]) => `${c}→#${id}`)
 .join(', ')}
 </td>
<td className="py-2.5 px-3">
 {node.words.length === 0 ? (

 {node.words.join(', ')}

) : (
 <span className="bg-emerald-900 text-white" style={
 {node.words.join(', ')}

)}
 </td>
</tr>
);

```



```

 go(2 * v, l, m);
 go(2 * v + 1, m + 1, r);
 tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
 }
 go(1, 0, n - 1);
 return tree;
}

```

// ===== STEP GENERATORS =====

// 1. Build steps (recursive top-down)

```

function genBuildSteps(arr: number[]): Step[] {
 const n = arr.length;
 const steps: Step[] = [];
 let id = 0;
 const tree: Record<number, number> = {};

 const CODE = [
 "build(v, l, r) {", // 1
 " if (l === r) {", // 2
 " tree[v] = A[l]; return", // 3
 " }", // 4
 " mid = (l + r) >> 1", // 5
 " build(2v, l, mid)", // 6
 " build(2v+1, mid+1, r)", // 7
 " tree[v] = tree[2v]+tree[2v+1]", // 8
 "}", // 9
];
 void CODE;
}

```

```

function go(v: number, l: number, r: number) {
 if (l === r) {
 steps.push({ id: id++, desc: `build(${v}, ${l}, ${r}`,
 , highlightNodes: [v], arrayHighlight: [l, r] });
 tree[v] = arr[l];
 steps.push({ id: id++, desc: `tree[${v}] = ${arr[l]}`,
 , highlightNodes: [v], arrayHighlight: [l, r] });
 return;
 }
}

```

```

 codeLine: 6, treeState: tree, highlightNodes: [v
const left = go(2 * v, l, m, qL, qR);
const right = go(2 * v + 1, m + 1, r, qL, qR);
const res = left + right;
steps.push({ id: id++, desc: `Возврат в узел ${v}:
 rgeChildren: [2 * v, 2 * v + 1], arrayHighlight:
return res;
}
const ans = go(1, 0, n - 1, qL, qR);
steps.push({ id: id++, desc: `Запрос sum([${qL}..${q
 ghlightNodes: [1], arrayHighlight: [qL, qR], result
return steps;
}

```

// 3. Bottom-Up Query steps (iterative on implicit tree

function genQueryBottomUpSteps(arr: number[], qL: number

```
const n = arr.length;
```

```
// Build iterative tree: leaves at [n..2n-1]
```

```
const tree: Record<number, number> = {};
```

```
for (let i = 0; i < n; i++) tree[n + i] = arr[i];
```

```
for (let i = n - 1; i > 0; --i) tree[i] = (tree[2 * i
```

```
const steps: Step[] = [];
```

```
let id = 0;
```

```
let l = qL + n;
```

```
let r = qR + n + 1; // half-open [l, r)
```

```
let res = 0;
```

```
steps.push({ id: id++, desc: `Старт: l = qL + n = ${q
 `, codeLine: 3, treeState: tree, highlightNodes: [l
```

```
while (l < r) {
```

```
 const highlights: number[] = [];
```

```
 if (l & 1) {
```

```
 res += tree[l] ?? 0;
```

```
 steps.push({ id: id++, desc: `l=${l} нечётный → l
 0}. Сумма = ${res}. l++.` , codeLine: 5, treeSta
```

```
 highlights.push(l);
```

```

 " build(2*v, l, mid);",
 " build(2*v+1, mid+1, r);",
 " tree[v] = tree[2*v] + tree[2*v+1];",
 "}"
];
const QUERY_TD_CODE = [
 "query(v, l, r, ql, qr) {",
 " if (ql > r || qr < l) return 0;",
 " // -----",
 " if (ql <= l && r <= qr) return tree[v];",
 " // -----",
 " mid = (l + r) >> 1;",
 " // спускаемся в обоих детей",
 " return query(left) + query(right);",
 "}"
];
const QUERY_BU_CODE = [
 "queryBU(ql, qr) {",
 " res = 0;",
 " l = ql + n; r = qr + n + 1;",
 " // -----",
 " while (l < r) {",
 " if (l & 1) res += tree[l++];",
 " // -----",
 " if (r & 1) res += tree[--r];",
 " l >>= 1; r >>= 1;",
 " }",
 " return res; }"
];

// ===== PLAYER COMPONENT =====
function Player({ steps, color }: { steps: Step[]; color: string }) {
 const [idx, setIdx] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [speed, setSpeed] = useState(700);

 useEffect(() => { setIdx(0); setPlaying(false); }, [steps]);

 useEffect(() => {

```



```

 = {step.result}

 })
</div>

{/* Controls */}
<div className="px-3 py-2.5 bg-slate-950 border-t-1 border-l-1 border-r-1 border-slate-800" >
 <div className="flex items-center bg-slate-900 rounded" >
 <button onClick={() => { setPlaying(false); setSpeed(1200); }} className="p-1 rounded" title="C6noc"><RotateCcw className="w-3 h-3" />
 <button onClick={() => { setPlaying(false); setSpeed(700); }} className="p-1 rounded" title="C6noc"><RotateCcw className="w-3 h-3" />
 <button onClick={() => setPlaying(!playing)} className="p-1 rounded" title="C6noc"><Play className="w-3 h-3" />
 <button onClick={() => { setPlaying(false); setSpeed(350); }} className="p-1 rounded" title="C6noc"><SlowDown className="w-3 h-3" />
 <button onClick={() => { setPlaying(false); setSpeed(120); }} className="p-1 rounded" title="C6noc"><Stop className="w-3 h-3" />
 </div>
 <select value={speed} onChange={e => setSpeed(Number(e.target.value))} className="w-100px text-[9px] font-bold rounded px-1.5 py-1 outline-1 outline-slate-800" >
 <option value={1200}>Медл.</option>
 <option value={700}>Норма</option>
 <option value={350}>Быстро</option>
 <option value={120}>Турбо</option>
 </select>
</div>

{/* Progress bar */}
<div className="h-1 bg-slate-950" ><div className="bg-rose-500 h-1" style={{width: ((currentStep + 1) / total) * 100 : 0}%` }} /></div>
</div>
);
}

// =====
// MAIN EXPORTED COMPONENT

```

```

<div className="mb-6 space-y-4">
 <div className="flex flex-col lg:flex-row gap-4">
 {/* Array input */}
 <form onSubmit={handleApply} className="flex">
 <div className="flex items-center justify-between">
 <label className="text-[10px] font-bold uppercase">
 label>
 <button type="button" onClick={randomize}>
 "><RefreshCw className="w-3.5 h-3.5" /></button>
 </div>
 <div className="flex gap-2">
 <input
 value={customInput}
 onChange={e => setCustomInput(e.target.value)}
 className="flex-1 bg-slate-900 border border-indigo-500 focus:border-indigo-500"
 placeholder="5, 8, 3, 12, 7, 2"
 />
 <button type="submit" className="bg-indigo-600 text-white px-4 py-2">
 Применить</button>
 </div>
 </form>
 <div className="flex flex-wrap gap-1.5 pt-1">
 {arr.map((v, i) => (

 A[{i}

 </div>
))}

 </div>
 </div>
 {/* Query range */}
 <div className="bg-slate-950 p-4 rounded-xl border border-indigo-500">
 <label className="text-[10px] font-bold uppercase">
 Query range</label>
 <div className="flex items-center gap-2">
 <div className="flex items-center gap-1">
 From
 <input
 type="text"
 value={queryRangeFrom}
 onChange={e => setQueryRangeFrom(e.target.value)}
 className="flex-1 bg-slate-900 border border-indigo-500 focus:border-indigo-500"
 />
 </div>
 <div className="flex items-center gap-1">
 To
 <input
 type="text"
 value={queryRangeTo}
 onChange={e => setQueryRangeTo(e.target.value)}
 className="flex-1 bg-slate-900 border border-indigo-500 focus:border-indigo-500"
 />
 </div>
 </div>
 </div>
 </div>
</div>

```

```

 er:text-slate-200'}`}>
 <Search className="w-3.5 h-3.5" /> Запрос: св
 </button>
 <button onClick={() => setView('theory')} class=
 n-colors ${view === 'theory' ? 'border-blue-5
 te-200'}`}>
 <Info className="w-3.5 h-3.5" /> Почему / Спра
 </button>
</div>

{/* ---- TAB CONTENT ---- */}
{view === 'build' && (
 <SimPanel
 key={`build-${arr.join('-')}`}
 title="Построение дерева (рекурсия)"
 icon=" "
 steps={buildSteps}
 code={BUILD_CODE}
 color="indigo"
 arr={arr}
 />
)}

{view === 'queries' && (
 <div className="grid grid-cols-1 xl:grid-cols-2"
 <SimPanel
 key={`qtd-${arr.join('-')}-${qL}-${qR}`}
 title={`Запрос sum([${qL}..${qR}]) – Сверху`
 icon="↑ "
 steps={queryTDSteps}
 code={QUERY_TD_CODE}
 color="emerald"
 arr={arr}
 />
 <SimPanel
 key={`qbu-${arr.join('-')}-${qL}-${qR}`}
 title={`Запрос sum([${qL}..${qR}]) – Снизу-н
 icon="↑ "

```

```

 <p className="text-xs text-slate-300 mb-3">
 Начинаем с корня. Если отрезок узла полн
 наче спускаемся в обоих детей.
 </p>
 <div className="space-y-1.5 text-xs">
 <div className="flex justify-between bo
 0(
 <div className="flex justify-between bo
 >
 <div className="flex justify-between"><
 d-400 font-bold">✓ легко</div>
 </div>
 </div>
 <div className="bg-slate-950 p-5 rounded-xl"
 <div className="absolute -top-3 left-4 bg
 rder border-amber-500 shadow-md">
 † Запрос Снизу-вверх (Итерация)
 </div>
 <p className="text-xs text-slate-300 mb-3">
 Стартуем с двух указателей (l и r) на л
 ём его левого соседа. Поднимаемся пока l {"
 </p>
 <div className="space-y-1.5 text-xs">
 <div className="flex justify-between bo
 <span className="text-emerald-400 font-mono":
 <div className="flex justify-between bo
 ><span className="text-emerald-400 font-mono
 <div className="flex justify-between"><
 00 font-bold">x сложно</div>
 </div>
 </div>
 </div>
 </div>

 {/* Verdict */}
 <div className="bg-slate-950 p-5 rounded-xl b
 <h4 className="font-bold text-blue-400 mb-2"
 <p className="text-sm text-slate-300 leading
 <strong className="text-white">Итеративны

```



```

 (map.get(key) as typeof chapters).push(c);
 }
 return Array.from(map.entries());
}, [query]);

return (
 <aside className="w-full bg-slate-900/80 lg:bg-tran
 <div className="p-4 pb-3 shrink-0">
 <h3 className="text-xs font-bold text-slate-400">
 <Compass className="w-4 h-4 text-indigo-400" />
 </h3>
 <div className="relative">
 <Search className="w-4 h-4 text-slate-500 abs
 <input
 value={query}
 onChange={(e) => setQuery(e.target.value)}
 placeholder="Поиск по темам.."
 className="w-full bg-slate-800/70 border bo
 500 focus:outline-none focus:border-indigo-!
 />
 {query && (
 <button
 onClick={() => setQuery("")}
 className="absolute right-2 top-1/2 -tran
 aria-label="Очистить поиск"
 >
 <X className="w-3.5 h-3.5" />
 </button>
)}
 </div>
 </div>

 <div className="flex-1 overflow-y-auto px-3 pb-4">
 {groups.length === 0 && <p className="text-sm t

 {groups.map(([category, items]) => (
 <div key={category}>
 <div className="text-[10px] font-bold upperc

```

```
 <div className="shrink-0 m-3 mt-0 bg-gradient-to-r from-indigo-500 to-purple-500 p-3 text-white" style={styles.container}>
 <div className="font-bold text-indigo-400 flex items-center justify-between" style={styles.header}>
 <Sparkles className="w-3.5 h-3.5" /> Как пользоваться?
 </div>
 <p className="leading-relaxed">
 Подчёркнутые термины в тексте раскрываются по нажатию на стрелки.
 </p>
 <p className="text-slate-400 italic">Стрелки ← и → указывают на подчёркнутые термины.
 </p>
 </div>
 </aside>
);
};
```

---

ФАЙЛ 61/85: src/components/Simulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState } from 'react';
import { Layers, AlignLeft, Award, Plus, ArrowRight, ArrowLeft } from 'lucide-react';
```

```
interface Plate {
 id: string;
 name: string;
 icon: string;
}
```

```
interface Person {
 id: string;
 name: string;
 icon: string;
}
```

```
interface Hypothesis {
 id: string;
```

```

 { name: 'Чашка от кофе', icon: '☺' },
 { name: 'Миска с остатками салата', icon: ' ' },
 { name: 'Тарелка от тако', icon: ' ' },
];
 const randomPreset = presets[Math.floor(Math.random() * presets.length)];
 const newPlate = {
 id: Date.now().toString(),
 name: randomPreset.name,
 icon: randomPreset.icon
 };
 setStack(prev => [...prev, newPlate]);
 setPopMessage(` Положили поверх стопки: ${newPlate.name} ${newPlate.icon} `);
};

const popPlate = () => {
 if (stack.length === 0) {
 setPopMessage("⚠ Стопка пуста! Все тарелки вымыты");
 return;
 }
 const topPlate = stack[stack.length - 1];
 setStack(prev => prev.slice(0, prev.length - 1));
 setPopMessage(` Вымыта верхняя тарелка (LIFO): ${topPlate.name} ${topPlate.icon} `);
};

const resetStack = () => {
 setStack([
 { id: '1', name: 'Тарелка из-под пасты', icon: '🍝' },
 { id: '2', name: 'Тарелка из-под пиццы', icon: '🍕' },
 { id: '3', name: 'Блюдец от торта', icon: '🍰' },
]);
 setPopMessage(" Стопка тарелок восстановлена.");
};

// Queue handlers
const addPerson = () => {
 const presets = [
 { name: 'Любитель борща', icon: '🥘' },
 { name: 'Курьер доставки', icon: '📦' },
];
 const randomPreset = presets[Math.floor(Math.random() * presets.length)];
 const newPerson = {
 id: Date.now().toString(),
 name: randomPreset.name,
 icon: randomPreset.icon
 };
 setQueue(prev => [...prev, newPerson]);
 setAddMessage(` Добавили в очередь: ${newPerson.name} ${newPerson.icon} `);
};

const removePerson = () => {
 if (queue.length === 0) {
 setAddMessage("🚫 Очередь пуста!");
 return;
 }
 const frontPerson = queue[0];
 setQueue(prev => prev.slice(1));
 setRemoveMessage(` Удален человек из очереди: ${frontPerson.name} ${frontPerson.icon} `);
};

const resetQueue = () => {
 setQueue([
 { id: '1', name: 'Повар', icon: '👨‍🍳' },
 { id: '2', name: 'Помощник повара', icon: '👩‍🍳' },
 { id: '3', name: 'Помощник курьера', icon: '📦' },
]);
 setAddMessage(" Очередь восстановлена.");
};

```

```

 text: `Кандидат: "${newCustomText}"`,
 probability: Number(newCustomProb)
 });
 setHeap(prev => [...prev, item]);
 setNewCustomText('');
 setHeapMessage(`    Добавлена гипотеза с вероятностью`);
};

const popHeap = () => {
 if (heap.length === 0) {
 setHeapMessage("⚠ Куча пуста! Нет кандидатов для");
 return;
 }
 // Find highest probability
 let maxIdx = 0;
 for (let i = 1; i < heap.length; i++) {
 if (heap[i].probability > heap[maxIdx].probability) {
 maxIdx = i;
 }
 }
 const best = heap[maxIdx];
 setHeap(prev => prev.filter((_, idx) => idx !== maxIdx));
 setHeapMessage(`    Выбран самый "тяжелый" кандидат:`);
};

const resetHeap = () => {
 setHeap([
 { id: '1', text: 'Кандидат: "Привет, я искусственный интеллект"', probability: 0.3 },
 { id: '2', text: 'Кандидат: "Привет, я испек вкусный хлеб"', probability: 0.4 },
 { id: '3', text: 'Кандидат: "Привет, я испанский язык"', probability: 0.5 }
]);
 setHeapMessage(" Гипотезы восстановлены.");
};

// Sorted heap for display (Max-Heap property view)
const sortedHeap = [...heap].sort((a, b) => b.probability - a.probability);

return {

```

```

 : 'bg-slate-800/60 border-slate-700/60 text-white'
 }`}
 >
 <div className="flex items-center gap-3">
 <AlignLeft className={`w-5 h-5 ${activeTab === 'fifo' ? 'bg-indigo-500/20 border-indigo-400/20' : 'bg-slate-800/60 border-slate-700/60 text-white'} `}>
 <div className="text-left">
 <div className="font-bold text-sm text-white">FIFO
 <div className="text-xs opacity-80">FIFO
 </div>
 </div>

 BFS

 </button>

 <button
 onClick={() => setActiveTab('heap')}
 className={`flex items-center justify-between gap-3 ${activeTab === 'heap' ? 'bg-emerald-600/20 border-emerald-500/20' : 'bg-slate-800/60 border-slate-700/60 text-white'} `}>
 >
 <div className="flex items-center gap-3">
 <Award className={`w-5 h-5 ${activeTab === 'heap' ? 'bg-emerald-500/20 border-emerald-400/20' : 'bg-slate-800/60 border-slate-700/60 text-white'} `}>
 <div className="text-left">
 <div className="font-bold text-sm text-white">Приоритетная
 <div className="text-xs opacity-80">Приоритетная
 </div>
 </div>

 Beam Search

 </button>
 </div>

 {/* STACK TAB */}
 {activeTab === 'stack' && (

```

```

 {popMessage && {
 <div className="bg-blue-950/60 border border-
 imate-fade-in">
 <span className="inline-block w-2 h-2 round
 {popMessage}
 </div>
)}

 {/* Visualization */}
 <div className="bg-slate-950/60 p-6 rounded-2
 >
 {stack.length === 0 ? {
 <div className="text-center py-12 text-sla
 <div className="text-5xl mb-3"> </div>
 <p className="text-base font-bold">Стор
 <p className="text-xs mt-1">Добавьте гр
 </div>
) : {
 <div className="w-full max-w-sm flex flex
 <div className="absolute top-0 text-xs
 full flex items-center gap-1">
 ВЕРШИНА СТЕКА (TOP)
 <ArrowDown className="w-3.5 h-3.5 anim
 </div>

 {/* Reversed map so top of stack is vis
 {stack.slice().reverse().map(({plate, ind
 const isTop = index === 0;
 return {
 <div
 key={plate.id}
 className={`w-full flex items-cen
 isTop
 ? 'bg-gradient-to-r from-blue
le-102'
 : 'bg-slate-900/90 border-sla
 }`}
 >

```

```

<div className="flex flex-wrap items-center"
 <button
 onClick={addPerson}
 className="flex-1 md:flex-none flex item
-2.5 rounded-xl text-sm font-bold shadow-lg
 >
 <Plus className="w-4 h-4" /> Встать в о
 </button>
 <button
 onClick={servePerson}
 className="flex-1 md:flex-none flex item
er border-indigo-500/40 px-4 py-2.5 rounded
 >
 Налить суп первому
 </button>
 <button
 onClick={resetQueue}
 title="Сбросить"
 className="p-2.5 bg-slate-800 hover:bg-
tion-colors cursor-pointer"
 >
 <RotateCcw className="w-5 h-5" />
 </button>
</div>
</div>

```

```

{dequeueMessage && {
 <div className="bg-indigo-950/60 border bor
p-3 animate-fade-in">
 <span className="inline-block w-2 h-2 roun
 {dequeueMessage}
 </div>
}}

```

```

{/* Visualization */}
<div className="bg-slate-950/60 p-6 rounded-2
overflow-x-auto">
 {queue.length === 0 ? {

```

```


 <span className={`text-[10px] font-
 isFirst ? 'bg-indigo-500/30 text-
 }`}>
 {isFirst ? 'Получает сун' : `В

 </div>
);
 }
}

 <div className="flex flex-col items-cent
-600 min-w-[120px] h-[120px]">
 <Plus className="w-6 h-6 mb-1" />
 <span className="text-xs font-mono up
 </div>
</div>
})
<div className="mt-6 text-xs text-slate-500
 ПЕРВЫМ ПРИШЕЛ → ПЕРВЫМ ПОЛУЧИЛ (FIFO) • И
</div>
</div>
</div>
})

```

```

{ /* HEAP TAB */
{activeTab === 'heap' && {
 <div className="space-y-6">
 <div className="bg-slate-800/80 p-5 rounded-x
 tify-between gap-4">
 <div>
 <h3 className="text-lg font-bold text-whi
 Приоритетная очередь / Куча</sp
 <span className="text-xs font-mono bg-e
 x-Heap
 </h3>
 <p className="text-slate-400 text-xs mt-1
 Здесь не важно, кто пришел первым. Важн
 </p>

```





```
 </div>
 </div>

 <div className="flex items-center" >
 { /* Custom progress bar */ }
 <div className="hidden sm:block" >
 <div
 className={`h-full rounded-
 style={{ width: `${percent}%`
 ></div>
 </div>
 <span className={`font-mono font-
 isTop ? 'text-emerald-400 text-
 }`}>
 {percent}%

 </div>
 </div>
);
 }
}
</div>
</div>
</div>
)
</div>
);
};
```

```

return (
 <div>
 <div className="mb-6">
 <h2 className="text-xl sm:text-2xl font-extrabo
 <p className="text-sm text-slate-400 leading-re
 Все интерактивные демонстрации в одном месте.
 здесь их можно открыть напрямую.
 </p>
 </div>

 <div className="relative mb-6 max-w-md">
 <Search className="w-4 h-4 text-slate-500 absol
 <input
 value={query}
 onChange={(e) => setQuery(e.target.value)}
 placeholder="Поиск тренажёра..."
 className="w-full bg-slate-900 border border-
 focus:outline-none focus:border-indigo-500
 />
 {query && (
 <button
 onClick={() => setQuery("")}
 className="absolute right-2 top-1/2 -transl
 aria-label="Очистить поиск"
 >
 <X className="w-4 h-4" />
 </button>
)}
 </div>

 {items.length === 0 && <p className="text-slate-5

 <div className="grid gap-3 sm:grid-cols-2 xl:grid
 {items.map(({ id, entry, chapterTitle }) => (
 <div
 key={id}
 className="flex flex-col bg-slate-900 borde
 >

```

-----  
import { CheckCircle2, Dices, Eye, EyeOff, RotateCcw, U

/\*\*

\* Песочница поворотов.

\*

\* Здесь ничего не подсказывается: дано настоящее дерев

\* поднять отмеченный узел в корень. Клик по узлу = оди

\* Порядок поворотов выбираешь сам; правильный ли он по

\* только после того, как узел окажется наверху (или по

\*/

export interface TNode {

key: number;

left: TNode | null;

right: TNode | null;

}

const leaf = (key: number): TNode => ({ key, left: null

const clone = (t: TNode | null): TNode | null =>

t ? { key: t.key, left: clone(t.left), right: clone(t

/\*\* Заготовки: линия (zig-zig), змейка (zig-zag) и прос

export const PRESETS: { name: string; hintCase: string;

{

name: "Линия влево",

hintCase: "Zig-Zig",

target: 1,

build: () => {

const n1 = leaf(1);

const n2: TNode = { key: 2, left: n1, right: leaf

const n4: TNode = { key: 4, left: n2, right: leaf

return { key: 6, left: n4, right: leaf(7) };

},

},

{

name: "Змейка",

hintCase: "Zig-Zag",

```
};
```

```
/** Поднять узел key на один уровень (поворот с родителем)
```

```
export function rotateUp(root: TNode, key: number): TNode {
```

```
 const path = findPath(root, key);
```

```
 if (path.length < 2) return root;
```

```
 const x = path[path.length - 1];
```

```
 const p = path[path.length - 2];
```

```
 const gg = path.length >= 3 ? path[path.length - 3] : null;
```

```
 if (p.left === x) {
```

```
 p.left = x.right;
```

```
 x.right = p;
```

```
 } else {
```

```
 p.right = x.left;
```

```
 x.left = p;
```

```
 }
```

```
 if (!gg) return x;
```

```
 if (gg.left === p) gg.left = x;
```

```
 else gg.right = x;
```

```
 return root;
```

```
}
```

```
/** Какой поворот сделал бы настоящий splay из текущего
```

```
export function canonicalNext(root: TNode, key: number): TNode {
```

```
 const path = findPath(root, key);
```

```
 if (path.length < 2) return null;
```

```
 const x = path[path.length - 1];
```

```
 const p = path[path.length - 2];
```

```
 const g = path.length >= 3 ? path[path.length - 3] : null;
```

```
 if (!g) return { node: x.key, kind: "Zig" };
```

```
 const xLeft = p.left === x;
```

```
 const pLeft = g.left === p;
```

```
 if (xLeft === pLeft) return { node: p.key, kind: "Zig" };
```

```

/* ----- КОМПОНЕНТ -----

export const SplayRotationSandbox: React.FC = () => {
 const [presetIdx, setPresetIdx] = useState(0);
 const preset = PRESETS[presetIdx];

 const [tree, setTree] = useState<TNode>(() => preset.root);
 const [history, setHistory] = useState<TNode[]>([]);
 const [moves, setMoves] = useState<{ node: number; correct: boolean }>([]);
 const [showAnswer, setShowAnswer] = useState(false);

 const target = preset.target;
 const solved = tree.key === target;

 const reset = useCallback(
 (idx = presetIdx) => {
 setPresetIdx(idx);
 setTree(PRESETS[idx].build());
 setHistory([]);
 setMoves([]);
 setShowAnswer(false);
 },
 [presetIdx]
);

 const { nodes, edges, width, height } = useMemo(() => preset.build(), [preset]);
 const path = useMemo(() => new Set(findPath(tree, target)), [tree, target]);
 const expected = useMemo(() => (solved ? null : canonicalNext(tree, target)), [tree, target]);

 const clickNode = (key: number) => {
 if (solved) return;
 const p = findPath(tree, key);
 if (p.length < 2) return; // корень крутить не с кем

 const want = canonicalNext(tree, target);
 setHistory((h) => [...h, clone(tree) as TNode]);
 setMoves((m) => [...m, { node: key, correct: want?.key === key }]);
 };
}

```

```

 className={`px-3 py-1.5 rounded-lg text-[11pt]
 i === presetIdx ? "bg-indigo-600 text-white" : "text-slate-600"
 }`
 >
 {p.name}
 </button>
)))
 </div>

```

```

<div className="bg-slate-900 rounded-xl border border-slate-700" style={{ width: 400px, height: 400px }}>
 <svg
 viewBox={`0 0 ${width} ${height}`}
 className="h-auto block mx-auto"
 style={{ minWidth: Math.min(width * 1.5, 420), height: 400px }}
 role="img"
 >
 {edges.map(([a, b], i) => {
 const na = nodes.find((n) => n.key === a);
 const nb = nodes.find((n) => n.key === b);
 if (!na || !nb) return null;
 const onPath = path.has(a) && path.has(b);
 return (
 <line
 key={i}
 x1={na.x}
 y1={na.y}
 x2={nb.x}
 y2={nb.y}
 stroke={onPath ? "#6366f1" : "#475569"}
 strokeWidth={onPath ? 3 : 1.8}
 style={{ transition: "all 700ms cubic-bezier(0.16, 1, 0.3, 0.9)" }}
 />
);
 })}

 {nodes.map((n) => {
 const isTarget = n.key === target;
 const clickable = !solved && n.depth > 0;

```

```

 onClick={() => reset()}
 className="flex items-center gap-1.5 px-3 py-3 border rounded-lg text-slate-500 font-bold transition-colors"
 >
 <RotateCcw className="w-3.5 h-3.5" /> Заново
 </button>
 <button
 onClick={() => setShowAnswer((s) => !s)}
 className={`flex items-center gap-1.5 px-3 py-3 border rounded-lg text-slate-500 font-bold transition-colors
 ${showAnswer
 ? "bg-amber-600/20 border-amber-500 text-amber-500"
 : "bg-slate-800 border-slate-700 text-slate-500"}
 `}
 >
 {showAnswer ? <EyeOff className="w-3.5 h-3.5" /> : "Скрыть разбор" />
 </button>

 поворотов: {moves.length} • минимум: {minimalMoves}

 </div>

 {showAnswer && !solved && expected && (
 <div className="mt-3 text-[12px] bg-amber-950/40 border rounded-lg padding-3">
 Сейчас splay крутил бы узел{" "}
 {expected}
 </div>
)}

 {solved && (
 <div
 className={`mt-3 rounded-lg border px-3 py-2.5 text-slate-500 font-medium
 ${wrong === 0
 ? "bg-emerald-950/40 border-emerald-700/60"
 : "bg-rose-950/30 border-rose-800/60 text-rose-500"}
 `}
 >

```



```

* Разбор трёх случаев операции Splay: Zig, Zig-Zig, Zig-Zag.
*
* Что сделано ради понятности:
* * у узлов настоящие числа (20 / 40 / 60), а не абстрактные;
* * сразу видно, что дерево остаётся деревом поиска;
* * под деревом печатается обход слева направо: он ОДНОУРОВНЕВНЫЙ;
* * это и есть доказательство, что поворот ничего не ломает;
* * кадр «прицел» ничего не двигает, только подсвечивает;
* * на кадре «результат» остаются призраки – пунктирные узлы в
* * местах и стрелки «откуда → куда»;
* * по умолчанию анимация НЕ крутится сама: пользователь
* * идёт в своём темпе. Автопрокрутка включается кнопкой.
*/
```

```
type NodeId = "x" | "p" | "g" | "A" | "B" | "C" | "D";
type Pos = Partial<Record<NodeId, [number, number]>>;
```

```
interface Frame {
 pos: Pos;
 edges: [NodeId, NodeId][];
 kind: "start" | "aim" | "result";
 title: string;
 text: string;
 /** Пара, которую крутим на этом шаге: кадр-«прицел». */
 focus?: [NodeId, NodeId];
 /** Кто переехал на этом кадре – рисуем призрак и стрелку. */
 moved?: NodeId[];
 ms: number;
}
```

```
interface Case {
 key: "zig" | "zig-zig" | "zig-zag";
 label: string;
 when: string;
 rotations: string;
 /** Номера узлов: роль → число. */
 keys: Partial<Record<NodeId, number>>;
 /** Обход слева направо – одинаковый на всех кадрах. */
}
```

```

 focus: ["p", "x"],
 ms: AIM_MS,
 },
 {
 pos: { x: [160, 50], A: [80, 130], p: [245, 130],
 edges: [["x", "A"], ["x", "p"], ["p", "B"], ["p",
 kind: "result",
 title: "Стало: 20 – корень",
 text: "40 стало правым сыном двадцатки. Поддерев
 ева от 40 – это одно и то же место.",
 moved: ["x", "p", "B"],
 ms: RESULT_MS,
 },
],
};

```

/\* ----- Zig-Zig -----

```

const ZZ_S0 = {
 pos: { g: [255, 40], D: [325, 112], p: [160, 112], C:
 edges: [["g", "p"], ["g", "D"], ["p", "x"], ["p", "C"]
} as unknown as Pick<Frame, "pos" | "edges">;

```

```

const ZZ_S1 = {
 pos: { p: [165, 40], x: [85, 112], C: [212, 112], g:
 edges: [["p", "x"], ["p", "g"], ["x", "A"], ["x", "B"]
} as unknown as Pick<Frame, "pos" | "edges">;

```

```

const ZIGZIG: Case = {
 key: "zig-zig",
 label: "Zig-Zig",
 when: "x и p – дети с ОДНОЙ стороны (оба слева или об
 rotations: "2 поворота, первым – верхний",
 keys: { x: 20, p: 40, g: 60 },
 inorder: ["A", "20", "B", "40", "C", "60", "D"],
 ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
 frames: [
 {
 ...ZZ_S0,

```

```

 moved: ["x", "p", "A", "B"],
 ms: RESULT_MS,
 },
],
};

/* ----- Zig-Zag -----
const ZG_S0 = {
 pos: { g: [258, 40], D: [328, 112], p: [148, 112], A:
 edges: [["g", "p"], ["g", "D"], ["p", "A"], ["p", "x"]
} as unknown as Pick<Frame, "pos" | "edges">;

const ZG_S1 = {
 pos: { g: [258, 40], D: [328, 112], x: [148, 112], p:
 edges: [["g", "x"], ["g", "D"], ["x", "p"], ["x", "C"]
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIGZAG: Case = {
 key: "zig-zag",
 label: "Zig-Zag",
 when: "x и p – дети с РАЗНЫХ сторон (змейка)",
 rotations: "2 поворота, первым – нижний",
 keys: { x: 40, p: 20, g: 60 },
 inorder: ["A", "20", "B", "40", "C", "60", "D"],
 ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
 frames: [
 {
 ...ZG_S0,
 kind: "start",
 title: "Было: 60 → влево на 20 → вправо на 40",
 text: "Узлы идут «змейкой»: сначала шаг влево, по
 ",
 ms: RESULT_MS,
 },
 {
 ...ZG_S0,
 kind: "aim",
 title: "Прицел 1: НИЖНЕЕ ребро 20 – 40",

```

```

g: { fill: "#f59e0b", stroke: "#fcd34d" },
sub: { fill: "#1e293b", stroke: "#475569" },
};

const ROLE_RU: Partial<Record<NodeId, string>> = {
 x: "x — ищем его",
 p: "p — родитель",
 g: "g — дед",
};

const isSubtree = (id: NodeId) => id === "A" || id ===

const Tree: React.FC<{ frame: Frame; prev: Pos; keys: C
 const { pos, edges, focus, moved } = frame;
 const dim = (id: NodeId) => (focus ? !focus.includes(
 const ghosts = frame.kind === "result" ? (moved ?? [])

 return (
 <svg viewBox={`0 0 ${VB.w} ${VB.h}`} className="w-f
 <defs>
 <marker id="splay-arrow" viewBox="0 0 10 10" re
 <path d="M0,0 L10,5 L0,10 z" fill="#a5b4fc" />
 </marker>
 </defs>

 /* призраки: где узел стоял до поворота */
 {ghosts.map((id) => {
 const from = prev[id]!;
 const to = pos[id]!;
 const r = isSubtree(id) ? 15 : 20;
 const dx = to[0] - from[0];
 const dy = to[1] - from[1];
 const len = Math.hypot(dx, dy) || 1;
 const pad = r + 8;
 return (
 <g key={`ghost-${id}`} opacity={0.55}>
 <circle cx={from[0]} cy={from[1]} r={r} fil
 {len > pad * 2 + 6 && {

```

```

const sub = isSubtree(id);
const c = COLOR[sub ? "sub" : id];
const r = sub ? 15 : 20;
const hot = Boolean(focus && focus.includes(id));
return (
 <g
 key={id}
 style={{
 transform: `translate(${p[0]}px, ${p[1]}px)`,
 transition: MOVE,
 opacity: dim(id) ? 0.28 : 1,
 }}
 >
 {hot && <circle r={r + 7} fill="none" stroke={c.stroke} />}
 <circle r={r} fill={c.fill} stroke={c.stroke} />
 <text
 textAnchor="middle"
 dominantBaseline="central"
 fontSize={sub ? 12 : 15}
 fontWeight="700"
 fill={sub ? "#94a3b8" : "#fff"}
 >
 {sub ? id : keys[id]}
 </text>
 {!sub && (
 <text textAnchor="middle" y={-r - 7} font-size={10}
 >{id}</text>
)}
 </g>
);
}}}
</svg>
);
};

```

```

export const SplayRotationsViz: React.FC = () => {
 const [caseIdx, setCaseIdx] = useState(0);

```

```

<div className="mb-3 text-xs sm:text-[13px] text-
 Кор,
 ({active.rotat
</div>

```

/\* шаг + пояснение стоят НАД картинкой: сначала

```

<div className="mb-3 rounded-xl border border-sla
 <div className="flex items-center gap-2 mb-1 fl
 <span
 className={`text-[10px] font-bold uppercase
 current.kind === "aim"
 ? "bg-amber-500/20 text-amber-300"
 : current.kind === "start"
 ? "bg-slate-700 text-slate-300"
 : "bg-indigo-500/20 text-indigo-300"
 }`
 >
 {current.kind === "aim" ? "прицел" : current

 <span className="text-sm font-bold text-white
 Шаг {idx + 1} из {total}. {current.title}

 </div>
 <p className="text-[13px] text-slate-400 leading
</div>

```

```

<div className="bg-slate-900 rounded-xl border bo
 <Tree frame={current} prev={prevPos} keys={acti

```

/\* обход слева направо – одинаковый на всех ка

```

<div className="mt-2 pt-2 border-t border-slate
 <div className="flex items-center gap-1.5 fle
 Обход
 {active.inorder.map((t, i) => (
 <span
 key={i}
 className={`px-1.5 py-0.5 rounded font-r
 /\d/.test(t) ? "bg-emerald-500/15 tex

```

```

 setPlaying(false);
 setFrame((f) => (f + 1) % total);
 }}
 className="px-4 py-2 rounded-lg bg-indigo-600
 w-indigo-900/40"
 >
 {last ? "⏮ Сначала" : "Дальше →"}
</button>
<button
 onClick={() => setPlaying((p) => !p)}
 className="flex items-center gap-1.5 px-3 py-1
 ext-xs font-bold transition-colors"
 >
 {playing ? <Pause className="w-3.5 h-3.5" />
 {playing ? "Стоп" : "Авто"}}
</button>
<button
 onClick={() => setSlow((s) => !s)}
 title="Автопрокрутка ещё медленнее"
 className={`flex items-center gap-1.5 px-3 py-1
 slow
 ? "bg-emerald-600/20 border-emerald-500 text-emerald-500"
 : "bg-slate-800 border-slate-700 text-slate-200"}
 >
 <Gauge className="w-3.5 h-3.5" /> {slow ? "0.5x" : "1x"}
</button>
<button
 onClick={() => {
 setFrame(0);
 setPlaying(false);
 }}
 className="flex items-center gap-1.5 px-3 py-1
 ext-xs font-bold transition-colors"
 >
 <RotateCcw className="w-3.5 h-3.5" /> Сброс
</button>

```

```
// Simple BST insertion to build initial tree
const insertBST = (root: SplayNode | null, key: number) => {
 if (!root) return { key, left: null, right: null };
 if (key < root.key) {
 root.left = insertBST(root.left, key);
 } else if (key > root.key) {
 root.right = insertBST(root.right, key);
 }
 return root;
};
```

```
// Right Rotation (Zig / Right Rotate)
const rightRotate = (x: SplayNode): SplayNode => {
 const y = x.left;
 if (!y) return x;
 x.left = y.right;
 y.right = x;
 return y;
};
```

```
// Left Rotation (Zag / Left Rotate)
const leftRotate = (x: SplayNode): SplayNode => {
 const y = x.right;
 if (!y) return x;
 x.right = y.left;
 y.left = x;
 return y;
};
```

```
// Clone tree helper
const cloneTree = (node: SplayNode | null): SplayNode | null => {
 if (!node) return null;
 return {
 key: node.key,
 left: cloneTree(node.left),
 right: cloneTree(node.right),
 x: node.x,
 };
};
```



```

while (current) {
 lastNode = current;
 if (key === current.key) {
 path.push({ node: current, dir: null });
 break;
 } else if (key < current.key) {
 path.push({ node: current, dir: "left" });
 current = current.left;
 } else {
 path.push({ node: current, dir: "right" });
 current = current.right;
 }
}

const targetKey = current ? key : lastNode.key;
const isFound = !!current;

if (!isFound) {
 steps.push(`Поиск ${key}: элемент отсутствует в д...`);
 steps.push(
 `Поиск споткнулся на узле [${lastNode.key}]. И...`
);
} else {
 steps.push(
 `Поиск ${key}: элемент найден! Запускаем подъем`
);
}

// Now let's implement standard Splay logic recursively
const splayAction = (
 node: SplayNode | null,
 k: number,
): SplayNode | null => {
 if (!node || node.key === k) return node;

 if (k < node.key) {
 if (!node.left) return node;

```

```

 }

 if (!node.right) return node;
 steps.push(
 `Финальный поворот Zag (Левое вращение корня)`
);
 return leftRotate(node);
 }
};

const finalRoot = splayAction(cloneTree(root), target);
return { newRoot: finalRoot, steps };
};

const handleSplay = (key: number) => {
 setHighlightedNode(key);
 const { newRoot, steps } = splayStepByStep(tree, key);
 if (newRoot) {
 setTree(newRoot);
 }
 setLog(steps);
};

// @ts-expect-error зарезервировано под кнопку вставки
const handleInsert = (e: React.FormEvent) => {
 e.preventDefault();
 const val = parseInt(inputValue);
 if (isNaN(val)) return;

 // Standard BST insert, then splay!
 let newTree = cloneTree(tree);
 newTree = insertBST(newTree, val);
 const { newRoot, steps } = splayStepByStep(newTree, val);
 if (newRoot) {
 setTree(newRoot);
 }
 setLog([`Вставили вершину [${val}].`, ...steps]);
 setInputValue("");
};

```

```
// Render lines and nodes
const renderLines = (node: SplayNode | null): React.R
 if (!node) return [];
 let lines: React.ReactNode[] = [];
 if (node.left && node.left.x && node.left.y) {
 lines.push(
 <line
 key={`l-${node.key}-${node.left.key}`}
 x1={node.x}
 y1={node.y}
 x2={node.left.x}
 y2={node.left.y}
 stroke="#475569"
 strokeWidth="2"
 />,
);
 lines = lines.concat(renderLines(node.left));
 }
 if (node.right && node.right.x && node.right.y) {
 lines.push(
 <line
 key={`r-${node.key}-${node.right.key}`}
 x1={node.x}
 y1={node.y}
 x2={node.right.x}
 y2={node.right.y}
 stroke="#475569"
 strokeWidth="2"
 />,
);
 lines = lines.concat(renderLines(node.right));
 }
 return lines;
};
```

```
const renderNodes = (node: SplayNode | null): React.R
 if (!node) return [];
 let circles: React.ReactNode[] = [];
```

```

<div className="bg-slate-900 rounded-xl border border-
 {/* Header */}
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <BookOpen className="w-5 h-5 text-indigo-400"
 Интерактивное Splay-дерево
 </h3>
 <p className="text-xs text-slate-400">
 Кликните на вершины дерева, чтобы «вытащить»
 Splay.
 </p>
 </div>

 <div className="grid grid-cols-1 lg:grid-cols-12
 {/* Playfield */}
 <div className="lg:col-span-7 bg-[#0f172a] p-4
 <svg
 className="w-full h-full max-w-[600px] max-h-
 viewBox="0 0 600 320"
 >
 {renderLines(drawTree)}
 {renderNodes(drawTree)}
 </svg>

 {/* Quick interactive controls */}
 <div className="absolute bottom-4 left-4 right-
 <form onSubmit={handleFind} className="flex
 <input
 type="number"
 placeholder="Поиск..."
 value={inputValue}
 onChange={(e) => setInputValue(e.target
 className="w-24 bg-slate-950 text-white
 00 focus:outline-none"
 />
 <button
 type="submit"
 onClick={handleFind}

```

```

<h4 className="text-xs font-bold text-slate-400">
 Ход вращения (Splay-логи)
</h4>
<div className="flex-1 space-y-2 mb-4 overflow-hidden">
 {log.map((step, idx) => (
 <div
 key={idx}
 className={`text-xs p-2.5 rounded transition-colors
 ${idx === log.length - 1
 ? "border-l-2 border-indigo-500 bg-indigo-50"
 : "text-slate-400 bg-slate-900/50"}
 `}
 >
 {step}
 </div>
))}
</div>

<div className="border-t border-slate-800 pt-3">
 <p>

 у самого корня. Делаем 1 вращение.

 </p>
 <p>

 оба левые или оба правые. Вращаем деда, по

 </p>
 <p>

 колено. Вращаем узел в разные стороны.

 </p>
</div>
</div>

```

```

/* Answers Section inside the visualizer widget
<div className="bg-slate-950/80 p-5 border-t border-slate-800">
 <h4 className="text-indigo-400 font-bold text-slate-400">

```

```

<code className="text-rose-400">[60]</code>
углы), дерево будет превращаться в палку
сторону. Первая операция Splay(60) стои
<strong className="text-white">O(n)
глубину n. Вторая Splay(10) заставляет
Постоянный свинг убивает производительн
сложности <strong className="text-white"
</p>
</div>
<div className="mt-3 text-[10px] text-rose-
 Кэшировать свингующие пары на концах – фа
</div>
</div>

<div className="bg-slate-900/60 p-4 rounded-l
 <div>
 <p className="font-bold text-slate-300 mb
 3. Амортизация O(log n)
 </p>
 <p className="text-slate-400 leading-rela
 Парадокс: один запрос за{" "}
 <code className="text-emerald-400">O(n)</code>
 всю структуру вдвое более плоской за сч
 при двойных вращениях {
 <code className="text-emerald-400 font-
 }. Дорогая операция инвестирует в дешев
 операций. Поэтому амортизированное врем
 <strong className="text-white">O(log n)
 </p>
 </div>
</div>
<div className="mt-3 text-[10px] text-emera
 Каждое "растяжение" компенсируется "склад
</div>
</div>
</div>
</div>
</div>
);

```

```

 tate: 'idle' },
 });

const [cleanRack, setCleanRack] = useState<Plate[]>([]);
const [isWashing, setIsWashing] = useState(false);
const [showSoap, setShowSoap] = useState(false);
const [shakeEmpty, setShake] = useState(false);
const [log, setLog] = useState<string[]>([]);

const addLog = (msg: string) => setLog(prev => [...prev, msg]);

// PUSH: Положить грязную тарелку сверху стопки
const pushPlate = useCallback(() => {
 if (isWashing) return;
 if (dirtyStack.length >= 7) {
 addLog('⚠ Стопка слишком высокая, тарелки могут упасть');
 setShake(true);
 setTimeout(() => setShake(false), 400);
 return;
 }

 const preset = PLATE_PRESETS[counter % PLATE_PRESETS.length];
 counter++;

 const newPlate: Plate = {
 id: Date.now().toString(),
 ...preset,
 isDirty: true,
 animState: 'in',
 };

 setDirtyStack(prev => [...prev, newPlate]);
 addLog(`  PUSH: Положили ${newPlate.emoji} сверху с ${newPlate.color} мылом`);

 setTimeout(() => {
 setDirtyStack(prev => prev.map(p => p.id !== newPlate.id ? p : { ...p, animState: 'out' }));
 }, 500);
}, [dirtyStack, isWashing]);

```

```

 setDirtyStack([
 { id: 'r1', name: 'Тарелка из-под спагетти', emoji: '🍝', State: 'in' },
 { id: 'r2', name: 'Тарелка из-под пиццы', emoji: '🍕', imState: 'in' },
 { id: 'r3', name: 'Блюдец от торта', emoji: '🍰', mState: 'in' },
]);
 setCleanRack([]);
 setIsWashing(false);
 setShowSoap(false);
 setLog([' Стол сброшен. Посуда снова грязная!']);
 };

```

```

const topIndex = dirtyStack.length - 1;

```

```

return (
 <div className="flex flex-col gap-6">
 {/* МНЕМОНИКА / ПРАВИЛО */}
 <div className="bg-blue-950/40 border border-blue-950/40 p-6">
 <div className="text-3xl"> </div>
 <div>
 <h3 className="font-black text-blue-400 text-2xl">ПРАВИЛО</h3>
 <p className="text-xs text-slate-300 mt-1 leading-tight">
 Ты складываешь грязные тарелки друг на друга.
 А когда приходит время мыть, ты берёшь именно последнюю.
 Попробуй вытащить нижнюю – всё рухнет!
 </p>
 </div>
 </div>

 {/* УПРАВЛЕНИЕ */}
 <div className="flex items-center gap-3 flex-wrap">
 <button
 onClick={pushPlate}
 disabled={isWashing}
 className="flex-1 min-w-[150px] bg-gradient-to-r from-blue-950/40 to-blue-950/0
 disabled:cursor-not-allowed text-white"
 >
 Добавить тарелку
 </button>
 </div>
 </div>
)

```



```

 РАКОВИНА
 </div>

 {/* Мыльные пузыри при мытье */}
 {showSoap && {
 <div className="absolute inset-0 pointer-events-none" >
 <div className="absolute w-6 h-6 rounded-full border-1 border-blue-400" >
 </div>
 <div className="absolute w-4 h-4 rounded-full border-1 border-blue-400" >
 </div>
 <div className="absolute w-5 h-5 rounded-full border-1 border-blue-400" >
 </div>
 <div className="absolute text-4xl anim-crash" >
 </div>
 </div>
 }}

 {dirtyStack.length === 0 ? {
 <div className="flex-1 flex flex-col items-center justify-center" >

 <p className="text-white font-black text-2xl" >
 <p className="text-slate-500 text-xs mt-1" >
 </div>
 } : {
 <div className={`flex-1 flex flex-col justify-center items-center`} >

 {/* Указатель TOP */}
 <div
 className="absolute right-8 flex items-center justify-center"
 style={{ bottom: `${24 + topIndex * 44}px` }} >

 er gap-1">
 <ArrowUp className="w-3.5 h-3.5" />
 ВЕРШИНА (TOP)

 </div>
 }
 }
 }
 }

```

```

<div className="absolute top-4 right-4 flex items-center justify-center"
 order-emerald-800/80 text-[9px] font-mono">
 <Sparkles className="w-3.5 h-3.5 animate-pulse" />
 ЧИСТО
</div>

<div className="flex-1 flex flex-col justify-between"
 {cleanRack.length === 0 ? {
 <div className="flex-1 flex flex-col item-center justify-center"

 <p className="text-xs font-bold font-mono">Пусто!
 <p className="text-[10px] mt-1">Здесь будет сушилка
 </div>
) : {
 <div className="flex flex-col gap-2 w-full"
 {cleanRack.map(plate => {
 <div
 key={plate.id}
 className="w-full h-8 rounded-full border border-slate-200"
 items-center justify-center gap-2 opacity-0.8">
 {plate.name}
 {plate.status}
 </div>
 </div>
 })}
 </div>
)}
</div>

{/* Столешница сушки */}
<div className="w-full h-4 bg-gradient-to-r from-emerald-500 to-teal-500" />
</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-800 p-4"
 <div className="text-[10px] font-mono text-slate-400">

```

```

 while (j > 0 && s[i] !== s[j]) {
 steps.push({ i, j, pi: [...pi], highlight:
 яд назад: откат j = pi[${j} - 1] = ${pi[j] -
 j = pi[j] - 1};
 }

 if (s[i] === s[j]) {
 steps.push({ i, j, pi: [...pi], highlight:
 ска увеличивается.` });
 j++;
 } else {
 steps.push({ i, j, pi: [...pi], highlight:
 овпадений нет.` });
 }

 pi[i] = j;
 steps.push({ i, j, pi: [...pi], desc: `Записывае

 if (j === pattern.length) {
 steps.push({ i, j, pi: [...pi], found: true
 }
 }
 return { s, steps, patternLength: pattern.length };
}

```

```

function generateZSteps(pattern: string, text: string)
 if (!pattern) pattern = " ";
 const s = pattern + "#" + text;
 const n = s.length;
 const z = new Array(n).fill(0);
 const steps: any[] = [];

 let l = 0, r = 0;
 steps.push({ i: 0, l, r, z: [...z], desc: `Начало.

 for (let i = 1; i < n; i++) {
 steps.push({ i, l, r, z: [...z], desc: `Шаг i=$

```

```

const [mode, setMode] = useState<"kmp" | "z">(defaultMode);
const [pattern, setPattern] = useState("aba");
const [text, setText] = useState("abacaba");

// Ensure inputs are valid dynamically
const safePattern = pattern || " ";
const safeText = text || " ";

const { s, steps, patternLength } = useMemo(() => {
 if (mode === "kmp") return generateKmpSteps(safePattern, safeText);
 else return generateZSteps(safePattern, safeText);
}, [mode, safePattern, safeText]);

const [autoPlay, setAutoPlay] = useState(false);
const [stepIdx, setStepIdx] = useState(0);
const timer = useRef<NodeJS.Timeout | null>(null);

// Reset when inputs or mode change
useEffect(() => {
 setStepIdx(0);
 setAutoPlay(false);
}, [s, mode]);

// Timer loop for autoPlay
useEffect(() => {
 if (autoPlay) {
 timer.current = setInterval(() => {
 setStepIdx(prev => {
 if (prev >= steps.length - 1) { setAutoPlay(false);
 return prev + 1;
 });
 }, 1200);
 }
 return () => { if (timer.current) clearInterval(timer.current); };
}, [autoPlay, steps.length]);

const playPause = () => setAutoPlay(!autoPlay);
const nextStep = () => { setAutoPlay(false); setStepIdx(stepIdx + 1); };

```

```

 {s.split("").map((char, index) => {
 const isPattern = index < pattern.length;
 const isSeparator = index === pattern.length;

 let borderColor = "border-slate-900";
 let bgColor = "bg-slate-900";
 let textColor = isSeparator ? "text-white" : "text-slate-900";

 // KMP Logic
 if (mode === "kmp") {
 if (step.highlight?.include(char)) {
 borderColor = "border-amber-400";
 bgColor = step.match === char ? "bg-amber-400" : "bg-slate-900/40";
 }
 }

 // Z Logic
 if (mode === "z") {
 if (index >= step.l && index <= step.r) {
 bgColor = "bg-emerald-900";
 borderColor = "border-emerald-900";
 }
 if (step.zTarget === index) {
 borderColor = "border-amber-400";
 bgColor = step.match === char ? "bg-amber-400" : "bg-slate-900/40";
 }
 }

 return (
 <div key={index} className={
 /* L/R markers for Z */
 {mode === "z" && step.l <= index && index <= step.r
 ? "border-l-2 border-r-2 border-slate-900"
 : ""}
 >
 {char}
 </div>
);
 })}
);
}

```

```

<div className="abs
 e z-20">
 <span className
 </div>
 })
 {mode === "z" && step.z
 <div className="abs
 <span className
 </div>
 })
 {mode === "z" && step.z
 <div className="abs
 <span className
 </div>
 })
 {/* Found flash effect
 {step.found && mode ===
 <div className="abs
ulse z-0 bg-emerald-500/20 shadow-[0_0_15px
 })
 {step.found && mode ===
 <div className="abs
ulse z-0 bg-emerald-500/20 shadow-[0_0_15px
 })
 </div>
)
 }
 </div>
</div>

<div className="flex flex-col sm:flex-row g
 {/* Controls */}
 <div className="flex bg-slate-900 borde
 <button onClick={prevStep} disabled
rounded-lg disabled:opacity-30 disabled:hov
 <Undo strokeWidth={3} size={16}
 </button>

```

```

 </p>
 </div>

 { /* Code Sneak Peek */
 <div className="bg-slate-900 border border-
 <div className="bg-slate-800 px-4 py-2 b
late-400 uppercase">
 Код C++ {mode === 'kmp' ? 'КМП
 </div>
 <pre className="p-4 text-xs font-mono te
{mode === 'kmp' ? `vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
}` : `int l = 0, r = 0;
for (int i = 1; i < n; i++) {
 if (i <= r) z[i] = min(r - i + 1, z[i - l]);
 while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
 if (i + z[i] - 1 > r) {
 l = i;
 r = i + z[i] - 1;
 }
}
}` }

 </pre>
 </div>
 </div>

);
}

```

---

ФАЙЛ 68/85: src/components/Tooltip.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState } from "react";
```

```
 <span
 className={`absolute left-1/2 h-2 w-2 -transl
 side === "top" ? "-bottom-[5px] border-b bo
 }`}
 />

);
};
```

---

ФАЙЛ 69/85: src/components/TopologicalSortViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

---

```
import React, { useState, useEffect } from "react";
import {
 Play,
 Pause,
 RotateCcw,
 Clock,
} from "lucide-react";
```

```
interface TNode {
 id: number;
 x: number;
 y: number;
 label: string;
 name: string;
}
```

```
interface TEdge {
 u: number;
 v: number;
}
```

```
const dagNodes: TNode[] = [
```



```

 desc,
 visited: new Set(visited),
 fullyProcessed: new Set(fullyProcessed),
 currentNode,
 topoList: [...topoList],
 dfsStack: [...dfsStack],
 });
};

recordStep(
 "Начало топологической сортировки. Используем кла
 null,
);

// Adj list
const adj: number[][] = Array.from({ length: 5 }, () => []);
dagEdges.forEach((e) => adj[e.u].push(e.v));

const dfs = (u: number) => {
 visited.add(u);
 dfsStack.push(u);
 recordStep(
 `DFS: зашли в вершину ${u} ("${dagNodes[u].name}
 u,
);

 for (const to of adj[u]) {
 if (!visited.has(to)) {
 dfs(to);
 } else if (!fullyProcessed.has(to)) {
 // Explaining potential cycle detected (not in
 recordStep(
 `⚠ Внимание: Рёбра в серую вершину ${to} о
 u,
);
 }
 }
}

```

```

useEffect(() => {
 let timer: NodeJS.Timeout;
 if (isPlaying && currentStepIdx < steps.length - 1)
 timer = setInterval(() => {
 setCurrentStepIdx((prev) => prev + 1);
 }, 1600);
 } else {
 setIsPlaying(false);
 }
 return () => clearInterval(timer);
}, [isPlaying, currentStepIdx, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
 setIsPlaying(false);
 setCurrentStepIdx(0);
};

const getTopoListString = () => {
 return [...step.topoList]
 .reverse()
 .map((id) => dagNodes[id].name)
 .join(" → ");
};

return (
 <div className="bg-slate-900 rounded-xl border border-
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <Clock className="w-5 h-5 text-indigo-400" />
 Топологическая сортировка (Зависимости обучени
 </h3>

 <div className="flex items-center gap-2 bg-slate-
 <button
 onClick={togglePlay}
 className="flex items-center gap-2 px-3 py-
 500 text-white"

```

```

 refX="24"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#8
 </marker>
</defs>

{/* Edges */}
{dagEdges.map(({edge, idx}) => {
 const u = dagNodes.find((n) => n.id === e
 const v = dagNodes.find((n) => n.id === e

 const isActive =
 (step.currentNode === edge.u || step.cu
 step.visited.has(edge.v));

 return (
 <line
 key={`edge-${idx}`}
 x1={u.x}
 y1={u.y}
 x2={v.x}
 y2={v.y}
 stroke={isActive ? "#818cf8" : "#334155"}
 strokeWidth={isActive ? "3" : "2"}
 markerEnd={`url(#${isActive ? "dag-ar
 className="transition-all duration-300
 />
);
}})

{/* Nodes */}
{dagNodes.map((node) => {
 const isCurrent = step.currentNode === no
 const isVisited = step.visited.has(node.id

```

```

 fill="white"
 fontSize="11px"
 fontWeight="bold"
 className="pointer-events-none"
 >
 ({node.label}) {node.name.split(" "
 </text>
 <text
 x={node.x}
 y={node.y + 12}
 textAnchor="middle"
 fill="#94a3b8"
 fontSize="9px"
 className="pointer-events-none"
 >
 {node.name.split(" ").slice(1).join
 </text>
 </g>
);
 }}}
</svg>
</div>

{/* Console column */}
<div className="lg:col-span-4 bg-slate-950 p-5
 00 overflow-y-auto">
 <h4 className="text-xs font-bold text-slate-400">
 Статус сортировки
 </h4>

 <div className="bg-slate-900 border border-slate-800 p-4">
 <p className="text-xs text-indigo-300 min-h-[100px]">
 {step.desc}
 </p>
 </div>

 <div className="flex-1 space-y-4 overflow-y-auto">
 {/* Topological List output */}

```

```
 Шаг {currentStepIdx + 1} из {steps.length}

 <div className="flex gap-1">
 <button
 disabled={currentStepIdx === 0}
 onClick={() => setCurrentStepIdx((prev) => prev - 1)}
 className="bg-slate-900 border border-slate-800 text-white"
 >
 Назад
 </button>
 <button
 disabled={currentStepIdx >= steps.length - 1}
 onClick={() => setCurrentStepIdx((prev) => prev + 1)}
 className="bg-slate-900 border border-slate-800 text-white"
 >
 Вперед
 </button>
 </div>
 </div>
</div>
</div>
</div>
);
};
```

---

ФАЙЛ 70/85: src/components/vizRegistry.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React from "react";
```

```
import { ArticulationPointsViz } from "../ArticulationPointsViz";
import BellmanFordViz from "../BellmanFordViz";
import BfsViz from "../BfsViz";
```

```


 Разобрался – проверь себя: ниже то же самое дерево
 не решишь.

 </p>
 <SplayRotationSandbox />
</div>
);

export interface VizEntry {
 /** Заголовок панели/модалки. */
 title: string;
 /** Короткое пояснение, что здесь можно потыкать. */
 hint?: string;
 Component: React.FC;
}

/**
 * Единый реестр интерактивных демонстраций.
 * Используется и для панели под главой, и для всплывающей подсказки
 * и для модального окна «Открыть симулятор».
 */
export const VIZ_REGISTRY: Record<string, VizEntry> = {
 "stack-dfs": {
 title: "Стек и обход в глубину",
 hint: "Кладите и снимайте тарелки – это ровно то, что нужно",
 Component: StackViz,
 },
 "queue-bfs": {
 title: "Очередь и обход в ширину",
 hint: "Очередь слева, живой BFS по графу справа.",
 Component: () => (
 <div className="space-y-10">
 <QueueViz />
 <BfsViz />
 </div>
),
 },
};

```

```

 hint: "Сверху – покадровый разбор поворота, снизу –
 Component: () => <SplayRotationsPair />,
},
"graph-dfs-bfs": { title: "DFS и BFS на графе", Component: () => {
"top-sort": { title: "Топологическая сортировка", Component: () => {
"scc-kosaraju": { title: "Компоненты сильной связности", Component: () => {
"graph-articulation": { title: "Точки сочленения", Component: () => {
"bridges-code": { title: "Мосты: DFS + tin/low", Component: () => {
"euler-path-vs-cycle": { title: "Эйлеров путь и цикл", Component: () => {
"planarity-euler-formula": { title: "Планарность: K5 и K3,3", Component: () => {
dijkstra: {
 title: "Дейкстра",
 hint: "Жадно забираем ближайшую вершину и релаксируем",
 Component: () => {
 <>
 <ChapterImage vizType="dijkstra" />
 <DijkstraViz />
 </>
 },
},
"bellman-ford": {
 title: "Форд–Беллман",
 Component: () => {
 <>
 <ChapterImage vizType="bellman-ford" />
 <BellmanFordViz />
 </>
 },
},
floyd: {
 title: "Флойд–Уоршелл",
 Component: () => {
 <>
 <ChapterImage vizType="floyd" />
 <FloydViz />
 </>
 },
},
},

```





```

 setActiveSearchCodeLine(1);
};

const stepSearchSimulation = () => {
 if (currentIndex >= textToScan.length) {
 setIsSearchDone(true);
 setSearchLog('Мы проплыли весь текст! Лосось доволен!');
 setActiveSearchCodeLine(4);
 return;
 }

 const char = textToScan[currentIndex];
 let curr = currentNode;
 let logMsg = `[Символ '${char}']`;

 let jumpType: 'normal' | 'fail' | null = null;
 void jumpType;
 let originalCurr = curr;

 if (TRIE_TRANSITIONS[curr][char] !== undefined) {
 const nextNode = TRIE_TRANSITIONS[curr][char];
 logMsg += `У пусла #${curr} (${WATERFALL_NODES[curr].label})`;
 setJumpPath({ from: curr, to: nextNode, type: 'normal' });
 curr = nextNode;
 setActiveSearchCodeLine(3);
 } else {
 setActiveSearchCodeLine(2);
 let jumps = 0;
 while (curr !== 0 && TRIE_TRANSITIONS[curr][char] === undefined) {
 curr = WATERFALL_NODES[curr].fail;
 jumps++;
 }
 const nextNode = TRIE_TRANSITIONS[curr][char] ?? 0;
 if (originalCurr === 0) {
 logMsg += `В корне нет перехода по '${char}'. Лосось доволен!`;
 setJumpPath(null);
 } else {

```

```

 targetNodeId: 0,
 title: 'Шаг 0 (Depth 0): Вершина #0 (ROOT)',
 desc: 'Начало алгоритма BFS. Корень (ROOT) не обра
 рние вершины сразу инициализируются с fail = RO
 codeLine: 1,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'Ø',
 checkingBranchesFrom: null,
},
{
 targetNodeId: 1,
 title: 'Шаг 1 (Depth 1): Вершина #1 (H)',
 desc: 'Мы обрабатываем первую вершину из очереди -
 просто не существует. fail[H] автоматически нап
 codeLine: 5,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'H',
 checkingBranchesFrom: null,
},
{
 targetNodeId: 7,
 title: 'Шаг 2 (Depth 1): Вершина #7 (S)',
 desc: 'Обрабатываем дочернюю вершину #7 (S) на Dep
 ,
 codeLine: 5,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'S',
 checkingBranchesFrom: null,
},
{
 targetNodeId: 2,
 title: 'Шаг 3 (Depth 2): Вершина #2 (HE)',
 desc: 'Переходим на Depth 2 к вершине #2 (HE). Ро
 а-разведчик плывет в ROOT и ищет ветку по "E".
 codeLine: 3,

```

```

 },
 {
 targetNodeId: 6,
 title: 'Шаг 7 (Depth 3): Вершина #6 (HIS)',
 desc: 'Обрабатываем #6 (HIS) на Depth 3. Родитель
 но ветка "S" → #7 (S) совпадает! fail[HIS] = #7
 codeLine: 4,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'S',
 checkingBranchesFrom: 0,
 },
 {
 targetNodeId: 9,
 title: 'Шаг 8 (Depth 3): Вершина #9 (SHE) — КАК С
 desc: ' МАГИЯ АХО-КОРАСИК! Обрабатываем #9 (SHE)
 ба плывет в #1 (H) и ищет ветку "E". У вершины
 codeLine: 4,
 scoutPos: 1,
 inspectedParentFail: 1,
 checkTargetChar: 'E',
 checkingBranchesFrom: 1,
 },
 {
 targetNodeId: 4,
 title: 'Шаг 9 (Depth 4): Вершина #4 (HERS)',
 desc: 'Финальная вершина #4 (HERS) на Depth 4. Ро
 #7 (S). fail[HERS] = #7 (S). Автомат полностью
 codeLine: 4,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'S',
 checkingBranchesFrom: 0,
 },
};

const currentTrainInfo = TRAINING_STEPS_INFO[trainStep];
const activeFishPosNode = activeMode === 'training' ?

```

```

 >
 <Play className="w-4 h-4" />
 Режим 2: Поиск в тексте
 </button>
</div>
</div>

{/* ПАНЕЛЬ УПРАВЛЕНИЯ В ЗАВИСИМОСТИ ОТ РЕЖИМА */}
{activeMode === 'training' ? (
 /* ПАНЕЛЬ РЕЖИМА ОБУЧЕНИЯ */
 <div className="grid grid-cols-1 lg:grid-cols-3" >
 {/* Управление шагами обучения */}
 <div className="bg-slate-900/90 p-5 rounded-x" >
 <div>
 <h5 className="text-indigo-300 font-bold" >
 Полный обход в ширину (В
 </h5>
 <p className="text-xs text-slate-300 mb-3" >
 Показаны все шаги очереди BFS (начиная с
 аг 8 (SHE → HE), где наглядно видно
 </p>
 <div className="space-y-1.5 mb-6 overflow" >
 {TRAINING_STEPS_INFO.map((step, idx) =>
 <button
 key={idx}
 onClick={() => setTrainStep(idx)}
 className={`w-full text-left px-3 p
 ${
 idx === trainStep
 ? 'bg-indigo-600 text-white fon
 : 'bg-slate-800 text-slate-400
 }` }
 >
 {step
 {idx === trainStep && <span classNa
 </button>
)}}
 </div>
 </div>

```

```

 {currentTrainInfo.codeLine === 1 && <
 тут}
 </div>
 <div className={`p-1.5 rounded flex item
-l-4 border-indigo-400 text-indigo-200 font
 2. let v = fail[r]; // Подъем к
ainInfo.inspectedParentFail].label}}
 {currentTrainInfo.codeLine === 2 && <
к родителю}
 </div>
 <div className={`p-1.5 rounded flex item
l-4 border-amber-500 text-amber-300 font-bo
 3. while (v !== root && trie[v]
 {currentTrainInfo.codeLine === 3 && <
т, возврат в корень}
 </div>
 <div className={`p-1.5 rounded flex item
r-l-4 border-emerald-500 text-emerald-300 f
 4. if (trie[v][char] !== undefin
/span>
 {currentTrainInfo.codeLine === 4 && <
ан IF!}
 </div>
 <div className={`p-1.5 rounded flex item
-4 border-blue-500 text-blue-300 font-bold'
 5. else fail[u] = root; // Bero
 {currentTrainInfo.codeLine === 5 && <
root}
 </div>
 </div>
</div>

<div>
 <h5 className="text-slate-200 font-bold m
 <Lightbulb className="w-4 h-4 text-ambe
 </h5>
 <div className="bg-slate-800 p-4 rounded-:
w-inner min-h-[72px] flex items-center">

```

```

 : 'bg-slate-800 text-slate-500'
 }` }
 >
 {char}

)))
 {currentIndex >= textToScan.length && (
 <span className="bg-emerald-600 text-
 ГОТОВО ✓

)}
</div>

<button
 onClick={stepSearchSimulation}
 disabled={isSearchDone || textToScan.length === 0}
 className={`w-full py-2.5 rounded-lg font-medium
 isSearchDone || textToScan.length === 0
 ? 'bg-slate-700 text-slate-500 cursor-not-allowed'
 : 'bg-gradient-to-r from-blue-600 to-blue-900/40 active:scale-[0.98]'
 }` }
 >
 {currentIndex === 0 ? 'Начать план' : ''}
 <ArrowRight className="w-4 h-4" />
 </button>
</div>
</div>

<div className="lg:col-span-2 bg-slate-900/60 p-4" >
 <div>
 <h5 className="text-white font-bold mb-3">
 ⚡ Текущее действие в коде
 </h5>
 <div className="bg-slate-950 p-4 rounded-lg">
 <div className={`p-1.5 rounded flex items-center gap-2
 border-blue-500 text-blue-300 font-bold` : ''}>
 1. curr = state; char = text[i]

```

```

<div className="bg-gradient-to-b from-blue-950 via-blue-900 to-transparent pointer-events-none"></div>
<div className="absolute top-0 left-1/2 -translate-x-1/2 -hidden shadow-2xl">
 <div className="absolute inset-0 opacity-10 bg-gradient-to-b from-blue-950 via-blue-900 to-transparent pointer-events-none"></div>
 <div className="absolute top-0 left-1/2 -translate-x-1/2">
 <div className="w-8 bg-blue-400 h-full animate-pulse"></div>
 <div className="w-12 bg-blue-300 h-full animate-pulse"></div>
 <div className="w-6 bg-blue-500 h-full animate-pulse"></div>
 </div>

 <div className="flex items-center justify-between">
 <h5 className="text-white font-bold text-base">
 Ветвящийся синий водопад (Борис)
 </h5>
 <div className="flex flex-wrap gap-4 text-xs">

 </div>
 </div>

 <div className="w-full overflow-x-auto custom-scrollbar">
 <svg viewBox="0 0 800 520" className="w-full h-full">
 <!-- Горизонтальные линии и подписи уровней -->
 <g>
 <!--
 {
 depth: 0, y: 60, label: 'Depth 0 (Корень)'
 depth: 1, y: 160, label: 'Depth 1 (H, S)'
 depth: 2, y: 260, label: 'Depth 2 (HE, L)'
 depth: 3, y: 360, label: 'Depth 3 (HER, L)'
 depth: 4, y: 460, label: 'Depth 4 (HERS, L)'
 }
 -->
 {
 <g key={`depth-line-${level.depth}`}>

```

```

 x2={toNode.x}
 y2={toNode.y}
 stroke={isHighlighted ? '#3b82f6' : '#f43f5f'}
 strokeWidth={isHighlighted || isTrainingExamined ? 2 : 1}
 className={isHighlighted || isTrainingExamined ? 'highlighted' : ''}
 />
 { /* Символ перехода */ }
 <circle cx={({fromNode.x + toNode.x} / 2) cy={({fromNode.y + toNode.y} / 2)} fill={isTrainingExamined ? (isMatchBranch ? '#10b981' : '#f43f5f') : 'none'} stroke={stroke} strokeWidth={strokeWidth} />
 <text
 x={({fromNode.x + toNode.x} / 2)
 y={({fromNode.y + toNode.y} / 2) + 10
 fill={isTrainingExamined ? (isMatchBranch ? '#10b981' : '#f43f5f') : 'none'}
 fontSize="12"
 fontWeight="bold"
 textAnchor="middle"
 >
 {char}
 </text>

 { /* ВИЗУАЛЬНЫЕ МЕТКИ ПРОВЕРКИ IF В ПУНКТУ */ }
 {isTrainingExamined && (
 <g transform={`translate(${(fromNode.x + toNode.x) / 2}, ${((fromNode.y + toNode.y) / 2) + 10})`} >
 <rect x="-15" y="-12" width="30" height="24" fill={isTrainingExamined ? (isMatchBranch ? '#10b981' : '#f43f5f') : 'none'} stroke={stroke} strokeWidth="2" />
 <text x="0" y="5" fontSize="14" font-weight="bold">
 {isMatchBranch ? 'ИСТИНА' : 'ЛОЖЬ'}
 </text>
 </g>
)}
 </g>
 </g>
);
});
}}}

{ /* Рендер fail-ссылок (пунктирные водопадные линии) */ }
Object.values(WATERFALL_NODES).map((node) => {

```



```
}}}
```

```

{/* Рендер вершин (бассейнов водопада) */}
{Object.values(WATERFALL_NODES).map((node) => {
 const isCurrent = activeFishPosNode.id === node.id;
 const isTargetChild = activeMode === 'target' && node.id === targetId;
 const hasWords = node.words.length > 0;

```

```

return (
 <g key={`node-${node.id}`} className={isCurrent ? 'current' : ''}>
 {/* Внешнее свечение для активной вершины */}
 {isCurrent && (
 <circle cx={node.x} cy={node.y} r={node.radius * 1.5} fill="none" stroke="black" stroke-width={2} />
)}
 {isTargetChild && (
 <circle cx={node.x} cy={node.y} r={node.radius * 1.5} fill="none" stroke="red" stroke-width={2} />
)}

```

```

 {/* Основной круг вершины */}
 <circle
 cx={node.x}
 cy={node.y}
 r={isCurrent || isTargetChild ? '24px' : node.radius}
 fill={isCurrent ? '#2563eb' : isTargetChild ? 'red' : node.fill}
 stroke={isCurrent ? 'black' : isTargetChild ? 'red' : node.stroke}
 stroke-width={isCurrent || isTargetChild ? 2 : node.strokeWidth}
 className="transition-all duration-300"
 />

```

```

 {/* Текст внутри вершины */}
 <text
 x={node.x}
 y={node.y + 5}
 fill={isCurrent || isTargetChild ? 'white' : node.textColor}
 fontSize={isCurrent || isTargetChild ? 14 : node.fontSize}
 fontWeight="bold"
 textAnchor="middle"
 />

```

```
>
```

```

 />
 { /* Сама рыба */ }
 <text
 x={activeFishPosNode.x + 15}
 y={activeFishPosNode.y - 19}
 fontSize="18"
 textAnchor="middle"
 style={{ transition: 'x 0.8s cubic-bezier(0.25, 0.8, 0.25, 0.75)' }}
 className="animate-float select-none"
 >
 {activeMode === 'training' ? ' ♂ ' : ''}
 </text>
 </g>

</svg>
</div>
</div>

{ /* Список найденных совпадений (только в режиме поиска) */ }
{activeMode === 'search' && (
 <div className="mt-6 bg-slate-900/50 p-5 rounded" style={{ width: '100%' }}>
 <h5 className="text-white font-bold mb-3 text-sm">
 Улов лосося (Найденные слова)
 </h5>
 {foundMatches.length === 0 ? (
 <p className="text-sm text-slate-400 italic">
 Пока ничего не поймали. Запустите шаги симуляции.
 </p>
) : (
 <div className="flex flex-wrap gap-2">
 {foundMatches.map((match, idx) => (
 <div
 key={idx}
 className="bg-emerald-950 border border-emerald-500 text-white text-sm font-bold shadow animate-[scaleUp_0.3s_ease-out]"
 >
 <CheckCircle2 className="w-4 h-4 text-emerald-400" />
 {match.word}
 </div>
))}
 </div>
)}
 </div>
)}

```

```

 [1, 2, 3, 4],
 [5, 6, 7, 8],
 [9, 1, 2, 3],
 [4, 5, 6, 7],
];

const cellBase =
 "flex items-center justify-center rounded-md font-monospace text-sm"
 "w-[clamp(30px,9vw,46px)] h-[clamp(30px,9vw,46px)] text-center";

export const PrefixSumViz: React.FC = () => {
 const [mode, setMode] = useState<"1d" | "2d">("1d");
 return (
 <div className="w-full bg-slate-950 rounded-2xl border border-slate-800">
 <div className="flex gap-2 mb-4 overflow-x-auto no-scrollbar">
 {(
 [
 ["1d", "1. Одномерные суммы"],
 ["2d", "2. Двумерные суммы"],
] as const
).map(([key, label]) => {
 <button
 key={key}
 onClick={() => setMode(key)}
 className={`px-3 py-2 rounded-lg text-xs sm:text-sm
 ${mode === key ? "bg-indigo-600 text-white" : "text-slate-400"}
 `}
 >
 {label}
 </button>
))}
 </div>
 {mode === "1d" ? <Prefix1D /> : <Prefix2D />}
 </div>
);
};

/* ----- 1D -----

```

```

 return (
 <div
 key={i}
 onMouseEnter={() => setHover({ kind:
 onMouseLeave={() => setHover(null)}
 onClick={() => setRange({r} => (i < r
 className={`${cellBase} cursor-pointer
 inCover
 ? "bg-indigo-500 text-white ring-1
 : inRange
 ? "bg-indigo-900/70 text-indigo
 : "bg-slate-800 text-slate-300
 }`}
 title={`a[${i}] = ${v}`}
 >
 {v}
 </div>
);
 }}}
 </div>

```

```

 {/* массив P */}
 <div className="flex gap-1.5 items-center">
 <div className="w-[46px] shrink-0 text-right
 {pref.map((v, i) => {
 const active = hover?.kind === "pref" &&
 const isL = i === range.l;
 const isR = i === range.r + 1;
 return (
 <div
 key={i}
 onMouseEnter={() => setHover({ kind:
 onMouseLeave={() => setHover(null)}
 className={`${cellBase} cursor-help $
 active
 ? "bg-emerald-500 text-white ring
 : isR
 ? "bg-emerald-700 text-white"

```

```

 входит во все префиксы, начиная с{" "}

 подвинуть границу запроса.
 </p>
) : (
 <p className="text-slate-500">
 Наведите курсор на любое число: у <b className="text-indigo-400">у
 у <b className="text-indigo-400">a – ку,
 </p>
)}
 </div>

 { /* Запрос */ }
 <div className="bg-slate-900 border border-indigo-400 border-radius-10px p-10">
 <div className="text-xs text-slate-400 mb-2">
 Запрос суммы на отрезке (кликайте по массиву)
 </div>
 <p className="font-mono text-sm sm:text-base text-slate-300">
 sum(a[{range.l}..{range.r}]) = P[{range.r + 1}..{range.l}]
 {pref[range.r + 1]}
 {pref[range.l]}
 {
 </p>
 <p className="text-[11px] text-slate-500 mt-1">
 Один вычитание вместо цикла: запрос за O(1) по префикс-сумме
 </p>
 </div>
</div>
);
};

/* ----- 2D -----
const Prefix2D: React.FC = () => {
 const n = A2.length;
 const m = A2[0].length;

 const P = useMemo(() => {
 const p = Array.from({ length: n + 1 }, () => Array.from(

```

```

 key={j}
 onClick={() =>
 setRect((r) =>
 i < r.r1 || j < r.c1 ? { r1:
)
 }
 className={` ${cellBase} cursor-po
 inRect ? "bg-indigo-500 text-wh
 }`}
 >
 {v}
 </div>
);
 }}}
</div>
)))
</div>
</div>

```

```

 { /* Матрица префиксных сумм */ }
 <div className="overflow-x-auto">
 <div className="text-xs text-slate-400 mb-2">
 <div className="inline-block">
 {P.map((row, i) => {
 <div key={i} className="flex gap-1.5 mb-1">
 {row.map((v, j) => {
 const corner = cornerOf(i, j);
 const isHover = hover?.i === i && hov
 const cornerColor =
 corner === "A"
 ? "bg-emerald-600 text-white"
 : corner === "D"
 ? "bg-sky-600 text-white"
 : corner
 ? "bg-rose-600 text-white"
 : "bg-slate-900 border border
 return {
 <div

```

```

 сумма = A<
 C + <s
 {
 </p>
 <p className="text-[11px] text-slate-500 mt-1">
 Вычли два «хвоста» и вернули дважды вычтенный
 </p>
 </div>
</div>
);
};

```

ФАЙЛ 73/85: src/components/visualizers/SparseTableViz.t  
КАТЕГОРИЯ: Визуализаторы (вложенные) | СТРОК: 709 | РАЗМ:

```

import React, { useState } from "react";
import { motion, AnimatePresence } from "motion/react";
import {
 Play,
 RotateCcw,
 ChevronRight,
 ChevronLeft,
 Lock,
 ChevronDown,
} from "lucide-react";

// Data for 1D Array
const arr1D = [2, 3, 5, 62, 3, 21, 1, 4];
const N = arr1D.length;
const LOG = 4;

const st1D: number[][] = Array.from({ length: N }, () => []);
for (let i = 0; i < N; i++) st1D[i][0] = arr1D[i];
for (let j = 1; j < LOG; j++) {
 for (let i = 0; i + (1 << j) <= N; i++) {

```





```

 </h3>
 <p className="text-sm text-slate-400">
 Формула: $ST[i][j] = \min(ST[i][j-1], ST[i + 1][j])$
 </p>
 </div>
 <div className="flex gap-2">
 <button
 onClick={() => setStep(Math.max(0, step - 1))}
 className="p-2 bg-slate-800 hover:bg-slate-700"
 disabled={step === 0}
 >
 <ChevronLeft size={20} />
 </button>
 <button
 onClick={() => setStep(Math.min(maxStep, step + 1))}
 className="p-2 bg-indigo-600 hover:bg-indigo-500"
 disabled={step === maxStep}
 >
 <ChevronRight size={20} />
 </button>
 <button
 onClick={() => setStep(0)}
 className="p-2 bg-slate-800 hover:bg-slate-700"
 >
 <RotateCcw size={20} />
 </button>
 </div>
</div>

<div className="bg-slate-900 rounded-xl border border-slate-800">
 <div className="text-[11px] uppercase tracking-wider">
 <div className="flex gap-1 sm:gap-1.5 overflow-x-auto">
 {arr1D.map((v, idx) => {
 const inCover = !!covered && idx >= covered
 return (
 <div key={idx} className="flex flex-col items-center justify-center">
 <div
 className={`flex items-center justify-center ${
 inCover ? 'bg-slate-800' : 'bg-slate-900'
 }`}
 >
 <div
 className="text-[10px] font-mono"
 style={inCover ? { color: 'white' } : { color: 'black' }}
 >
 {v}
 </div>
 </div>
 </div>
)
 })}
 </div>
 </div>
</div>

```

```

 <div className="text-slate-500 font-mono text-
 i \ j
 </div>
 <div className="text-center font-bold text-
 rap">
 2^0 (L=1)
 </div>
 <div className="text-center font-bold text-
 rap">
 2^1 (L=2)
 </div>
 <div className="text-center font-bold text-
 rap">
 2^2 (L=4)
 </div>
 <div className="text-center font-bold text-
 rap">
 2^3 (L=8)
 </div>

 {/* Matrix */}
 {Array.from({ length: N }).map((_, i) => {
 <React.Fragment key={i}>
 <div className="text-slate-500 font-mono
 {i}
 </div>
 {Array.from({ length: LOG }).map((_, j) => {
 const isValid = i + (1 << j) <= N;
 const isVisible =
 isValid &&
 (j === 0 ||
 computeSteps.findIndex((s) => s.i
 step));

 // Active computing state
 let isActive = false;
 let isSrc1 = false;
 let isSrc2 = false;

```

```

 ? "bg-sky-500 text-white"
 : isActive
 ? "bg-indigo-500 text-white"
 : isSrc1
 ? "bg-emerald-500 text-white"
 : isSrc2
 ? "bg-amber-500 text-white"
 : isVisible
 ? "bg-slate-800 text-white"
 : "bg-transparent text-white"
 }` }
 >
 {st1D[i][j]}
 </motion.div>
)}

{/* Arrows visualization */}
{({isSrc1 || isSrc2) && (
 <motion.svg
 initial={{ opacity: 0 }}
 animate={{ opacity: 1 }}
 className="absolute pointer-events-none"
 style={{
 width: 100,
 height: isSrc2 ? 40 + (1 <<
 right: -50,
 top: 24,
 overflow: "visible",
 }}
 >
 <path
 d={`M 0 0 C 30 0, 30 ${isSrc1 ? "0 0" : "0 100%"}
 fill="none"
 stroke={isSrc1 ? "#10b981" : "#f96"}
 strokeWidth="3"
 />
 </motion.svg>
)}

```

```

);
 })()
) : (
 <p className="text-slate-500 italic">
 Нажмите далее, чтобы увидеть, как блоки сво
 </p>
)}
 </div>
</div>
);
};

```

```

const Viz2DBuild: React.FC = () => {
 const [k, setK] = useState(1);
 const [hover, setHover] = useState<{ r: number; c: number}>({ r: 0, c: 0 });
 const [locked, setLocked] = useState<{ r: number; c: number}>({ r: 0, c: 0 });

 const L = 1 << k;
 const activeCell = locked || hover;

 return (
 <div className="flex flex-col xl:flex-row gap-6">
 <div className="flex-1 min-w-0 bg-slate-900 p-3 scrollable">
 <h3 className="text-xl font-bold text-indigo-400">Визуализация
 <p className="text-sm text-slate-400 mb-6 text-pretty">
 Для наглядности показываем только <b className="text-rose-300">клетку

 активную!
 </p>

 <div className="flex bg-slate-950 p-1 rounded-lg">
 {[0, 1, 2, 3].map(step => (
 <button
 key={step}
 onClick={() => { setK(step); setHover(null); setLocked(null); }}
 className={`px-4 py-2 rounded-md text-sm text-slate-400 hover:text-white hover:bg-slate-900`}
 />
))}
 </div>
 </div>
 </div>
);
};

```



```

="text-amber-300">1});

 // Ква
 ST</
 <span className="text-rose
e="text-amber-300">1], <sp
 <span className="text-blue
ext-amber-300">1][k-<span className=
pan>

 <span className="text-emerg
="text-amber-300">1][k-<span classNa
span>

 <span className="text-amber
white">c + pL][k-<span className="te
ame="text-slate-500">// Правый Низ<b
 {''}}));
</div>

```

```

{activeCell ? {
 <div className="bg-slate-950 p-5 rounded
.2)] mt-2 transition-all">
 <p className="text-slate-300 mb-3 t
 Пример для ячейки:
 {locked && <span className="text
ded border border-rose-500/20"><Lock size={
 </p>
 <p className="text-indigo-300 borde
 <span className="font-bold text-w
}}[k]][k]][k]

 </p>

 <div className="space-y-2 pt-3 pl-2
 {(() => {
 const prevL = 1 << (k - 1);
 return (
 <
 <div className="flex item
 <span className="flex

```

```

 </div>
)}
 </div>
)}
</div>
</div>
</div>
);
};

const Viz2DQuery: React.FC = () => {
 const [r1, setR1] = useState(1);
 const [c1, setC1] = useState(1);
 const [r2, setR2] = useState(5);
 const [c2, setC2] = useState(6);

 const H = r2 - r1 + 1;
 const W = c2 - c1 + 1;
 const kx = Math.floor(Math.log2(H));
 const ky = Math.floor(Math.log2(W));
 const Lx = 1 << kx;
 const Ly = 1 << ky;

 const matRows = 8 - Lx + 1;
 const matCols = 8 - Ly + 1;

 return (
 <div className="flex flex-col xl:flex-row gap-6"
 <div className="flex-1 min-w-0 bg-slate-900"
 <h3 className="text-xl font-bold text-ro
 <p className="text-slate-400 text-[13px]
 еpx, r2, c2 - правый низ).</p>

 <div className="flex flex-col items-cent
 <div className="grid grid-cols-[repea
 ative shadow-inner w-max max-w-full">
 {Array.from({length: 64}).map((_,
 const r = Math.floor(idx / 8);

```

```

</div>

 <div className="bg-slate-950 p-4 rounded-
er-slate-800 shadow-[inset_0_2px_10px_rgba(
 <label className="flex items-cen
 <span className="text-slate-
 <div className="flex gap-2">
 <input type="number" min={
14 bg-slate-900 border border-slate-700 py-
 <input type="number" min={
14 bg-slate-900 border border-slate-700 py-
 </div>
 </label>
 <label className="flex items-cen
 <span className="text-slate-
 <div className="flex gap-2">
 <input type="number" min={
14 bg-slate-900 border border-slate-700 py-
 <input type="number" min={
14 bg-slate-900 border border-slate-700 py-
 </div>
 </label>
 </div>
</div>
</div>

<div className="flex-1 bg-slate-900 p-6 rounded-
 <h3 className="text-xl font-bold text-em
 <p className="text-sm font-sans text-sla
 Мы знаем, что максимальная степень дв
rounded text-white">{Lx}. Аналогично
te">{Ly}.
 </p>
 <p className="text-sm font-sans text-sla
 Чтобы покрыть весь прямоугольник, мы
Name="font-mono bg-[#0a0fle] px-1 rounded b
роса. (Цветные прямоугольники слева).
 </p>

```



```

<div className="w-full flex justify-center"
 <div className="flex flex-col items-center"
 <h4 className="text-slate-400 font-bold"
 Откуда берутся эти 4 числа: матрица
border-slate-700 shadow-sm">ST[][][kx]][ky]
 </h4>
 <div className="grid gap-[2px] p-2.5"
Columns: `repeat(${matCols}, 1fr)`>}}>
 {Array.from({length: matRows * matCols}, (v, idx) => {
 const r = Math.floor(idx / matCols);
 const c = idx % matCols;
 let isTL = r === r1 && c === c1;
 let isTR = r === r1 && c === c2;
 let isBL = r === r2 - Lx + 1 && c === c1;
 let isBR = r === r2 - Lx + 1 && c === c2;

 let color = "bg-slate-800 text-white";
 let txt = ST2D[r][c][kx][ky];

 let badge = null;
 if (isTL) { color = "bg-rose-500 text-white";
0 border-2"; badge = "TL"; }
 else if (isTR) { color = "bg-lime-500 text-white";
ue-300 border-2"; badge = "TR"; }
 else if (isBL) { color = "bg-emerald-500 text-white";
-emerald-300 border-2"; badge = "BL"; }
 else if (isBR) { color = "bg-amber-500 text-white";
mber-300 border-2"; badge = "BR"; }

 return (
 <div key={idx} className={color}
tion-all duration-300 relative shrink-0 ${c === c1 ? 'w-1/2' : 'w-1/4'} h-1/4"
 {txt}
 {badge && <div className="border-slate-700 text-white px-1 font-bold rounded"
 </div>
)
)
 })
 </div>
 </div>
)
}
}
}

```

```

 </main>
 </div>

 <script>
 function setMode(mode) {
 document.getElementById('btn-mode-normal').className =
 ? 'px-3 py-1 rounded text-sm font-bold' : 'px-3 py-1 rounded text-sm font-bold';

 document.getElementById('btn-mode-wtf').className =
 ? 'px-3 py-1 rounded text-sm font-bold' : 'px-3 py-1 rounded text-sm font-bold';

 const aside = document.querySelector('aside');
 const headerMain = document.querySelector('header main');
 const questionsContainer = document.getElementById('questions-container');
 const wtfContainer = document.getElementById('wtf-container');

 if (mode === 'wtf') {
 document.body.classList.add('wtf-mode');
 if (aside) aside.style.display = 'none';
 if (headerMain) headerMain.style.display = 'none';
 if (questionsContainer) questionsContainer.style.display = 'none';
 if (wtfContainer) wtfContainer.classList.remove('hidden');

 // Remove the URL hash to avoid scrolling to the top of the page
 history.pushState("", document.title, window.location.pathname + window.location.search);
 window.scrollTo(0, 0);
 } else {
 document.body.classList.remove('wtf-mode');
 if (aside) aside.style.display = '';
 if (headerMain) headerMain.style.display = '';
 if (questionsContainer) questionsContainer.style.display = '';
 if (wtfContainer) wtfContainer.classList.add('hidden');
 }
 }

 function setTheme(theme) {

```

```

 mobileDrawer.classList.remove("-transl
 if (drawerOverlay) {
 drawerOverlay.classList.remove(
 // Timeout allows display:block
 setTimeout(() => drawerOverlay.
 }
 document.body.style.overflow = "hid
} else {
 mobileDrawer.classList.add("-transl
 if (drawerOverlay) {
 drawerOverlay.classList.add("op
 setTimeout(() => drawerOverlay.
 }
 document.body.style.overflow = "";
}
}

if (mobileMenuBtn && mobileDrawer) {
 mobileMenuBtn.addEventListener("click",
}
if (closeDrawerBtn && mobileDrawer) {
 closeDrawerBtn.addEventListener("click"
}
if (drawerOverlay) {
 drawerOverlay.addEventListener("click",
}

// Close on link click
document.querySelectorAll("#mobile-drawer a
 link.addEventListener("click", () => {
 if (mobileDrawer && !mobileDrawer.c
 toggleDrawer();
 }
 });
});
});

const observerOptions = {
 root: null,

```

```

<meta name="viewport" content="width=device-width,
<title>Пособие по алгебре: От пиццы к предикатам</t
<script src="https://cdn.tailwindcss.com"></script>
<script src="https://polyfill.io/v3/polyfill.min.js
<script id="MathJax-script" async src="https://cdn.
<script>
 window.MathJax = {
 tex: {
 inlineMath: [['$', '$'], ['\\(', '\\)']],
 displayMath: [['$$', '$$'], ['\\[', '\\
 }
 };
</script>
<style>
 @import url('https://fonts.googleapis.com/css2?
 @reference "./src/index.css";
 .math-font { font-family: 'Fira Code', monospace;
 .uno-card {
 width: 70px; height: 105px; border-radius: 10px;
 box-shadow: 0 4px 8px rgba(0,0,0,0.5); position: relative;
 align-items: center; justify-content: center; text-align: center;
 color: white; overflow: hidden; flex-shrink: 0;
 text-shadow: 2px 2px 0 #000, -1px -1px 0 #000, 1px 1px 0 #000;
 font-family: 'Arial Black', Impact, sans-serif;
 }
 .uno-card::before, .uno-card::after {
 content: attr(data-val); position: absolute; top: 50%; left: 50%;
 text-shadow: 1px 1px 0 #000, -1px -1px 0 #000, 1px 1px 0 #000;
 }
 .uno-card::before { top: 2px; left: 4px; }
 .uno-card::after { bottom: 2px; right: 4px; transform: rotate(180deg); }
 .uno-card.mini { width: 45px !important; height: 60px !important; }
 .uno-card.mini::before, .uno-card.mini::after {
 font-size: 0.8em;
 }

 /* Global formula scroll settings */
 .formula-scroll {
 overflow-x: auto;
 overflow-y: hidden;

```

```

 /* Allow horizontal scroll for equations in
 .formula-scroll { flex-wrap: nowrap !important;
 px; }
 }
 .uno-inner {
 position: absolute; width: 110%; height: 75%;
 border-radius: 50%; transform: rotate(-30deg);
 box-shadow: inset 0 0 5px rgba(0,0,0,0.3);
 }
 .uno-val { position: relative; z-index: 2; }
 .card-red { background-color: #e52521; } .card-
 .card-blue { background-color: #004dcf; } .card-
 .card-green { background-color: #339e35; } .card-
 .card-yellow { background-color: #ffc400; } .ca
 .card-black { background-color: #222; } .card-b
 .card-black::before, .card-black::after { text-
 .info-block { @apply bg-slate-800/80 border-l-4
 .info-title { @apply font-bold mb-2 flex items-

/* Визуализация (Аналогии) */
.analogy-block { @apply bg-slate-900 border-2 b
 ems-center gap-6; }
.analogy-badge { @apply absolute -top-3 left-4 l
 king-widest border border-slate-500 shadow-md
.img-placeholder { @apply flex flex-col items-c
 r shrink-0 shadow-lg w-full md:w-auto; }
.img-caption { @apply font-mono text-sm mt-4 fo

/* THEME OVERRIDES */
body.theme-light {
 background-color: #f8fafc !important;
 color: #0f172a !important;
}
body.theme-light .bg-slate-900, body.theme-ligh
body.theme-light .bg-slate-800 { background-col
body.theme-light .bg-slate-700, body.theme-ligh
 important; }
body.theme-light .text-slate-200, body.theme-li

```

```

 transform-origin: center;
 transform-box: fill-box;
 animation: creepy-blink 5.2s infinite;
 animation-delay: 1.5s;
 }
 .creepy-emoji {
 display: inline-block;
 transform-origin: center;
 animation: creepy-blink 3.8s infinite;
 }
</style>
</head>
<body class="bg-slate-900 text-slate-200 font-sans lead">

 <div class="sticky top-0 z-50 w-full bg-slate-950/90
 center items-center gap-4 sm:gap-8 transition-colors">
 <div class="flex items-center gap-3">
 ТЕМА
 <div class="bg-slate-800 p-1 rounded-lg flex items-center gap-2
 transition-colors">Список тем</div>
 <button onclick="setMode('wtf')" id="btn-wtf" class="bg-slate-800
 transition-colors">WTF</button>
 </div>
 </div>
 <div class="flex items-center gap-3">
 ТЕМА
 <div class="bg-slate-800 p-1 rounded-lg flex items-center gap-2
 transition-colors">Темная</div>
 <button onclick="setTheme('dark')" id="btn-dark" class="bg-slate-800
 transition-colors">Темная</button>
 <button onclick="setTheme('light')" id="btn-light" class="bg-slate-800
 transition-colors">Светлая</button>
 <button onclick="setTheme('notebook')" id="btn-notebook" class="bg-slate-800
 transition-colors">Заметки</button>
 </div>
</div>
</div>

```

```

 <g id="lego-rem">
 <rect x="0" y="0" width="40" height="10"
 <rect x="6" y="-5" width="8" height="6"
 <rect x="26" y="-5" width="8" height="6"
 <line x1="2" y1="2" x2="38" y2="2" stro
 </g>
 <!-- МЯСОРУБКА -->
 <g id="meat-grinder">
 <path d="М 10 40 L 15 20 Q 25 15 35 20
 <rect x="5" y="40" width="40" height="2
 <!-- Ручка -->
 <path d="М 45 50 Q 60 50 60 30 L 75 30"
 <circle cx="75" cy="30" r="4" fill="#33
 <!-- Выход -->
 <path d="М 5 45 L -5 45 L -5 55 L 5 55
 <circle cx="25" cy="50" r="15" fill="#f
 </g>
 <g id="lego-green">
 <rect x="0" y="10" width="20" height="1
 <rect x="3" y="5" width="14" height="5"
 </g>
 <g id="lego-white">
 <rect x="0" y="0" width="40" height="20
 <rect x="6" y="-5" width="8" height="6"
 <rect x="26" y="-5" width="8" height="6"
 <line x1="0" y1="2" x2="40" y2="2" stro
 <text x="20" y="14" fill="#475569" font
 </g>
 </defs>
</svg>

<!-- Мобильная кнопка меню (Гамбургер) -->
<button id="mobile-menu-btn" class="lg:hidden fixed
 0/50 z-50 hover:bg-blue-500 transition-transform
 <svg xmlns="http://www.w3.org/2000/svg" class="
 <path stroke-linecap="round" stroke-linejoin
 </svg>
</button>

```

```
</details>

<details class="group">
 <summary class="list-none cursor-pointer-indigo-500 pl-3 font-bold text-indigo-300">
 2. Комплексные числа

 </summary>
 <div class="pl-6 space-y-2 text-gray-500">

 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer-teal-500 pl-3 font-bold text-teal-300">
 3. Тригонометрическая форма

 </summary>
 <div class="pl-6 space-y-2 text-gray-500">

 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer-amber-500 pl-3 font-bold text-amber-300">
 4. Степени и корни

 </summary>
```



```
 <a href="#q6-6" class="block ho
</div>
</details>

<details class="group">
 <summary class="list-none curso
 <a href="#question-7" class=
er-pink-500 pl-3 font-bold text-pink-300">
 7. Взаимно простые числ

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#q7-1" class="bloc
 <a href="#q7-2" class="bloc
 <a href="#q7-3" class="bloc
 <a href="#q7-4" class="bloc
 <a href="#q7-5" class="bloc
 </div>
</details>

<details class="group">
 <summary class="list-none curso
 <a href="#question-8" class=
er-violet-500 pl-3 font-bold text-violet-300">
 8. Сравнимость и Поля В

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#q8-1" class="bloc
 <a href="#q8-2" class="bloc
 <a href="#q8-3" class="bloc
 <a href="#q8-4" class="bloc
 <a href="#q8-5" class="bloc
 <a href="#q8-6" class="bloc
 <a href="#q8-7" class="bloc
 </div>
</details>
```

```
 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#q11-1" class="blo
 <a href="#q11-2" class="blo
нственности НОД
 <a href="#q11-3" class="blo
ПИРАЛЬ)
 <a href="#q11-4" class="blo
вращается!)
 </div>
 </details>
```

```
 <details class="group">
 <summary class="list-none curso
 <a href="#question-12" clas
der-blue-500 pl-3 font-bold text-blue-400">
 12. Взаимно простые мно

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#q12-1" class="blo
 <a href="#q12-2" class="blo
 <a href="#q12-3" class="blo
еление)
 </div>
 </details>
```

```
 <details class="group">
 <summary class="list-none curso
 <a href="#question-13" clas
der-emerald-500 pl-3 font-bold text-emerald
 13. Корни многочленов.

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#q13-1" class="blo
 <a href="#q13-2" class="blo
 <a href="#q13-3" class="blo
```

```
der-red-500 pl-3 font-bold text-red-400">
 16. Неприводимые многоч.
```

```

```

```
 </summary>
```

```
 <div class="pl-6 mt-2 space-y-2">
```

```

```

```

```

```

```

```
 </div>
```

```
</details>
```

```
 <div class="mt-4 mb-2 text-xs font-l">
```

```

```

```
-500 pl-3 text-slate-400">
```

```

```

```

```

```

```

```
-500 pl-3 text-slate-400">
```

```

```

```

```

```

```

```
ald-500 pl-3 text-slate-400">
```

```

```

```

```

```

```

```
-500 pl-3 text-slate-400">
```

```

```

```

```

```

```

```
go-500 pl-3 text-slate-400">
```

```

```

```

```

```

```

```
sia-500 pl-3 text-slate-400">
```

```

```

```

```

```
 <a href="#proof-algorithms" class="b
```

```

 visibleId = entry.target.id;

 // Highlight active link
 document.querySelectorAll('a').forEach(a => {
 if (a.href.endsWith(entry.url)) {
 a.style.fontWeight = 'bold';
 }
 });

 // Open the parent details if needed
 const correspondingLink = entry.url.startsWith(details.url) ? details : null;
 if (correspondingLink) {
 const details = document.querySelector(`details[id="${correspondingLink.id}"]`);
 if (details && !details.open) {
 details.open = true;
 document.querySelector(`a[href="#${correspondingLink.id}"]`).click();
 }
 }
 });
}

// Initialize the observer
const observer = new IntersectionObserver(
 (entries) => {
 entries.forEach(entry => {
 if (entry.isIntersecting) {
 // Trigger the fallback modal
 fallbackModal(entry.target, entry.intersectionRatio);
 }
 });
 }, {
 root: null,
 rootMargin: '-10% 0px -70% 0px',
 threshold: [0, 0.5]
 });

// Observe the sections
sections.forEach(section => {
 observer.observe(section);
});
});

function fallbackModal(contentRaw, intersectionRatio) {
 const modal = document.createElement('div');
 modal.innerHTML = contentRaw;
 modal.style.position = 'relative';
 modal.style.overflow = 'hidden';
 modal.style.backgroundColor = 'white';
 modal.style.padding = '10px';
 modal.style.zIndex = 1000;
 modal.style.opacity = 0;
 modal.style.transform = 'scale(0.8)';
 modal.style.transition = 'opacity 0.3s, transform 0.3s';
 document.body.appendChild(modal);
 modal.style.opacity = 1;
 modal.style.transform = 'scale(1)';
 modal.addEventListener('click', () => {
 modal.style.opacity = 0;
 modal.style.transform = 'scale(0.8)';
 modal.addEventListener('transitionend', () => {
 modal.remove();
 });
 });
}

```

```
textarea.style.padding = '10px'
textarea.style.fontFamily = 'monospace'
```

```
const btnContainer = document.createElement('div')
btnContainer.style.display = 'flex'
btnContainer.style.gap = '10px'
btnContainer.style.marginTop = '10px'
```

```
const copyBtn = document.createElement('button')
copyBtn.innerText = 'Копировать'
copyBtn.style.padding = '10px'
copyBtn.style.backgroundColor = 'black'
copyBtn.style.color = '#fff'
copyBtn.style.border = 'none'
copyBtn.style.borderRadius = '5px'
copyBtn.style.cursor = 'pointer'
copyBtn.onclick = () => {
 if (navigator.clipboard) {
 navigator.clipboard.writeText(textarea.value)
 copyBtn.innerText = 'Скопировано'
 } else {
 textarea.select()
 document.execCommand('copy')
 copyBtn.innerText = 'Скопировано'
 }
 setTimeout(() => copyBtn.innerText = 'Копировать', 2000)
};
```

```
const closeBtn = document.createElement('button')
closeBtn.innerText = 'Закрыть'
closeBtn.style.padding = '10px'
closeBtn.style.backgroundColor = 'black'
closeBtn.style.color = '#fff'
closeBtn.style.border = 'none'
closeBtn.style.borderRadius = '5px'
closeBtn.style.cursor = 'pointer'
closeBtn.onclick = () => modal.close()
```

```

 const blob = new Blob([text], {
 const url = URL.createObjectURL(blob);
 const link = document.createElement('a');
 link.href = url;
 link.download = 'vyisshaya_algebra.pdf';
 document.body.appendChild(link);
 link.click();
 document.body.removeChild(link);
 }

 function setTheme(theme) {

 const body = document.body;
 body.classList.remove('bg-slate-900');

 document.getElementById('theme-dark').checked = theme === 'dark';
 document.getElementById('theme-light').checked = theme === 'light';
 document.getElementById('theme-auto').checked = theme === 'auto';

 let oldStyle = document.getElementById('theme-style');
 if (oldStyle) oldStyle.remove();
 const style = document.createElement('style');
 style.id = 'theme-style';

 if (theme === 'dark') {
 body.classList.add('bg-slate-900');
 document.getElementById('theme-dark').checked = true;
 } else if (theme === 'light') {
 body.classList.add('bg-white');
 document.getElementById('theme-light').checked = true;
 style.innerHTML = `
 body.bg-white section h1 { color: #000; }
 body.bg-white main { color: #000; }
 body.bg-white main p { color: #000; }
 /* Universal text color */
 body.bg-white .text-slate-900 { color: #000; }
 `;
 } else {
 body.classList.add('bg-slate-900');
 document.getElementById('theme-auto').checked = true;
 }
 }

```

```

 body.bg-white .text-rose
 body.bg-white .text-emerald
 body.bg-white .text-cyan
 body.bg-white .text-amber
 body.bg-white .text-fuchsia
 body.bg-white .text-lime
 body.bg-white .text-blue
 body.bg-white .text-green
 body.bg-white .text-indigo
 `;
} else if (theme === 'terminal')
 body.classList.add('bg-black')
 document.getElementById('theme')
 style.innerHTML = `
 body.bg-black .bg-slate-950 {
 background-color: #000000;
 }
 body.bg-black section header {
 color: #4ade80 !important;
 border-color: #22c55e;
 font-family: monospace;
 }
 body.bg-black svg path,
 stroke: #22c55e !important;
 fill: transparent !important;
 `;
}

document.head.appendChild(style)
}
</script>

```

```
 </div>

 </div>
</aside>

<!-- Основной контент -->
<main class="flex-1 bg-slate-800 p-4 sm:p-8 md:p-12 lg:p-16"
 -base min-w-0">
 <header class="text-center mb-12 border-b border-white pb-4">
 <h1 class="text-4xl md:text-5xl font-bl
4">
 ВЫСШАЯ АЛГЕБРА
 </h1>
 <p class="text-slate-400 italic text-lg">

 <!-- Print Table of Contents -->
 <div class="hidden print:block mt-8 text-center">
 <h2 class="font-bold text-xl mb-4">
 <ul class="text-base space-y-2">
 1. Ал
 2. Ко
 3. Тр
 4. Ст
 5. Гр
 6. Де
 7. Ал
 8. Кл
 9. Ко
 10.
 11.
 12.
 13.
 14.
 15.
 17.
 18.
 19.
```



```

<!-- Row 2: Базовые операции / Евклид
<a href="#question-6" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
 <div class="text-xs text-yellow
 <div class="font-bold text-slate

<a href="#question-10" onclick="setM
-l-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange
 <div class="font-bold text-slate

<a href="#question-3" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans.
 <div class="text-xs text-cyan-5
 <div class="font-bold text-slate

<!-- Row 3: НОД и НОК / Алгоритм Ев
<a href="#question-7" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
 <div class="text-xs text-yellow
 <div class="font-bold text-slate

<a href="#question-12" onclick="setM
-l-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange
 <div class="font-bold text-slate

<!-- Empty 3rd row cell -->
<div class="hidden md:block"></div>

<!-- Row 4: Факторизация / Структур
<a href="#question-8" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
 <div class="text-xs text-yellow
 <div class="font-bold text-slate

<a href="#question-11" onclick="setM

```

```


 <div class="hidden md:block"></div>

 <!-- КЛАСТЕР 3: НЕПРИВОДИМОСТЬ -->
 <div class="col-span-1 md:col-span-1
b-2 border-b border-emerald-500/30 pb-2">Кл

 <a href="#question-17" onclick="setf
-l-4 border-teal-500 hover:bg-slate-700 tra
 <div class="text-xs text-teal-5
 <div class="font-bold text-slate

 <a href="#question-18" onclick="setf
-l-4 border-blue-500 hover:bg-slate-700 tra
 <div class="text-xs text-blue-5
 <div class="font-bold text-slate

 <a href="#question-19" onclick="setf
-l-4 border-emerald-500 hover:bg-slate-700
 <div class="text-xs text-emeral
 <div class="font-bold text-slate

 <a href="#question-20" onclick="setf
-l-4 border-rose-500 hover:bg-slate-700 tra
 <div class="text-xs text-rose-5
 <div class="font-bold text-slate

 <a href="#question-21" onclick="setf
-l-4 border-indigo-500 hover:bg-slate-700 t
 <div class="text-xs text-indigo
 <div class="font-bold text-slate

 <a href="#question-22" onclick="setf
-l-4 border-fuchsia-500 hover:bg-slate-700
 <div class="text-xs text-fuchsi
 <div class="font-bold text-slate


```

```

 <p class="text-sm text-slate-500">
 <ul class="list-disc pl-5 space-y-2">

 k$).

 ово.

 </div>

 <!-- От противного -->
 <div class="bg-slate-800/80 p-6">
 <h3 class="text-xl text-fuchsia-400">
 <p class="text-sm text-slate-500">
 <ul class="list-disc pl-5 space-y-2">

 i>

 т.д.).

 что так и задумано: "Ой, противоречие, теорема не верна".

 </div>

 <!-- Единственность -->
 <div class="bg-slate-800/80 p-6">
 <h3 class="text-xl text-emerald-400">
 <p class="text-sm text-slate-500">
 <ul class="list-disc pl-5 space-y-2">

 , r_2$).

 счет ограничений (размер остатка) докажи, что не существует.

 </div>

 <!-- Индукция -->

```

```
<div class="bg-slate-800 p-4">
 <h4 class="text-red-400">
 <p class="text-sm text-slate-400">
 <p class="text-xs font-slate-400">
 c. Пока получается 0 – корень жив. Как то.
 корня минус один.</p>
 </p>
 </h4>
 </div>
```

```
<div class="bg-slate-800 p-4">
 <h4 class="text-red-400">
 <p class="text-sm text-slate-400">
 <p class="text-xs font-slate-400">
 ициентов (они делятся на p), кроме старше
 зложить.</p>
 </p>
 </h4>
 </div>
```

```
<div class="bg-slate-800 p-4">
 <h4 class="text-red-400">
 <p class="text-sm text-slate-400">
 <p class="text-xs font-slate-400">
 . Чаще всего после этого Фейс-контроль p .
 </p>
 </h4>
 </div>
```

```
<h2 class="text-2xl font-bold bg-slate-800 p-4">
 (Правила экстрасенса)</h2>
 <p class="text-sm text-slate-400">
 тношения), а ты их не помнишь. Юзай базовые
 нейность) для делимости.</p>
```

```
<ul class="space-y-4 pb-10">
 <li class="bg-slate-800/50 p-4">

 <div>
 <h4 class="text-cyan-400">
 <p class="text-sm text-slate-400">
```

```
<div id="questions-container">
```

```
<!-- ===== ВОПРОС 1 =====
```

## РАЗДЕЛ: УТИЛИТЫ

ФАЙЛ 76/85: src/utils/cn.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 7 | РАЗМЕР: 169 В

```
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";
```

```
export function cn(...inputs: ClassValue[]) {
 return twMerge(clsx(inputs));
}
```

ФАЙЛ 77/85: src/utils/exportHtml.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 228 | РАЗМЕР: 9.8 KB

```
import type { Chapter } from "../types";
import { GLOSSARY_BY_ID } from "../data/glossary";
import { annotateTerms } from "../hooks/useTermHints";
```

```
/**
```

```
 * Выгрузка главы (или всего пособия) в самостоятельный
 *
```

```
 * Файл получается автономным: стили защиты внутрь, интерактив
 * его можно открыть с флешки, отправить в мессенджер и т.д.
 * Живые React-визуализации в статику не переносятся — вместо
 * подставляется заметка со ссылкой на онлайн-версию, а
```

```

.export-term h3 { color: #c7d2fe; font-size: .95rem; font-weight: bold; }
.export-term p { margin: 0 0 .3rem; font-size: .85rem; }
.export-term .formal { color: #94a3b8; font-size: .8rem; font-weight: bold; }
.export-term .cx { color: #6ee7b7; font-size: .78rem; font-weight: bold; }
.export-foot { margin-top: 3rem; border-top: 1px solid #ccc; padding-top: 1rem; }
@media print {
 body { background: #fff; color: #0f172a; }
 .export-note, details { break-inside: avoid; }
 details { display: block; }
 details > div, details > p { display: block !important; }
 pre { white-space: pre-wrap; }
};
```

```
const escapeHtml = (s: string) =>
 s.replace(/&/g, "&").replace(/</g, "<").replace(/>/g, ">");
```

```
/**
```

```
 * Размечает термины и превращает их в якорные ссылки на термины
```

```
 * Возвращает готовый HTML главы и список найденных терминов
```

```
 */
```

```
function prepareBody(html: string): { body: string; terms: string[] } {
 const host = document.createElement("div");
 host.innerHTML = html;
```

```
 try {
```

```
 annotateTerms(host);
```

```
 } catch {
```

```
 /* если браузер не осилил регулярку – выгружаем текст как есть */
```

```
 }
```

```
 const ids: string[] = [];
```

```
 host.querySelectorAll<HTMLElement>(".term-hint").forEach((el) => {
```

```
 const id = el.dataset.termId;
```

```
 if (!id) return;
```

```
 if (!ids.includes(id)) ids.push(id);
```

```
 const link = document.createElement("a");
```

```
 link.className = "term-hint";
```

```

<html lang="ru">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>${escapeHtml(opts.title)}</title>
<style>${opts.css}</style>
<style>${EXTRA_CSS}</style>
</head>
<body>
<div class="export-shell">
 <header class="export-head">
 <div style="font-size:.7rem;text-transform:uppercase">
 <h1 style="color:#fff;font-size:1.6rem;font-weight:bold">
 <div style="font-size:.72rem;color:#475569;margin-top:10px">
 </div>

 <div class="export-note">
 Это автономная копия: стили внутри файла, интернет не нужен.
 Интерактивные тренажёры и анимации в статичный HTML превращены.
 Они живут в онлайн-версии: онлайн
 </div>

 <article class="prose prose-invert max-w-none">
 ${opts.body}
 </article>

 ${opts.gloss}

 <footer class="export-foot">Универсальное пособие по созданию
 </div>
</body>
</html>`;
}

```

```

function triggerDownload(html: string, filename: string) {
 const blob = new Blob([html], { type: "text/html;charset=utf-8" });
 const url = URL.createObjectURL(blob);
 const a = document.createElement("a");
 a.href = url;
 a.download = filename;
 a.click();
 URL.revokeObjectURL(url);
}

```

```

 .join("\n");

 chapters.forEach((c) => {
 const { body, termIds } = prepareBody(c.content);
 termIds.forEach((t) => {
 if (!allTerms.includes(t)) allTerms.push(t);
 });
 bodies.push(`<div id="ch-${escapeHtml(c.id)}" style="
<p style="text-align:right;font-size:.72rem;margin:0 0 10px 0;"
 `);

 const body = `<nav id="toc" style="background:#0f172a
 rem">
 <h2 style="color:#fff;font-size:1.05rem;font-weight:800;">
 <ol style="margin:0;padding-left:1.1rem;font-size:.85rem;">
</nav>
 ${bodies.join("\n")}`;

 const html = wrap({
 title: "Универсальное пособие по алгоритмам",
 subtitle: `Все темы, ${chapters.length} шт.`,
 css,
 body,
 gloss: glossarySection(allTerms),
 origin: window.location.origin,
 });
 triggerDownload(html, "posobie-vse-temy.html");
 }

```

## РАЗДЕЛ: СКРИПТЫ СБОРКИ И ЭКСПОРТА



```

 return {
 id: c.id,
 title: c.title,
 num: Number.parseInt(c.title.match(/^(\d+)\./)?.[1]),
 analogies,
 hasAnalogy: analogies > 0,
 inlineSvg: /<svg[\s>]/i.test(html),
 image: /<img[\s>]/i.test(html),
 interactive: vizIds.has(c.id),
 chars: html.length,
 };
 });
});

const has2D = (r) => r.interactive || r.inlineSvg || r.image;
const gapBoth = rows.filter((r) => !r.hasAnalogy && !has2D(r));
const gapAnalogy = rows.filter((r) => !r.hasAnalogy && has2D(r));
const gapViz = rows.filter((r) => r.hasAnalogy && !has2D(r));
const orphanViz = [...vizIds].filter((id) => !chapters[id]);

const nums = rows.map((r) => r.num).filter(Boolean);
const missingNums = Array.from({ length: 24 }, (_, i) => i + 1).filter(n => !nums.includes(n));

const flag = (b) => (b ? " " : "-");

if (process.argv.includes("--md")) {
 console.log("| # | Глава | Аналогия | 2D-виз. | SVG |");
 console.log("| --- | --- | --- | --- | --- |");
 for (const r of rows) {
 console.log(
 `| ${r.num || "-"} | ${r.title} | ${r.hasAnalogy ? "да" : "нет"} |`
);
 }
} else {
 console.log(`Глав в пособии: ${rows.length}\n`);
 const dump = (title, list) => {
 console.log(`${title} (${list.length}):`);
 list.forEach((r) => console.log(`  * ${r.title} [${r.num || "-"}]`));
 };
 dump("Главы с аналогиями", gapBoth);
 dump("Главы без аналогий", gapAnalogy);
 dump("Главы без визуализации", gapViz);
 dump("Остаток глав", orphanViz);
}

```

```

 "giu"
);
 const map = new Map(GLOSSARY_LABELS.map((l) => [l.label, l]));

 // те же лимиты, что и в src/hooks/useTermHints.ts
 const MAX_PER_SURFACE = 2;
 const MAX_PER_TERM = 8;

 const bad = [];
 const used = new Set();
 console.log("тем | подсв. | глава");
 for (const c of chapters) {
 const text = c.content
 .replace(/<(script|style|code|pre)[\s\S]*?<\/\1>/gi, "")
 .replace(/<[^>]+>/g, " ");

 const perTerm = new Map();
 const perSurface = new Map();
 let highlights = 0;
 const ids = new Set();

 for (const m of text.matchAll(re)) {
 const surface = m[1].toLowerCase();
 const id = map.get(surface);
 if (!id) continue;
 const ut = perTerm.get(id) ?? 0;
 const us = perSurface.get(surface) ?? 0;
 if (ut >= MAX_PER_TERM || us >= MAX_PER_SURFACE) continue;
 perTerm.set(id, ut + 1);
 perSurface.set(surface, us + 1);
 highlights += 1;
 ids.add(id);
 used.add(id);
 }

 if (ids.size === 0) bad.push(c.title);
 console.log(
 String(ids.size).padStart(3), "|", String(highlights)
);
 }
}

```

```
ensureDir(OUT_DIR);

const doc = new PDFDocument({
 size: [A4.width, A4.height],
 margin: 0,
 autoFirstPage: false,
 info: {
 Title: "Универсальное пособие – полный исходный код",
 Author: "CI: rebuild-pdf",
 Subject: "Экспорт всего кода проекта (код -> txt)",
 CreationDate: new Date(),
 },
});

const stream = fs.createWriteStream(PDF_PATH);
doc.pipe(stream);

for (const file of pages) {
 doc.addPage({ size: [A4.width, A4.height], margin: 0 });
 doc.image(path.join(PAGES_DIR, file), 0, 0, { width: A4.width });
}

doc.end();
await new Promise((resolve, reject) => {
 stream.on("finish", resolve);
 stream.on("error", reject);
});

console.log("[3/3] png -> pdf");
console.log(`    страниц: ${pages.length}`);
console.log(`    выход:   ${path.relative(ROOT, PDF_PATH)}`);
}

main().catch((err) => {
 console.error("Ошибка сборки PDF:", err.message);
 process.exit(1);
});
```

```
[/^src\/utils\/$/, "Утилиты"],
[/^src\/$/, "Ядро приложения"],
[/^scripts\/$/, "Скрипты сборки и экспорта"],
[/^\.github\/$/, "CI / автоматизация"],
[/^[^\/]+$/, "Конфигурация проекта"],
];

const categoryOf = (rel) => CATEGORIES.find(([re]) => re.test(rel));

/** Порядок разделов в документе. */
const ORDER = [
 "Конфигурация проекта",
 "Ядро приложения",
 "Контент и данные пособия",
 "Билеты (учебный контент)",
 "Интерактивные компоненты и симуляторы",
 "Визуализаторы (вложенные)",
 "HTML-разметка (layout)",
 "Утилиты",
 "Скрипты сборки и экспорта",
 "CI / автоматизация",
 "Прочее",
];

function listFiles() {
 let files;
 try {
 files = execFileSync("git", ["ls-files", "--cached"], {
 cwd: ROOT,
 encoding: "utf8",
 })
 .split("\n")
 .filter(Boolean);
 } catch {
 files = [];
 const walk = (dir) => {
 for (const e of fs.readdirSync(dir, { withFileTypes: true })) {
 const abs = path.join(dir, e.name);
 if (e.isDirectory()) {
 walk(abs);
 } else {
 files.push(abs);
 }
 }
 };
 walk(ROOT);
 }
}
```

```

 const nb = parseInt(b.title.match(/(\d+)/)?[0] ??
 return na - nb;
 });
}

function main() {
 const files = listFiles();
 const chapters = extractChapters(files);
 ensureDir(OUT_DIR);

 const out = [];
 const push = (s = "") => out.push(s);

 const stamp = new Date().toLocaleString("ru-RU", { ti
 const totalBytes = files.reduce((s, f) => s + fs.stat

 push(HR);
 push(" УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТ
 push(" ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФА
 push(HR);
 push();
 push(`Дата генерации:                ${stamp}`);
 push(`Файлов исходного кода:         ${files.length}`);
 push(`Суммарный объём кода: ${humanSize(totalByt
 push(`Глав/билетов в пособии:        ${chapters.length}`);
 push();

 push(HR);
 push(" ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕ
 push(HR);
 chapters.forEach((c, i) => {
 push(` ${String(i + 1).padStart(3, " ")} ${c.titl
 });
 push();

 push(HR);
 push(" ОГЛАВЛЕНИЕ: ФАЙЛЫ ИСХОДН
 push(HR);

```

```
const txt = out.join("\n") + "\n";
fs.writeFileSync(TXT_PATH, txt, "utf8");

console.log("[1/3] код -> txt");
console.log(`      файлов:  ${files.length}`);
console.log(`      глав:    ${chapters.length}`);
console.log(`      строк:   ${txt.split("\n").length}`);
console.log(`      выход:   ${path.relative(ROOT, TXT_PATH)}`);
}

main();
```

---

ФАЙЛ 82/85: scripts/export/common.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 66 | РАЗМЕР: 1.5 КБ

---

```
/**
 * Общие настройки и утилиты пайплайна экспорта.
 *
 * код -> full_code_all_pages.txt (collect-code.mjs)
 * txt -> page-XXXX.png (render-pages.mjs)
 * png -> full_code_all_pages.pdf (build-pdf.mjs)
 */
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { execFileSync } from "node:child_process";

export const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)), "..");

/** Куда кладём финальные артефакты (эта папка отдаётся клиенту) */
export const OUT_DIR = path.join(ROOT, "public", "export");

/** Промежуточные PNG-страницы (в .gitignore, в CI уезжают в бакет) */
export const PAGES_DIR = path.join(ROOT, "tmp", "export");
```

```
 } catch {
 /* пробуем следующий */
 }
 }
 throw new Error(
 "ImageMagick не найден. Установите его: sudo apt-get install imagemagick
);
}
```

---

ФАЙЛ 83/85: scripts/export/render-pages.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 117 | РАЗМЕР: 2.5 КБ

---

```
/**
 * ШАГ 2: TXT -> PNG
 *
 * Разбивает full_code_all_pages.txt на страницы формата
 * каждую страницу в PNG через ImageMagick (шрифт DejaVu)
 *
 * Результат: tmp/export-pages/page-0001.png, page-0002.png, ...
 */
import fs from "node:fs";
import path from "node:path";
import os from "node:os";
import { execFile } from "node:child_process";
import { promisify } from "node:util";
import {
 ROOT, PAGES_DIR, TXT_PATH, PAGE, ensureDir, humanSize
} from "../common.mjs";

const execFileAsync = promisify(execFile);

/** Раскрываем табы и жёстко переносим длинные строки,
function wrap(line) {
 const expanded = line.replace(/\t/g, " ").replace(
 /(\s+)$/g, " ";
 if (expanded.length <= PAGE.cols) return [expanded];
 }
}
```

```

 }}стр. ${idx + 1} / ${total}`;
 const body = [header, "-".repeat(PAGE.cols), ...lines];
 const txtFile = path.join(PAGES_DIR, `page-${num}.txt`);
 const pngFile = path.join(PAGES_DIR, `page-${num}.png`);
 fs.writeFileSync(txtFile, body + "\n", "utf8");
 return { txtFile, pngFile, num };
 });

```

```

const args = (job) => [
 "-density", String(PAGE.density),
 "-page", PAGE.size,
 "-font", fs.existsSync(PAGE.fontFile) ? PAGE.fontFile : "",
 "-pointsize", String(PAGE.pointsize),
 `text:${job.txtFile}`,
 "-background", "white",
 "-flatten",
 "-colorspace", "Gray",
 ...(PAGE.monochrome ? ["-monochrome"] : []),
 "-strip",
 "-define", "png:compression-level=9",
 job.pngFile,
];

```

```

const concurrency = Math.max(2, Math.min(os.cpus().length, 4));
let done = 0;
let cursor = 0;

```

```

async function worker() {
 while (cursor < jobs.length) {
 const job = jobs[cursor++];
 await execFileAsync(im, args(job));
 fs.rmSync(job.txtFile, { force: true });
 done += 1;
 if (done % 25 === 0 || done === total) {
 process.stdout.write(`отрендерено ${done} страниц\n`);
 }
 }
}

```



```
on:
 push:
 branches:
 - main
 # Позволяет проверить Pages прямо из рабочей ветки
 - "arena/**"
 workflow_dispatch:

permissions:
 contents: read
 pages: write
 id-token: write

concurrency:
 group: pages
 cancel-in-progress: true

jobs:
 build:
 name: Build static site
 runs-on: ubuntu-latest
 steps:
 - name: Checkout
 uses: actions/checkout@v6

 - name: Setup Node.js
 uses: actions/setup-node@v7
 with:
 node-version: 24
 cache: npm

 - name: Install dependencies
 run: npm ci --no-audit --no-fund

 - name: Build for the repository subpath
 env:
 VITE_BASE: /algv0/
```



- run: npm run export:txt
- name: Step 2 – txt → png  
env:  
EXPORT\_DENSITY: \${github.event.inputs.density}  
run: npm run export:png
- name: Step 3 – png → pdf  
run: npm run export:pdf
- name: Type-check приложения  
run: npx tsc --noEmit  
continue-on-error: true
- name: Summary  
run: |  
{  
echo "### Экспорт пересобран"  
echo ""  
echo "| Артефакт | Размер |"  
echo "| --- | --- |"  
echo "| \`public/export/full\_code\_all\_pages`"  
echo "| \`public/export/full\_code\_all\_pages`"  
echo "| PNG-страниц | \$(ls tmp/export-pages.  
} >> "\$GITHUB\_STEP\_SUMMARY"
- name: Upload artifacts (PDF + TXT)  
uses: actions/upload-artifact@v7  
with:  
name: full-code-export  
path: |  
public/export/full\_code\_all\_pages.pdf  
public/export/full\_code\_all\_pages.txt  
if-no-files-found: error  
retention-days: 30
- name: Upload artifacts (PNG-страницы)  
uses: actions/upload-artifact@v7